



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Introduction

Team DDCO

Department of Computer Science and
Engineering

Digital Design And Computer Organisation

Faculty Team:

RR Campus:

Dr.Chaitra N
Prof. Jayashree S
Prof.Rajeshwari CN
Prof.Shwetha Mugalihal
Dr.Suresh Babu Doarswamy
Dr.Chetana Srinivas
Ms. Sonali Kalekar K
Prof.Vivek Kashyap
Prof.Bharathi DS
Prof.Arpitha K

EC Campus:

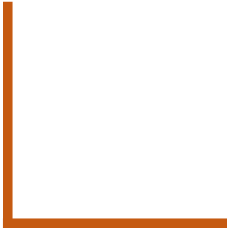
Dr. Prajwala T R
Prof. Deepti C
Prof. Swati Priya
Prof. Swetha Patil
Dr. Monika
Prof. Lenish



Digital Design And Computer Organisation

Course Code :UE24CS251A

Prerequisites :Basic Electronics, Programming in C



Course Objective

The objectives of this course are to provide a sound understanding of:

- Fundamental (combinational and sequential) building blocks of digital logic circuits.
- Design of more complex logic circuits such as adders, multipliers and register files.
- Design of Finite State Machines based on problem specification.
- Construction, using above logic circuits, of a microprocessor, and its functioning at the clock cycle level.
- **Tools to implement: iVerilog**

Unit 1: Gate-Level Minimization and Combinational logic-1

Introduction, The map method Four variable K-map, Product of Sums simplification, Don't Care conditions, NAND and NOR implementation, Combinational circuits, Analysis procedure Design Procedure, Combinational logic-1: Binary Combinational logic: Adder- Subtractor, Decimal Adder, Binary multiplier, Magnitude comparator Decoders Encoders, Multiplexers.

14 Hours

Unit 2: Synchronous Sequential Logic-I

Synchronous Sequential Logic: Introduction, Sequential circuits, Storage elements: Latches, Flip flops, Analysis of clocked sequential circuits, State reduction and assignment, Design procedure Registers and counters: Registers, Shift register, Ripple counters, Synchronous counters Other counters.

14 Hours

Unit 3: Basic structure of computers, Standard I/O interface, Interrupts, Memory System

Computer Types, Functional Units: Input Unit, Memory Unit, ALU, Output Unit, Control Unit, Basic operational concepts, Number representation and arithmetic Operations, Character representation, Memory locations and addresses, Memory Operations, Instruction and instruction sequencing , Addressing modes, Assembly Languages, Accessing I/O Devices, Interrupts, Standard I/O Interfaces. Semiconductor RAM memories.

14 Hours

Unit 4: Arithmetic Processing Unit and Control Unit Design Arithmetic:

Multiplication of Positive numbers, Signed operand Multiplication, Fast multiplication, Integer division, floating point numbers operation and Architecture, Some fundamental concepts, Execution of a complete instruction, Multiple Bus Organization, Hardwired control ,Micro programmed control

14 Hours

DIGITAL DESIGN AND COMPUTER ORGANIZATION

Syllabus



Textbook(s):

“Digital Design”, M Morris Mano, Michael D Ciletti, Pearson, 5th Edition, 2012.

“Computer Organization”, Carl Hamacher, Zvonko Vranesic, Safwat Zaky, McGraw Hill, 5th Edition, 2002

References:

Digital Design & Computer Architecture, David Money Harris, Sarah L. Harris, 2nd Edition, Elsevier, 2013

Computer Organization and Design, David A. Patterson, John L. Hennessey 5th Edition, Elsevier, 2013

DIGITAL DESIGN AND COMPUTER ORGANIZATION

Evaluation Policy

	Marks to be conducted for	Marks to be scaled to
ISA 1 (Units 1 and 2)	40	20
ISA 2 (Units 3 and 4)	40	20
Banana Problem	4 Marks- It is conducted for 20 marks, each unit banana problem for 5 marks ($5 \times 4 = 20$ Marks) Reduced to 4 marks	10
Orange Problem	6 Marks (3 marks for Unit 1, 2 and 3 marks for Unit 3,4). Orange problem 1 conducted for 10 marks and Orange Problem 2 conducted for 10 marks ($10 \times 2 = 20$ Marks) this is reduced to 6 Marks.	
Laboratory (1–credit)	• 10 marks Mini Project component • 5 marks Quiz conducted for 20 marks, (half an hour) reduced to 5 marks (conducted during 14th week) • 5 Marks continuous evaluation of lab programs	20
ESA	100	50
Total		120*
*The total of 120 marks will further be scaled down to 100 for grading.		

What is Engineering?

Engineering

- From latin **ingenium**: innate *talent/capacity/intelligence*
- To design and build structures and machines (with *skill/art/expertise/ingenuity*)

Objective of Engineering?

- Optimize fundamental physical quantities of *time, space and energy*
- In current course, *increase logic circuit speed, decrease logic resources required and decrease power consumed*

Basic Electronics- the unit 3 of Basic electronics

- Digital Electronics: Basic gates(review), Boolean Algebra, Boolean laws and theorems, Simplification of Boolean expressions, Universal gates – NAND and NOR- This is the basics for solving K maps and representing them in circuits in Unit 1 of DDCO.
- Arithmetic building blocks – Half and Full Adder, Multiplexers, Demultiplexers, RS Flip-Flop – Basic idea, NAND Gate latch, Clocked RS and D Flip-Flops. – This is used in sequential Logic design of circuits in unit2 of DDCO
- Using this knowledge the data path and control path of ALU is designed
- Basics of C programming is desired knowledge for Lab-The Verilog language used in created of simple circuits like adders, multiplexers decoders and design of ALU has syntax similar to the C programming language.

DIGITAL DESIGN AND COMPUTER ORGANIZATION

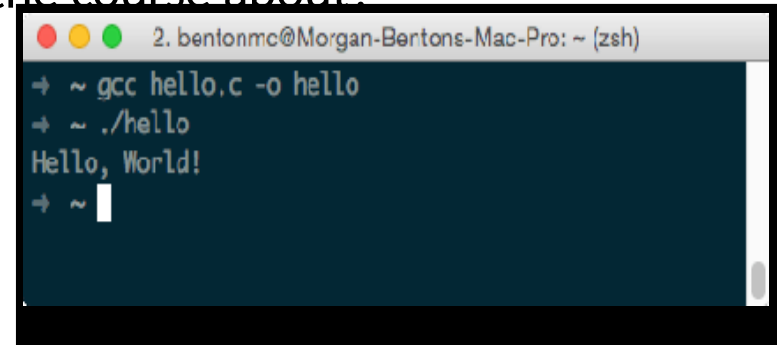
What is the course about?

■ Digital Design and Computer Organization: What is the course about?

You have learnt programming in C

Compile hello_world.c

Running program outputs “Hello World!”



```
2. bentonmc@Morgan-Bertons-Mac-Pro: ~ (zsh)
→ ~ gcc hello.c -o hello
→ ~ ./hello
Hello, World!
→ ~
```

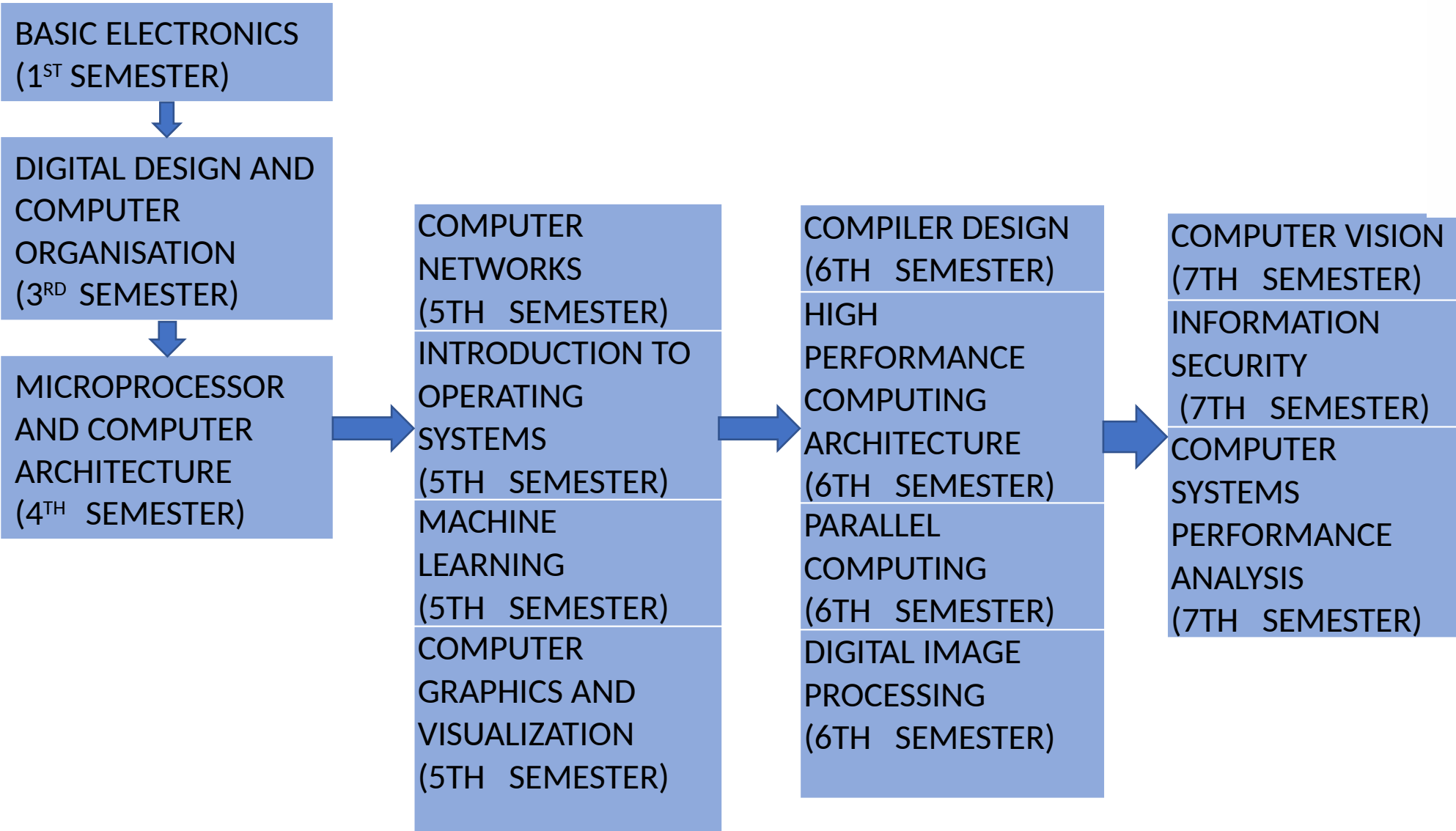
From starting a program to the time it displays output What goes on inside your computer?

That in a nutshell is what DDCO is about

Design, organization and operation of various components in your computer at different levels of abstractions

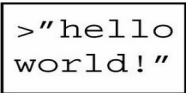
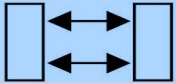
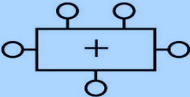
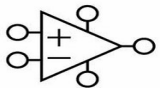
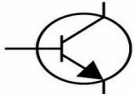
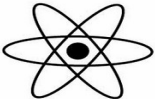
DIGITAL DESIGN AND COMPUTER ORGANIZATION

Course Flow



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Computer Organization & Computer Architecture

Application software	
Operating systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

DIGITAL DESIGN AND COMPUTER ORGANIZATION

Computer Organization & Computer Architecture

Why Digital Design and Computer Organization?

Foundational to Computing Systems

Forms the basis of understanding how computers work — from gates to systems.
Essential to bridge hardware and software design.

Hardware Understanding Aids Better Software Design

Enables efficient coding by understanding memory, cache, instruction cycles.
Crucial for embedded systems, real-time systems, and OS development.
Drives performance-aware programming (e.g., low-latency apps, game engines).

Write Smarter, Faster Software

Good software is not just about code — it's about knowing the machine you're coding for.
Learn how memory, CPU, and I/O work → write efficient code.
Understand why some programs are faster than others.

DIGITAL DESIGN AND COMPUTER ORGANIZATION

Computer Organization & Computer Architecture

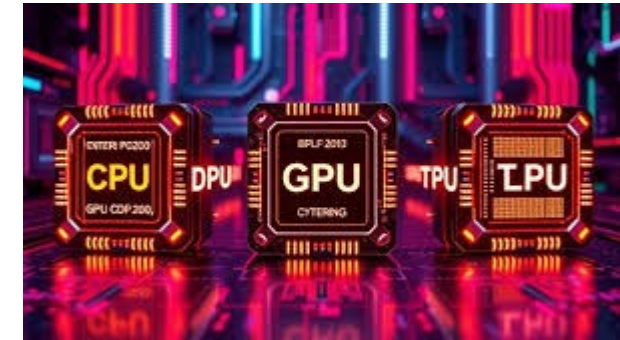
Why Digital Design and Computer Organization?

CPU Alone is Not Enough Anymore

Modern computing = GPU + TPU + custom hardware.

Needed for Machine Learning, Gaming, AR/VR, Data Centers.

helps you understand why and how these accelerators work.



Understand the Cloud & Networking Backbones

Ever wondered how YouTube streams, or ML runs on the cloud?

Concepts like pipelining, caching, and buses matter in networking, cloud, and IoT.

Helps in designing scalable, efficient systems.

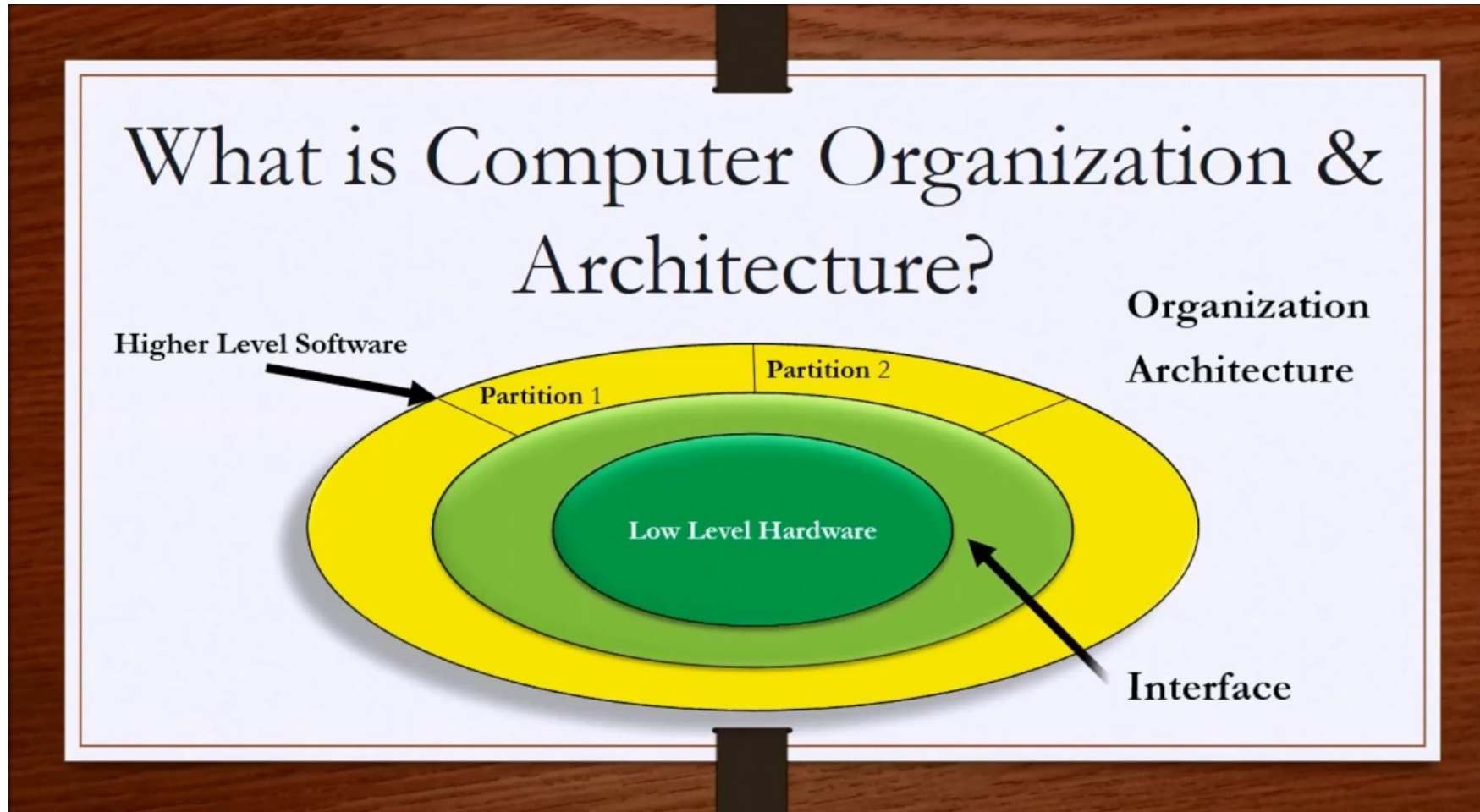
What Top Tech Leaders Say

"To be a good programmer, understand how computers work." – Steve Jobs

"Programming without understanding hardware is like flying blind." – Linus Torvalds

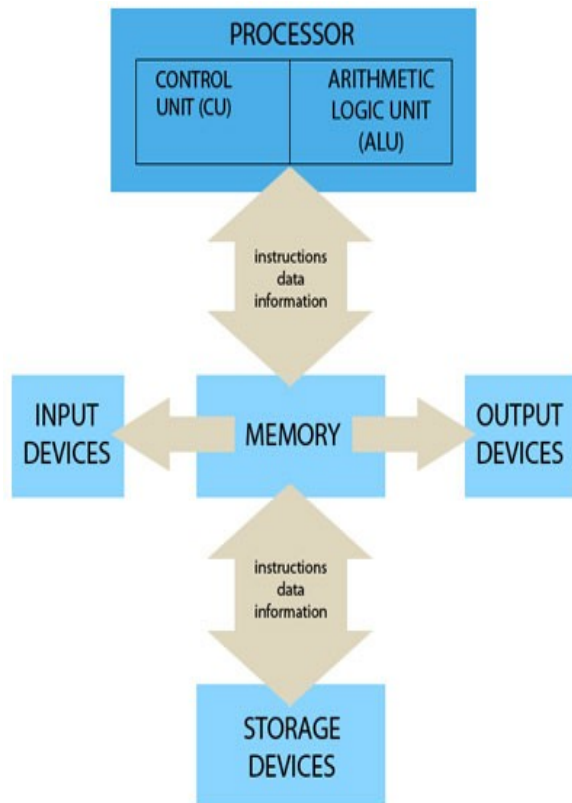


What is Computer Organization ?



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Computer Organization & Computer Architecture



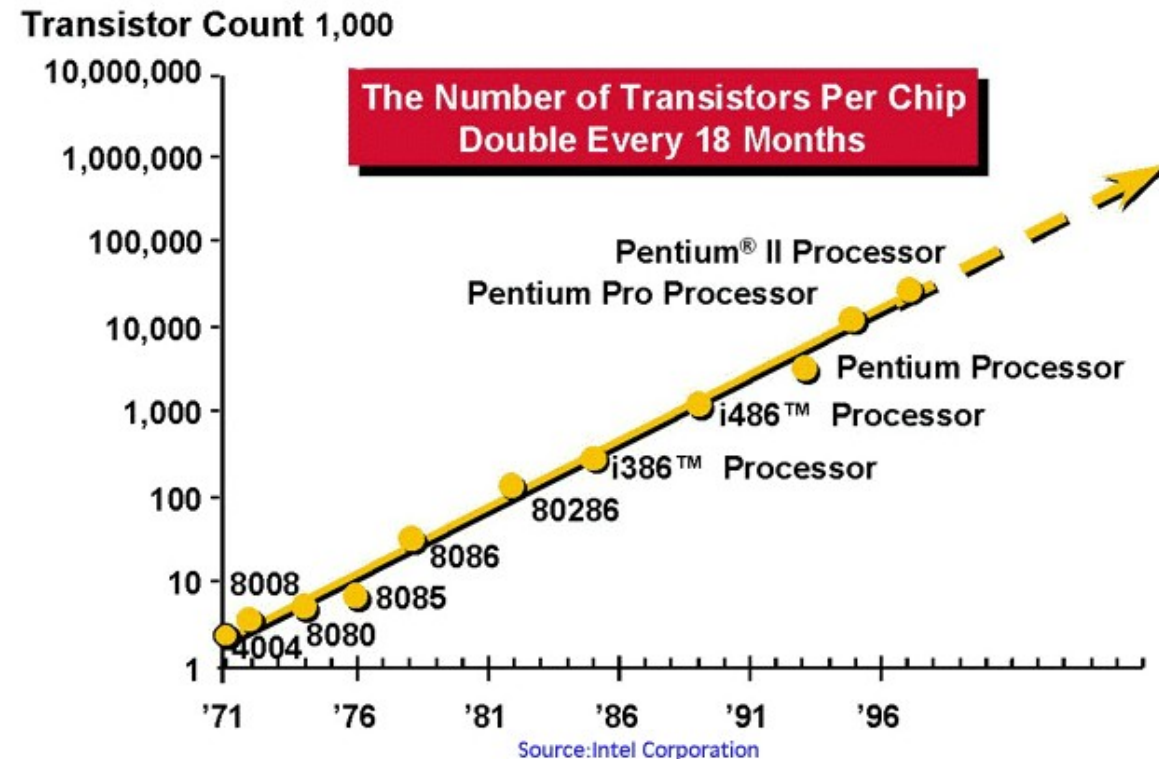
- Computer Organization refers to the **Operational Units and their interconnections** that realize or recognize the specifications of Computer Architecture.
- Organizational **attributes** includes **Hardware details transparent** to the programmer such as **control signals** , **Interface** between computers and peripherals and the memory technology used.

Moore's Law

Moore's Law Every 18 months or so:

- Number of transistors (per unit chip area) doubles
- Transistor speed doubles
- Transistor power consumption halves

Moore's Law is slowing down:
Different computer companies show
that the term is not very popular but the
rule is still



For Example:

- Moore's Law means ever-more powerful personal computers for less and less money.
- A computer chip that contained 2,000 transistors and cost \$1,000 in 1970, \$500 in 1972, \$250 in 1974, and \$0.97 in 1990 costs less than \$0.02 to manufacture today.
- Moore's Law effectively means that approximately every two years personal computers and other electronic devices can do twice as many new, innovative, and unexpected things than before

Why Moore's Law is slowing down?

- Moore's Law predicted that the number of transistors on a chip would double every two years, leading to faster and cheaper computers.
- But today, we are reaching the **physical limits** of how small we can make transistors.
- As components get smaller and packed more tightly, they **produce more heat**, which can **damage or melt the chip**.
- Also, shrinking them further becomes extremely **expensive and complex**.
- So, **we can't keep doubling performance just by making things smaller anymore** — and that's why Moore's Law is slowing down.

DIGITAL DESIGN AND COMPUTER ORGANIZATION

Job Description

Position	Company	Job Profile	Job Requirement
CPU Design Engineer - Hardware	Nvidia Corporation 	Design and implementation of modules. Come up with micro-architecture, implement in RTL, and deliver a fully verified, synthesis/timing clean design	Good understanding of ASIC design flow ,logic synthesis.
Logic Design Engineer	Intel Bengaluru, Karnataka 	Will be responsible for design and development of Graphics, Media and Display IPs as well as discrete Graphics SoCs (GPUs), targeting both Client Device and Datacenter markets.	Good knowledge of digital logic design IP/ SoC architecture and microarchitecture.

Job Description 1

Job Description

Software Developer for Microprocessor based embedded system



BOSCH

- Perform Board Bring-up on ARM/Intel SoC architecture.
- Reverse engineer HW-SW issues by referring schematics & data sheet.
- Realize digital design on Intel/Xilinx FPGA.
- Optimize device drivers (Ethernet, PCIe, DMA, CAN) taking into consideration security aspects.
- Migrate software components (Device Drivers & Middleware) from Linux to QNX.
- Design systems using SoC, PCIe Switch, FPGA & SoC's.
- Design software architecture using state of the art tools (like Rapsody).
- Port open source software & benchmark performance on development kits.
- Positively influence Bosch position by contributing to open source software.
- Create quick demonstrators/PoC & represent Bosch offerings in trade shows

Qualifications

- Qualification: B.E./B.Tech./ M.E./ M.Tech. – Electronics & Communication or Computer Science
- Work Experience: 1 to 4 years

Technical Know-how:

- Background experience in using Digital Storage Oscilloscope.
- Awareness of Intel and ARM SoC architecture from secure boot point of view.
- Awareness of ARM Instruction Set Architecture & Vectorization techniques.
- Practical experience in real-time Linux/ RTOS based embedded software development using C & C++.
- Awareness of latest technical happenings in open source software (OSADL, Linux mainline, Autoware, Apollo, Adaptive AUTOSAR).
- Practical experience in debugging software for embedded systems.
- Practical overview (via. academic literatures) of technology trends in SoC, Ethernet Switch, PCIe & FPGAs.

Job Description 2

System Software Engineer, GPU Tools Development

NVIDIA is searching for a highly motivated, creative engineer with experience in system software to join the GPU Software team. As someone who is hardworking and passionate about their work, you will design key aspects of our production GPU kernel drivers and embedded Software. You should demonstrate the ability to excel in an environment with complex software and hardware designs.



What you'll be doing:

- Define, design, develop and verify features for our GPUs; collaborating with hardware engineers and fellow software engineers
- You will follow the devices all the way through the development process to the customer desktops, notebooks, workstations, and gaming console products that are used throughout the world
- Heavily involved with the early modeling and simulation required to produce our world-class products
- Multiple opportunities to collaborate and communicate effectively with teams from all around the globe

What we need to see:

- BS or MS degree in Computer Engineering, Computer Science, or related degree
- Strong C programming skills as well as having shown initiative in pursuing independent coding projects
- Familiarity with computer system architecture, microprocessor, and microcontroller fundamentals (caches, buses, memory controllers, DMA, etc.)
- Kernel experience with Linux, Android, Chrome, or Windows systems
- 2+ years of meaningful software development experience

Ways to stand out from the crowd:

- Background and strength with complex system-level debugging is invaluable
- Deep understanding of memory management and virtualization concepts
- Familiarity with kernel level security concepts
- Experience with embedded system SW concepts, e.g.: RTOS and overlay programming models

Job Description 3

Role Description (Role & Responsibilities)

1. **Mandatory skills 16bit 32bit Microcontroller Microprocessor**
2. Embedded systems Firmware Device driver development experience Programming Strong in C
3. Communication protocols UART CAN SPI Ethernet Modbus TCP IP IDE usage
4. Code composer studio IAR workbench Code warrior RTOS VxWorks FreeRTOS Ti RTOS Bootloader
5. Multi threading concepts Preferred skills Strong in Cplusplus Assembly language experience



Capgemini
**Firmware
Developer**

Mobiveil: CPU Processor Design

Job Summary:

Bachelors or Masters degree in **Computer Science** or Electrical/Computer Engineering.

Understanding of general purpose CPU micro architecture, including knowledge of areas such as processor pipelines, caches, memory hierarchy, and multi-processor systems.

Knowledge and or Experience in RTL Design hardware development using Verilog, ideally block design in a CPU design project or similar high performance project.

Understanding of CPU instruction set architecture and assembly language.

Familiarity with ARM architecture and micro-architecture for current ARM CPU cores is helpful but not required.

Software development skills and/ or experience is helpful (C/C++, Python/Perl, Shell scripting)

Experience modelling microprocessors using higher-level languages, like C/C++, is helpful but not required

Effective communication skills and the ability to collaborate with a team



Digital Design

Combinational logic design

Sequential logic design

Computer Organization

Architecture (microprocessor instruction set)

Microarchitecture (microprocessor operation)



THANK YOU

Team DDCO

Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Boolean Algebra and Boolean Function

Team DDCO

Department of Computer Science and Engineering

Mathematical function: defines a relationship between an independent variable and a dependent variable. E.g. $A = \pi r^2$

- Example: Parabola
- Domain and range are set of real numbers
- Specified on Cartesian Plane

Boolean function: A mathematical function whose arguments, as well as the function itself, assume values from a two-element set (usually $\{0,1\}$).

Boolean algebra, is a mathematical system, defined with a set of elements, a set of operators, and a number of unproved axioms or postulates.

A **Boolean function** is a function whose domain and range are the set $\{0, 1\}$.

A Boolean function has:

- At least one Boolean variable
- At least one Boolean operator
- At least one input from the set $\{0, 1\}$

It produces an output that is also a member of the set $\{0, 1\}$

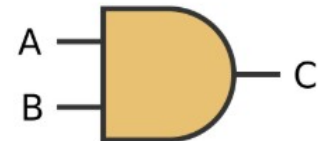
Boolean Constants and Variables

- Inputs and outputs of Boolean function are from the set $\{0, 1\}$
 - **0 and 1 are called Boolean constants**
- In general, inputs and outputs of mathematical functions are represented by variables (like $y = x^2$)
 - **Inputs and outputs of Boolean functions are called as Boolean variables (like a, b and y)**

Specified as a truth table:

A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

Specified as a logic gate:



Basic Logic Gates

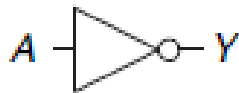
Logic gates are **physical implementations of Boolean functions**.

A logic gate is a hardware component that performs a specific Boolean function.

Single Input Logic gates

The NOT gate's output is the inverse of its input

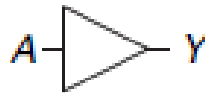
NOT



$$Y = \bar{A}$$

A	Y
0	1
1	0

BUF



$$Y = A$$

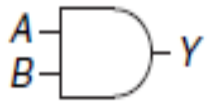
A	Y
0	0
1	1

Buffer gate simply copies the input to the output.

Basic Logic Gates

Two-Input Logic Gates

AND



$$Y = AB$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR



$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

XOR



$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

NAND



$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

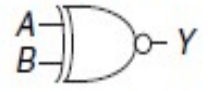
NOR



$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XNOR

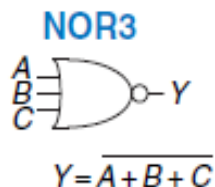


$$Y = \overline{A \oplus B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

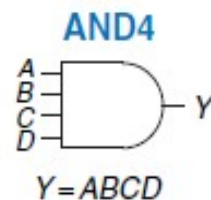
Basic Logic Gates

Multiple Input Logic Gates



A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Figure 1.20 Three-input NOR truth table

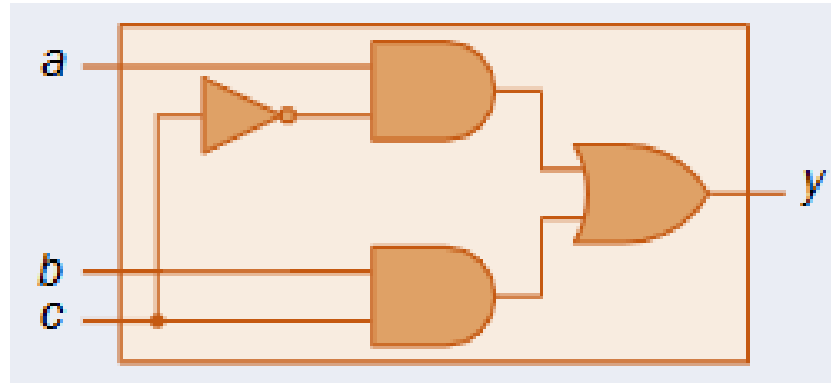


A	C	B	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

Figure 1.22 Four-input AND truth table

What is a logic circuit?

Multiple logic gates combined together, with the output of one gate being connected to the input of another, form a **logic circuit**.



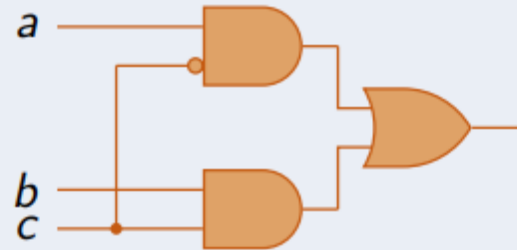
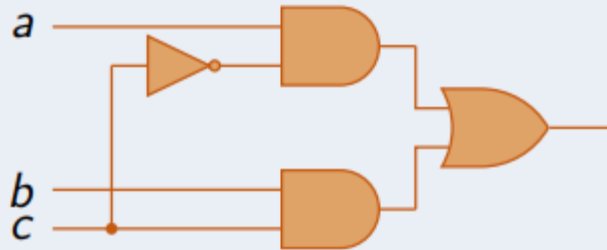
A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Logic Gates with Inverted Inputs

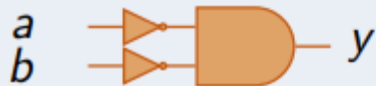
Bubble

If a logic gate has a NOT gate on an input, the NOT gate can be replaced by a **bubble** on that input

Bubble Example



Bubbled AND Example



Boolean Algebra(T1-Section 2.4)



Algebra

In mathematics, an Algebra is composed of four things: a set of elements, operations on those elements, identity elements and laws/identities

Standard Algebra

- **Set** Real numbers
- **Operations** Add, subtract, multiply, divide
- **Identity elements** 0 (for add), 1 (for multiply)
- **Laws/Identities** Commutative, associative, distributive, . . .

Boolean Algebra

- **Set** $\{0, 1\}$
- **Operations** AND, OR, NOT
- **Identity elements:** 1 (for AND), 0 (for OR)
- **Laws/Identities** Commutative, associative, distributive, . . .

Boolean Laws

Table 2.1
Postulates and Theorems of Boolean Algebra

Postulate 2	(a)	$x + 0 = x$	(b)	$x \cdot 1 = x$
Postulate 5	(a)	$x + x' = 1$	(b)	$x \cdot x' = 0$
Theorem 1	(a)	$x + x = x$	(b)	$x \cdot x = x$
Theorem 2	(a)	$x + 1 = 1$	(b)	$x \cdot 0 = 0$
Theorem 3, involution		$(x')' = x$		
Postulate 3, commutative	(a)	$x + y = y + x$	(b)	$xy = yx$
Theorem 4, associative	(a)	$x + (y + z) = (x + y) + z$	(b)	$x(yz) = (xy)z$
Postulate 4, distributive	(a)	$x(y + z) = xy + xz$	(b)	$x + yz = (x + y)(x + z)$
Theorem 5, DeMorgan	(a)	$(x + y)' = x'y'$	(b)	$(xy)' = x' + y'$
Theorem 6, absorption	(a)	$x + xy = x$	(b)	$x(x + y) = x$

Useful Identity $a + \bar{a} \cdot b = a + b$ $a \cdot (\bar{a} + b) = a \cdot b$

Boolean Laws

THEOREM 1(a): $x + x = x$.

Statement	Justification
$x + x = (x + x) \cdot 1$	postulate 2(b)
$= (x + x)(x + x')$	5(a)
$= x + xx'$	4(b)
$= x + 0$	5(b)
$= x$	2(a)

THEOREM 1(b): $x \cdot x = x$.

Statement	Justification
$x \cdot x = xx + 0$	postulate 2(a)
$= xx + xx'$	5(b)
$= x(x + x')$	4(a)
$= x \cdot 1$	5(a)
$= x$	2(b)

Boolean Laws

THEOREM 2(a): $x + 1 = 1$.

Statement	Justification
$x + 1 = 1 \cdot (x + 1)$	postulate 2(b)
$= (x + x')(x + 1)$	5(a)
$= x + x' \cdot 1$	4(b)
$= x + x'$	2(b)
$= 1$	5(a)

THEOREM 2(b): $x \cdot 0 = 0$.

Boolean Laws

Navigation icons: back, forward, search, etc.

THEOREM 6(a): $x + xy = x$.

Statement	Justification
$x + xy = x \cdot 1 + xy$	postulate 2(b)
$= x(1 + y)$	4(a)
$= x(y + 1)$	3(a)
$= x \cdot 1$	2(a)
$= x$	2(b)

Operator Precedence

The operator precedence for evaluating Boolean expressions is

(1) parentheses,

(2) NOT,

(3) AND, and

(4) OR.

In other words, expressions inside parentheses must be evaluated before all other operations. The next operation that holds precedence is the complement, and then follows the AND and, finally, the OR.

Boolean Functions (T1- section 2.5)



Boolean function described by an **algebraic expression** consists of binary variables, the **constants 0 and 1**, and the logic operation symbols

A Boolean function expresses the **logical relationship** between binary variables and is evaluated by determining the binary value of the expression for all possible values of the variables.

EX: $F_1 = x + y'z$

Value of F_1 if $x = 1$ or $y = 0$ and $z = 1$?

Boolean Functions

A Boolean function can be transformed from an algebraic expression into a **circuit diagram** composed of logic gates connected in a particular structure.

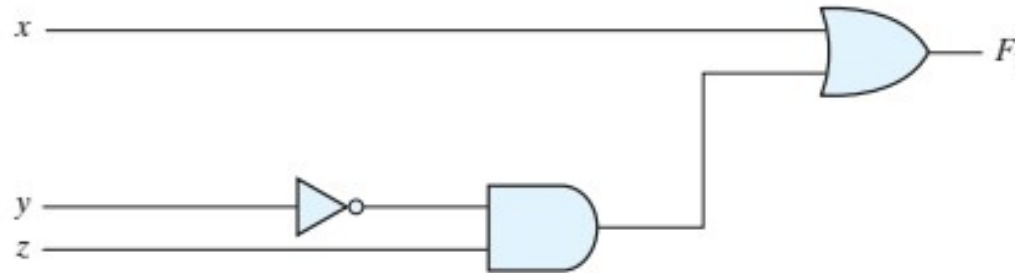


FIGURE 2.1

Gate implementation of $F_1 = x + y'z$

Boolean Functions

For N = Number of variables in the function:

- There exists 2^N possible input combinations
- This results in 2^N rows in the truth table
- Binary numbers counting from 0 through $2^N - 1$
- So, there are 2^{2^N} different truth tables for Boolean functions of N variables.
- 2^{2^N} different Boolean functions

x	y	z	F_1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Boolean Functions



There is only one way that a Boolean function can be represented in a truth table. However, when the function is in algebraic form, it can be expressed in a variety of ways, all of which have equivalent logic.

Boolean algebra, it is sometimes possible to obtain a simpler expression for the same function and thus reduce the number of gates in the circuit and the number of inputs to the gate.

Designers are motivated to reduce the complexity and number of gates because their effort can significantly reduce the cost of a circuit.

Boolean Functions



What is simplest form of below equation?

$$F_2 = x'y'z + x'yz + xy'$$

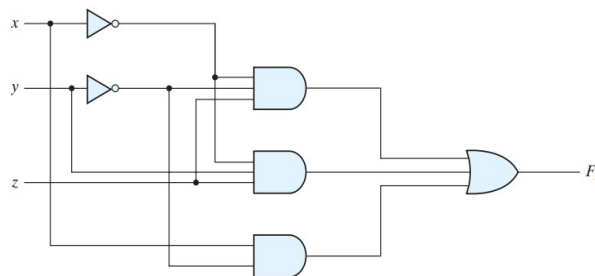
Boolean Functions

What is simplest form of below equation?

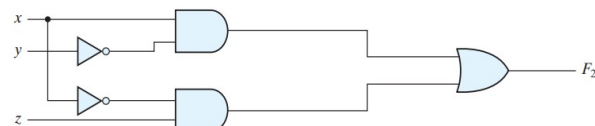
$$F_2 = x'y'z + x'yz + xy'$$

$$F_2 = x'y'z + x'yz + xy'$$

$$F_2 = x'y'z + x'yz + xy' = x'z(y' + y) + xy' = x'z + xy'$$



(a) $F_2 = x'y'z + x'yz + xy'$



(b) $F_2 = xy' + x'z$

FIGURE 2.2
Implementation of Boolean function F_2 with gates

Questions

Simplify the following Boolean expressions to a minimum number of literals:

1. $x(x' + y)$
2. $x + x'y$
3. $(x + y)(x + y')$
4. $xy + x'z + yz$
5. $(x + y)(x' + z)(y + z)$

Questions

Simplify the following Boolean functions to a minimum number of literals.

1. $x(x' + y) = xx' + xy = 0 + xy = xy.$

2. $x + x'y = (x + x')(x + y) = 1(x + y) = x + y.$

3. $(x + y)(x + y') = x + xy + xy' + yy' = x(1 + y + y') = x.$

4.
$$\begin{aligned} xy + x'z + yz &= xy + x'z + yz(x + x') \\ &= xy + x'z + xyz + x'yz \\ &= xy(1 + z) + x'z(1 + y) \\ &= xy + x'z. \end{aligned}$$

5. $(x + y)(x' + z)(y + z) = (x + y)(x' + z),$ by duality from function 4.

(4) and (5) are together known as consensus theorem.

Complement of a Function

The complement of a function F is F' and is obtained from an interchange of 0's for 1's and 1's for 0's in the value of F . The complement of a function may be derived algebraically through DeMorgan's theorems.

$$\begin{aligned}(A + B + C)' &= (A + x)' && \text{let } B + C = x \\ &= A'x' && \text{by theorem 5(a) (DeMorgan)} \\ &= A'(B + C)' && \text{substitute } B + C = x \\ &= A'(B'C') && \text{by theorem 5(a) (DeMorgan)} \\ &= A'B'C' && \text{by theorem 4(b) (associative)}\end{aligned}$$

Questions



Find the complement of the functions $F_1 = x'yz' + x'y'z$ and $F_2 = x(y'z' + yz)$.

Questions

Find the complement of the functions $F_1 = x'yz' + x'y'z$ and $F_2 = x(y'z' + yz)$.

Find the complement of the functions $F_1 = x'yz' + x'y'z$ and $F_2 = x(y'z' + yz)$. By applying DeMorgan's theorems as many times as necessary, the complements are obtained as follows:

$$F_1' = (x'yz' + x'y'z)' = (x'yz')'(x'y'z)' = (x + y' + z)(x + y + z')$$

$$\begin{aligned} F_2' &= [x(y'z' + yz)]' = x' + (y'z' + yz)' = x' + (y'z')'(yz)' \\ &= x' + (y + z)(y' + z') \\ &= x' + yz' + y'z \end{aligned}$$



A simpler procedure for deriving the complement of a function is to take the dual of the function and complement each literal. This method follows from the generalized forms of DeMorgan's theorems. Remember that the dual of a function is obtained from the interchange of AND and OR operators and 1's and 0's.

Questions



Find the complement of the functions $F1 = x'yz' + x'y'z$ and $F2 = x(y'z' + yz)$ taking their duals and complementing each literal.

Questions

Find the complement of the functions $F1 = x'yz' + x'y'z$ and $F2 = x(y'z' + yz)$ taking their duals and complementing each literal.

1. $F_1 = x'yz' + x'y'z.$

The dual of F_1 is $(x' + y + z')(x' + y' + z).$

Complement each literal: $(x + y' + z)(x + y + z') = F'_1.$

2. $F_2 = x(y'z' + yz).$

The dual of F_2 is $x + (y' + z')(y + z).$

Complement each literal: $x' + (y + z)(y' + z') = F'_2.$

Boolean Formula

Each Boolean formula means a Boolean function as well as a logic circuit.

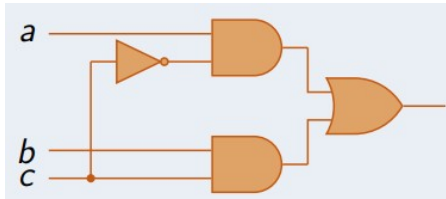
Syntax Rules for Boolean Formulas

1. A Boolean constant (0 or 1) is a Boolean Formula
2. A Boolean variable (say x) is a Boolean formula
3. If P and Q are Boolean formulas then so are:
 - a. $(P \cdot Q)$
 - b. $(P + Q)$
 - c. P'

Example of Boolean Formula: $((a \cdot c' + (b \cdot c))$

- From rule 2, Boolean variable a is a Boolean formula
- From rule 2, Boolean variable b is a Boolean formula
- From rule 2, Boolean variable c' is a Boolean formula
- From rule 3c and step (iii) above c' , is a Boolean formula
- From rule 3a, and steps (i) and (iv) above, $(a \cdot c')$ is a Boolean formula
- From rule 3a, and steps (ii) and (iii) above, $(b \cdot c)$ is a Boolean formula
- From rule 3b, and steps (v) and (vi) above,
- $((a \cdot c') + (b \cdot c))$ is a Boolean formula

The Boolean formula can be converted into a combinational logic circuit:



From Truth Table to Boolean Formula and its Minimization

- Given a combinational logic circuit or Boolean formula, we have learnt to construct its truth table
- **But, given a truth table, how to construct a Boolean formula (or combinational logic circuit) for it?**
- **Also, as there are multiple Boolean formulas / logic circuits for each truth table, how to pick the minimal one?**
- Above problem is called **logic minimization**
 - many metrics: smallest, fastest, least power consumption
 - our metric: smallest two level Sum of Products formula
 - may be more than one solution

Questions

- *Consider Boolean functions of four inputs. What is the number of rows in the truth table of a four input Boolean function?*
- *What is the total number of Boolean functions of four inputs? Specify your answer as an integer.*

Questions

- *Consider Boolean functions of four inputs. What is the number of rows in the truth table of a four input Boolean function?*

A four-input Boolean function's truth table will have 16 rows. (2^4)

- *What is the total number of Boolean functions of four inputs? Specify your answer as an integer.*

There are 2^{2^N} different Boolean functions on n Boolean variables

Here = 65536

Questions

- *Is a logic gate. . .*
 - *A Boolean function? or*
 - *A digital electronic circuit?*

Questions

- *Is a logic gate. . .*
 - *A Boolean function? or*
 - *A digital electronic circuit?*

It is both

This wonderful fact enables us to create the machines that perform mathematics, which we call computers.

Questions

- *There are 16 different truth tables for Boolean functions of two variables. List each truth table. Give each one a short descriptive name (such as OR, NAND, and so on).*

A	B	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇	Y ₈	Y ₉	Y _A	Y _B	Y _C	Y _D	Y _E	Y _F
0	0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
0	1	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	0	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

Questions

- *There are 16 different truth tables for Boolean functions of two variables. List each truth table. Give each one a short descriptive name (such as OR, NAND, and so on).*

A	B	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇	Y ₈	Y ₉	Y _A	Y _B	Y _C	Y _D	Y _E	Y _F
0	0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
0	1	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	0	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
		Zeros	NOR	A' B	NOT A	AB'	NOT B	XOR	NAND	AND	XNOR	B	A' +B	A	A+B'	OR	Ones

Questions

- simplify the Boolean expression:

$$x(x' + y)$$

- A. 1
- B. $x + y$
- C. $x \cdot y$
- D. y

Questions

- simplify the Boolean expression:

$$x(x' + y)$$

- A. 1
- B. $x + y$
- C. $x \cdot y$
- D. y

Ans: C



THANK YOU

Team DDCO

Department of Computer Science and Engineering



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Combinational logic

Department of Computer Science and Engineering

DIGITAL DESIGN AND COMPUTER

ORGANIZATION canonical and Standard forms

Department of Computer Science and Engineering

From Truth Table to Boolean Formula and its Minimization

- Given a combinational logic circuit or Boolean formula, we have learnt to construct its truth table
 - But, given a truth table, how to construct a Boolean formula (or combinational logic circuit) for it?
 - Also, as there are multiple Boolean formulas / logic circuits for each truth table, how to pick the minimal one?
- Above problem is called **logic minimization**
????

From Truth Table to Boolean Formula and its Minimization

- Given a combinational logic circuit or Boolean formula, we have learnt to construct its truth table
 - But, given a truth table, how to construct a Boolean formula (or combinational logic circuit) for it?
 - Also, as there are multiple Boolean formulas / logic circuits for each truth table, how to pick the minimal one?
- Above problem is called **logic minimization**
- › many metrics: smallest, fastest, least power consumption
 - › our metric: smallest two level Sum of Products formula
 - › may be more than one solution

canonical and Standard forms

Canonical and Standard form (T1: section 2.6)

- A binary variable may appear either in its normal form (x) or in its complement form (x'). Now consider two binary variables x and y combined with an AND operation. Since each variable may appear in either form, there are four possible combinations: xy , $x'y'$, $x'y$, and xy' . Each of these four AND terms is called a minterm, or a standard product.
- In a similar manner, n variables can be combined to form 2^n minterms. The binary numbers from 0 to 2^n-1 are listed under the n variables.

canonical and Standard forms

Canonical and Standard form (T1: section 2.6)

Table 2.3

Minterms and Maxterms for Three Binary Variables

x	y	z	Minterms	
			Term	Designation
0	0	0	$x'y'z'$	m_0
0	0	1	$x'y'z$	m_1
0	1	0	$x'yz'$	m_2
0	1	1	$x'yz$	m_3
1	0	0	$xy'z'$	m_4
1	0	1	$xy'z$	m_5
1	1	0	xyz'	m_6
1	1	1	xyz	m_7

canonical and Standard forms

Canonical and Standard form (T1: section 2.6)

In a similar fashion, n variables forming an OR term, with each variable being primed or unprimed, provide 2^n possible combinations, called **maxterms**, or **standard sums**.

Table 2.3
Minterms and Maxterms for Three Binary Variables

x	y	z	Minterms		Maxterms	
			Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

canonical and Standard forms

Canonical and Standard form (T1: section 2.6)



It is important to note that

(1) each maxterm is obtained from an OR term of the n variables, with each variable being unprimed if the corresponding bit is a 0 and primed if a 1, and

(2) each maxterm is the complement of its corresponding minterm and vice versa. A Boolean function can be expressed algebraically from a given truth table by forming a minterm for each combination of the variables that produces a 1 in the function and then taking the OR of all those terms.

canonical and Standard forms

Canonical and Standard form (T1: section 2.6)

Table 2.4

Functions of Three Variables

<i>x</i>	<i>y</i>	<i>z</i>	Function f_1	Function f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

f_1 in Table 2.4 is determined by expressing the combinations 001, 100, and 111. Since each one of these minterms results in $f_1=1$, we have

$$\begin{aligned} f_1 &= x'y'z + xy'z' + xyz \\ &= m_1 + m_4 + m_7 \end{aligned}$$

$$\begin{aligned} f_2 &= x'yz + xy'z + xyz' + xyz \\ &= m_3 + m_5 + m_6 + m_7 \end{aligned}$$

Sum of Min Terms

Canonical and Standard form (T1: section 2.6)

Now consider the complement of a Boolean function. It may be read from the truth table by forming a minterm for each combination that produces a 0 in the function and then ORing those terms. The complement of f_1 is read as

$$F'_1 = x'y'z' + x'yz' + x'yz + xy'z + xyz'$$

Which is

$$\begin{aligned} f_1 &= (x + y + z)(x + y' + z)(x' + y + z')(x' + y' + z) \\ &= M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6 \end{aligned}$$

Similarly

$$\begin{aligned} f_2 &= (x + y + z)(x + y + z')(x + y' + z)(x' + y + z) \\ &= M_0 M_1 M_2 M_4 \end{aligned}$$

Product of max terms

canonical and Standard forms



Canonical and Standard form (T1: section 2.6)

- The procedure for obtaining the product of maxterms directly from the truth table is as follows: Form a maxterm for each combination of the variables that produces a 0 in the function, and then form the AND of all those maxterms.
- **Boolean functions expressed as a sum of minterms or product of maxterms are said to be in canonical form**

canonical and Standard forms

Canonical and Standard form

- Application of SOP and POS form:

SOP Form:

Fire Alarm System

Fire alarm activates if:

Smoke is detected (S)

Temperature is high (T)

CO gas is detected (C)

Fire Alarm = $S + T + C$

This is a **classic SOP** — the output is true if **any one danger is present**.

POS Form:

Traffic Light Control

Used to control light states **only when certain combinations of sensors are false**.

For example, a green light should **not** be given if any pedestrian button is pressed **or** vehicle is waiting.

POS is used to describe “green light condition” as:

Green = $(P' + V')$ → where P = pedestrian, V = vehicle.

canonical and Standard forms



Canonical and Standard form

Sum of Minterms

- The minterms whose sum defines the Boolean function are those which give the 1's of the function in a truth table
- Since the function can be either 1 or 0 for each minterm, and since there are 2^n minterms, one can calculate all the functions that can be formed with n variables to be 2^{2^n}
- Express the Boolean function $F = A + B'C$ as a sum of minterms. The function has three variables: A, B, and C.

canonical and Standard forms



Canonical and Standard form

Sum of Minterms

- Express the Boolean function $F = A + B'C$ as a sum of minterms. The function has three variables: A, B, and C.
- $F = A'B'C + AB'C + AB'C + ABC' + ABC = m_1 + m_4 + m_5 + m_6 + m_7$
- When a Boolean function is in its sum-of-minterms form, it is sometimes convenient to express the function in the following brief notation:
- **$F(A, B, C) = \sum (1, 4, 5, 6, 7)$**
- **The $\sum$ summation symbol stands for the OR ing of terms;**
- An alternative procedure for deriving the minterms of a Boolean function is to obtain the truth table of the function directly from the algebraic expression and then read the minterms from the truth table

canonical and Standard forms

Canonical and Standard form

Sum of Minterms

The truth table shown in Table 2.5 can be derived directly from the algebraic expression by listing the eight binary combinations under variables A, B, and C and inserting 1's under F for those combinations for which $A=1$ and $B'C=01$. From the truth table, we can then read the five minterms of the function to be 1, 4, 5, 6, and 7.

Table 2.5
Truth Table for $F = A + B'C$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

canonical and Standard forms



Canonical and Standard form

Product of Maxterms

Each of the 2^{2^n} functions of n binary variables can be also expressed as a product of maxterms.

To express a Boolean function as a product of maxterms, it must first be brought into a form of OR terms. This may be done by using the distributive law, $x+yz=(x+y)(x+z)$.

Express the Boolean function $F=xy+x'z$ as a product of maxterms.

canonical and Standard forms



Canonical and Standard form

Product of Maxterms

Express the Boolean function $F=xy+x'z$ as a product of maxterms.

First, convert the function into OR terms by using the distributive law:

$$F=xy+x'z$$

$$=(xy+x')(xy+z)$$

$$=(x+x')(y+x')(x+z)(y+z)$$

$$=(x'+y)(x+z)(y+z)$$

Each OR term is missing one variable;

$$=M_0M_2M_4M_5$$

$$F(x, y, z) = \pi(0, 2, 4, 5)$$

π denotes the ANDing of maxterms; the numbers are the indices of the maxterms of the function.

$$x' + y = x' + y + zz' = (x' + y + z)(x' + y + z')$$

$$x + z = x + z + yy' = (x + y + z)(x + y' + z)$$

$$y + z = y + z + xx' = (x + y + z)(x' + y + z)$$

canonical and Standard forms

Conversion between Canonical Forms

The complement of a function expressed as the sum of minterms equals the sum of min terms missing from the original function. This is because the original function is expressed by those minterms which make the function equal to 1, whereas its complement is a 1 for those minterms for which the function is a 0. As an example, consider the function $F(A, B, C) = \sum(1, 4, 5, 6, 7)$

This function has a complement that can be expressed as

$$F'(A, B, C) = \pi(0, 2, 3) = m_0 + m_2 + m_3$$

Now, if we take the complement of F' by DeMorgan's theorem, we obtain F in a different form:

$$F = (m_0 + m_2 + m_3)' = m_0' \cdot m_2' \cdot m_3' = M_0 M_2 M_3 = \pi(0, 2, 3)$$

$$m_j' = M_j$$

canonical and Standard forms

Conversion between Canonical Forms

Consider, for example, the Boolean expression $F = xy + x'z$

$$F(x, y, z) = \sum(1, 3, 6, 7)$$

$$F(x, y, z) = \pi(0, 2, 4, 5)$$

Table 2.6

Truth Table for $F = xy + x'z$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Minterms

Maxterms

canonical and Standard forms

Standard Forms-quick pointers



There are two types of standard forms: the sum of products and products of sums. The sum of products is a Boolean expression containing AND terms, called product terms, with one or more literals each. The sum denotes the ORing of these terms.

canonical and Standard forms

Standard Forms-quick pointers

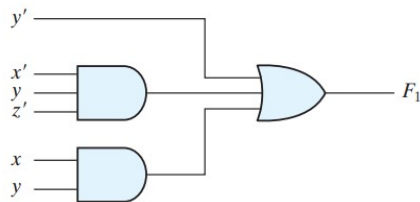


The product is an AND operation. The use of the words product and sum stems from the similarity of the AND operation to the arithmetic product (multiplication) and the similarity of the OR operation to the arithmetic sum (addition). The gate structure of the product-of-sums expression consists of a group of OR gates for the sum terms (except for a single literal), followed by an AND gate, as shown in Fig. 2.3 (b). This standard type of expression results in a two-level structure of gates.

canonical and Standard forms

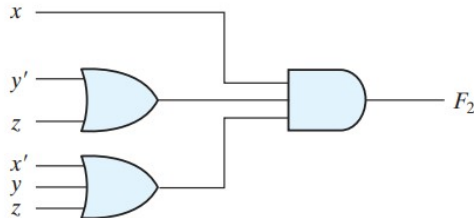
Canonical forms- two level implementation

$$F_1 = y' + xy + x'yz'$$



(a) Sum of Products

$$F_2 = x(y' + z)(x' + y + z')$$



(b) Product of Sums

FIGURE 2.3

Two-level implementation

canonical and Standard forms

Canonical forms- two level implementation



A Boolean function may be expressed in a nonstandard form.

For example, the function

$$F_3 = AB + C(D + E)$$

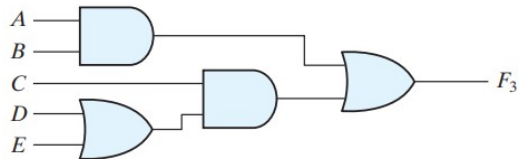
is neither in sum-of-products nor in product-of-sums form. The implementation of this expression is shown in Fig. 2.4 (a) and requires two AND gates and two OR gates. There are three levels of gating in this circuit.

It can be changed to a standard form by using the distributive law to remove the parentheses:

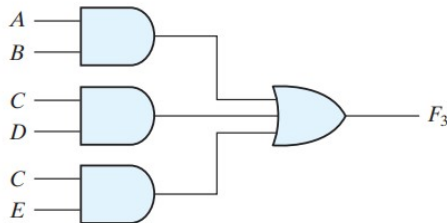
$$F_3 = AB + C(D + E) = AB + CD + CE$$

Canonical and Standard forms

Canonical forms- three level implementation



(a) $AB + C(D + E)$



(b) $AB + CD + CE$

FIGURE 2.4

Three- and two-level implementation

canonical and Standard forms

Canonical forms- three level implementation



In general, a two-level implementation is preferred because it produces the least amount of delay through the gates when the signal propagates from the inputs to the output.

Given the Boolean function $f(x, y, z) = \sum m(3, 5, 6, 7)$ (i.e., in SOP form), its equivalent POS (product-of-sum) form is:

- A) $\prod M(3, 5, 6, 7)$
- B) $\prod M(0, 1, 2, 4)$
- C) $\prod M(0, 2, 4)$
- D) $\prod M(1, 3, 5)$

Given the Boolean function $f(x, y, z) = \sum m(3, 5, 6, 7)$ (i.e., in SOP form), its equivalent POS (product-of-sum) form is:

- A) $\prod M(3, 5, 6, 7)$
- B) $\prod M(0, 1, 2, 4)$
- C) $\prod M(0, 2, 4)$
- D) $\prod M(1, 3, 5)$

Answer: B

Because the POS corresponds to the maxterms *not included* in the minterm list: $\{0, 1, 2, 4\}$

The maxterm corresponding to the binary combination $X = 1, Y = 0, Z = 1$ is:

- A) $x + y + z$
- B) $x' + y' + z'$
- C) $x' + y + z'$
- D) $x + y' + z$

The maxterm corresponding to the binary combination $X = 1, Y = 0, Z = 1$ is:

- A) $x + y + z$
- B) $x' + y' + z'$
- C) $x' + y + z'$
- D) $x + y' + z$

Answer: C



THANK YOU

Team DDCO
Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Gate-Level Minimization :

Karnaugh Maps

Team DDCO

Department of Computer Science and Engineering

Gate-Level Minimization and Combinational logic

Introduction :Chapter 3: 3.1,3.2,3.3,3.5



Gate-level minimization is the design task of finding an optimal gate-level implementation of the Boolean functions describing a digital circuit.

Logic minimizations

Boolean Identities

K-map.(Karnaugh map)

Simplest algebraic expression is an algebraic expression with a **minimum number of terms** and with the **smallest possible number of literals in each term.** This expression produces a circuit diagram with a minimum number of gates and the minimum number of inputs to each gate.

- The complexity of the digital logic gates that implement a Boolean function is directly related to the complexity of the algebraic expression from which the function is implemented.
- Boolean expressions may be simplified by algebraic means. However, this procedure of minimization is awkward because it lacks specific rules to predict each succeeding step in the manipulative process.
- **The map method provides a simple, straightforward procedure for minimizing Boolean functions.**
- This method may be regarded as a pictorial form of a truth table. The map method is also known as the *Karnaugh map* or *K-map*

Karnaugh Map

- **K-Map** is a **visual tool** used to simplify Boolean expressions.
- Introduced by **Maurice Karnaugh in 1953**, based on earlier work from 1881.
- **Each square in the K-map represents a minterm** of a Boolean function.
- A Boolean function can be expressed as a **sum of minterms** – this makes it easy to map onto a K-map.
- **Adjacent squares** represent minterms that differ by **only one literal**.
- Uses our brain's **pattern recognition ability** to simplify logic.
- Groups of 1s (e.g., 1, 2, 4, 8...) are formed to find **common literals**, which helps in **minimization**.

Two-Variable K-Map

- There are four minterms for two variables; hence, the map consists of four squares, one for each minterm.
- The map is redrawn in (b) to show the relationship between the squares and the two variables x and y .
- The 0 and 1 marked in each row and column designate the values of variables.

$$m_1 + m_2 + m_3 = x'y + xy' + xy = x + y$$

m_0	m_1
m_2	m_3

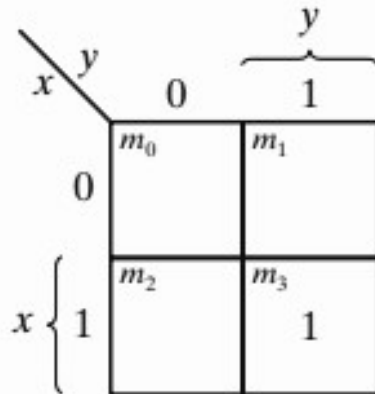
(a)

		y	
		0	1
x	0	m_0 $x'y'$	m_1 $x'y$
	1	m_2 xy'	m_3 xy

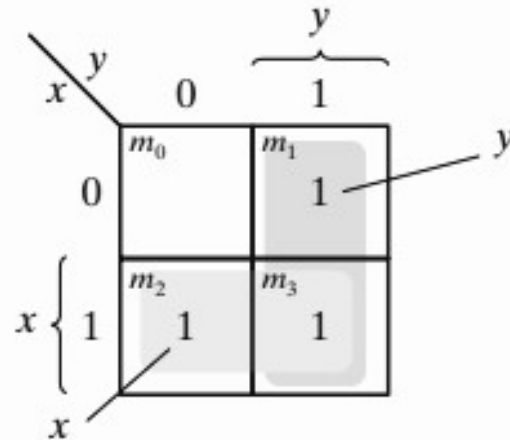
(b)

Two-Variable K-Map

Representation of functions in the map: The function $x + y$ is represented in the map as follows:



(a) xy



(b) $x + y$

Two-Variable K-Map



- The three squares could also have been determined from the intersection of variable x in the second row and variable y in the second column, which encloses the area belonging to x or y .
- In each example, the minterms at which the function is asserted are marked with a 1.

Three-Variable K-Map

- The characteristic of this sequence is that only one bit changes in value from one adjacent column to the next.
- The map drawn in part (b) is marked with numbers in each row and each column to show the relationship between the squares and the three variables.
- **Gray code** ensures that only one variable changes between each pair of adjacent cells.

Three-Variable K-Map

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

(a)

		y			
		yz	00	01	11
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'
		z			

(b)

$$m_5 + m_7 = xy'z + xyz = xz(y' + y) = xz$$

Three-Variable K-Map

a	b	c	y	minterm
0	0	0	0	$\bar{a}\bar{b}\bar{c}$
0	0	1	0	$\bar{a}\bar{b}c$
0	1	0	0	$\bar{a}b\bar{c}$
0	1	1	1	$\bar{a}bc$
1	0	0	1	$a\bar{b}\bar{c}$
1	0	1	0	$a\bar{b}c$
1	1	0	1	$ab\bar{c}$
1	1	1	1	abc

		bc			
		00	01	11	10
a	0	$\bar{a}\bar{b}\bar{c}_0$	$\bar{a}\bar{b}c_1$	$\bar{a}b\bar{c}_3$	$\bar{a}bc_2$
	1	$a\bar{b}\bar{c}_4$	$a\bar{b}c_5$	abc_7	$ab\bar{c}_6$

		bc			
		00	01	11	10
a	0	0 ₀	0 ₁	1 ₃	0 ₂
	1	1 ₄	0 ₅	1 ₇	1 ₆

- Each square corresponds to a row of the truth table
- Any two adjacent squares differ only in one literal
- Achieved using two rows and binary order 00,01,11,10
- Notion of “wrap-around”: far left and right squares are adjacent

K-Map Method

K-Map Implicants

- Implicant
 - ▶ K-Map area composed of squares containing 1's
 - ▶ Area is square or rectangular (wraparound allowed)
 - ▶ No. of squares in area is a power of two (1, 2, 4, ...)
 - ▶ Each implicant corresponds to a product of literals
 - ★ Double the area, one less literal
- Prime implicant
 - ▶ Implicant having largest number of squares obeying above rules
- Essential prime implicant
 - ▶ Prime implicant containing a square not in any other prime implicant

K-Map Method

- Include all required prime implicants
 - ▶ Include all essential prime implicants
 - ▶ Include other prime implicants such that:
 - ★ Each square containing 1 is covered
 - ★ Boolean formula is minimal (may not be unique)
- Convert required implicants to Boolean formula
 - ▶ Each implicant is a product of literals
 - ▶ Include literals which do not change over its area

K-Map Method



In choosing adjacent squares in a map, we must ensure that

- (1) all the minterms of the function are covered when we combine the squares,
- (2) the number of terms in the expression is minimized, and
- (3) there are no redundant terms (i.e., minterms already covered by other terms)

A prime implicant is a product term obtained by combining the maximum possible number of adjacent squares in the map.

If a minterm in a square is covered by only one prime implicant, that prime implicant is said to be essential.

The prime implicants of a function can be obtained from the map by combining all possible maximum numbers of squares.

K-Map Example

- Prime implicants: green colour
- Essential prime implicants: blue colour
- Required prime implicants: dark red colour

		bc			
		00	01	11	10
a	0	0	0	1	0
	1	1	0	1	1

a	b	c	y	minterm
0	0	0	0	$\bar{a}\bar{b}\bar{c}$
0	0	1	0	$\bar{a}\bar{b}c$
0	1	0	0	$\bar{a}b\bar{c}$
0	1	1	1	$\bar{a}bc$
1	0	0	1	$a\bar{b}\bar{c}$
1	0	1	0	$a\bar{b}c$
1	1	0	1	$ab\bar{c}$
1	1	1	1	abc

K-Map Example

- Prime implicants: green colour
- Essential prime implicants: blue colour
- Required prime implicants: dark red colour

		bc			
		00	01	11	10
a	0	0 ₀	0 ₁	1 ₃	0 ₂
	1	1 ₄	0 ₅	1 ₇	1 ₆

K-Map Example

- Prime implicants: green colour
- Essential prime implicants: blue colour
- Required prime implicants: dark red colour

		bc			
		00	01	11	10
a	0	0	0	1	0
	1	1	0	1	1

K-Map Example

- Prime implicants: green colour
 - Essential prime implicants: blue colour
 - Required prime implicants: dark red colour
- Three prime implicants

		bc			
		00	01	11	10
a	0	0 ₀	0 ₁	1 ₃	0 ₂
	1	1 ₄	0 ₅	1 ₇	1 ₆

K-Map Example

- Prime implicants: green colour
 - Essential prime implicants: blue colour
 - Required prime implicants: dark red colour
- Three prime implicants

		bc			
		00	01	11	10
a	0	0	0	1	0
	1	1	0	1	1

K-Map Example

- Prime implicants: green colour
- Essential prime implicants: blue colour
- Required prime implicants: dark red colour

- Three prime implicants
- Two essential prime implicants

		bc			
		00	01	11	10
a	0	0	0	1	0
	1	1	0	1	1

K-Map Example

- Prime implicants: green colour
- Essential prime implicants: blue colour
- Required prime implicants: dark red colour

- Three prime implicants
- Two essential prime implicants

		bc			
		00	01	11	10
a	0	0	0	1	0
	1	1	0	1	1

K-Map Example

- Prime implicants: green colour
- Essential prime implicants: blue colour
- Required prime implicants: dark red colour

- Three prime implicants
- Two essential prime implicants
- Required prime implicants: bc , $a\bar{c}$

		bc			
		00	01	11	10
a	0	0 ₀	0 ₁	1 ₃	0 ₂
	1	1 ₄	0 ₅	1 ₇	1 ₆

K-Map Example

- Prime implicants: green colour
- Essential prime implicants: blue colour
- Required prime implicants: dark red colour

- Three prime implicants
- Two essential prime implicants
- Required prime implicants: bc , $a\bar{c}$
- Boolean formula:

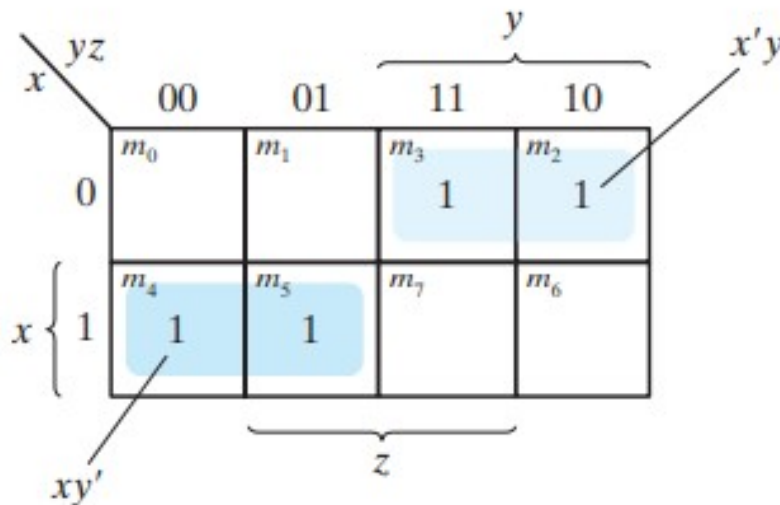
$$f(a, b, c) = a\bar{c} + bc$$

		bc			
		00	01	11	10
a	0	0	0	1	0
	1	1	0	1	1

Three Variable K-Map

Simplify the Boolean function

$$F(x, y, z) = \Sigma(2, 3, 4, 5)$$



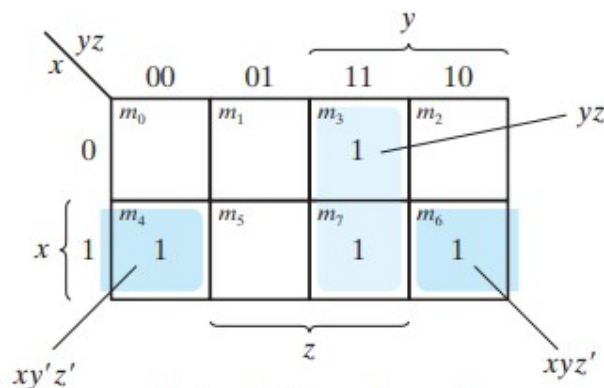
$$F = x'y + xy'$$

Three Variable K-Map

Simplify the Boolean function

$$F(x, y, z) = \Sigma(3, 4, 6, 7)$$

$$F = yz + xz'$$



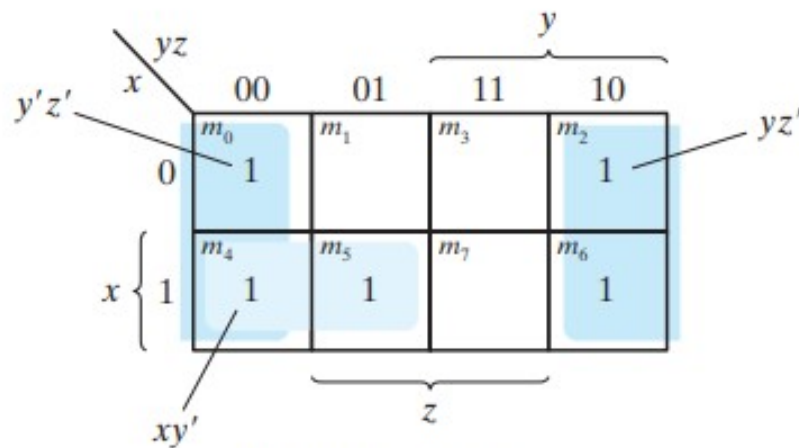
Note: $xy'z' + xyz' = xz'$

Three Variable K-Map

Simplify the Boolean function

$$F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$$

$$F = z' + xy'$$



$$\text{Note: } y'z' + yz' = z'$$

Quick Pointers

- The number of adjacent squares that may be combined must always represent a number that is a power of two, such as 1, 2, 4, and 8.
- As more adjacent squares are combined, we obtain a product term with fewer literals.
- One square represents one minterm, giving a term with three literals.
- Two adjacent squares represent a term with two literals.
- Four adjacent squares represent a term with one literal.
- Eight adjacent squares encompass the entire map and produce a function that is always equal to 1.

Example

For the Boolean function

$$F = A'C + A'B + AB'C + BC$$

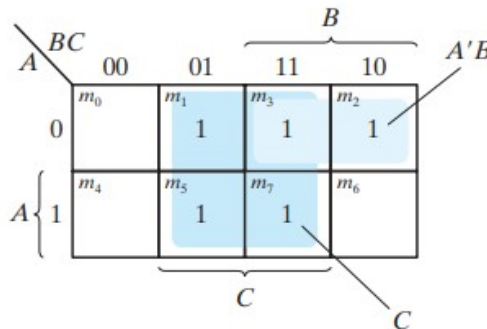
- (a) Express this function as a sum of minterms.
- (b) Find the minimal sum-of-products expression.

sum-of-minterms form as

$$F(A, B, C) = \Sigma(1, 2, 3, 5, 7)$$

The sum-of-products expression, as originally given, has too many terms. It can be simplified, as shown in the map, to an expression with only two terms:

$$F = C + A'B$$



Four Variable K-Map

- The map minimization of four-variable Boolean functions is similar to the method used to minimize three-variable functions. Adjacent squares are defined to be squares next to each other.
- For example, m_0 and m_2 form adjacent squares, as do m_3 and m_{11} .
- The combination of adjacent squares that is useful during the simplification process is easily determined from inspection of the four-variable map:

Four Variable K-Map

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

(a)

		y			
		yz	00	01	11
w	00	m_0 $w'x'y'z'$	m_1 $w'x'y'z$	m_3 $w'x'yz$	m_2 $w'x'yz'$
	01	m_4 $w'xy'z'$	m_5 $w'xy'z$	m_7 $w'xyz$	m_6 $w'xyz'$
	11	m_{12} $wxy'z'$	m_{13} $wxy'z$	m_{15} $wxyz$	m_{14} $wxyz'$
	10	m_8 $wx'y'z'$	m_9 $wx'y'z$	m_{11} $wx'yz$	m_{10} $wx'yz'$
		z			

(b)

Quick Pointers

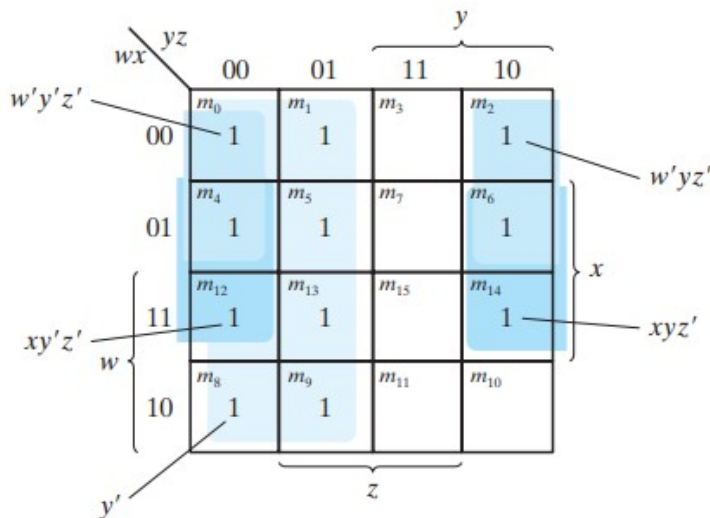
- One square represents one minterm, giving a term with four literals.
- Two adjacent squares represent a term with three literals.
- Four adjacent squares represent a term with two literals.
- Eight adjacent squares represent a term with one literal.
- Sixteen adjacent squares produce a function that is always equal to 1.

Example

Simplify the Boolean function

$$F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$$

$$F = y' + w'z' + xz'$$



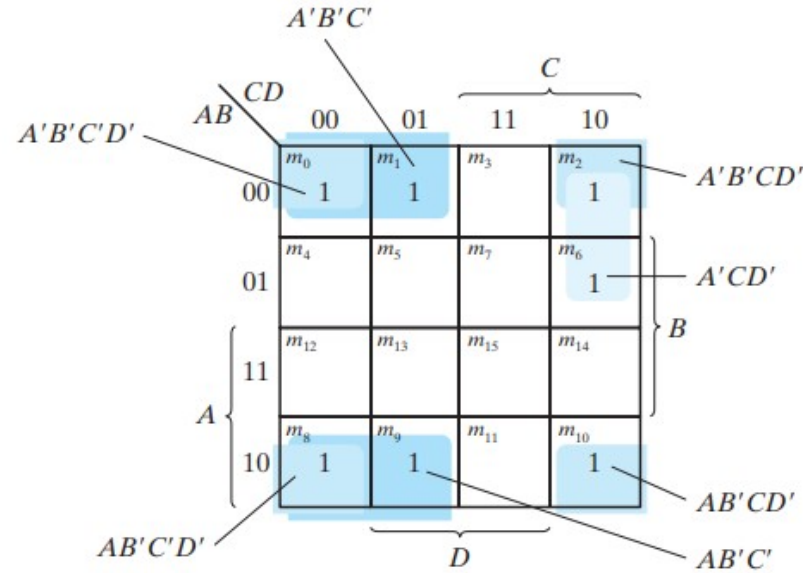
Note: $w'y'z' + w'yz' = w'z'$
 $xy'z' + xyz' = xz'$

Example

Simplify the Boolean function

$$F = A'B'C' + B'CD' + A'BCD' + AB'C'$$

Example



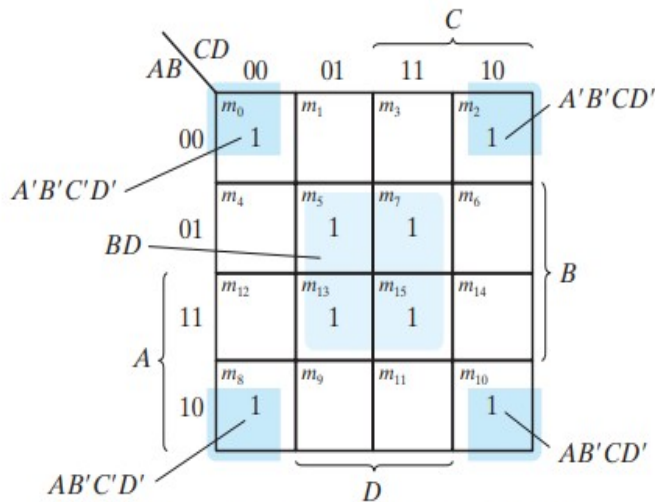
Note: $A'B'C'D' + A'B'CD' = A'B'D'$
 $AB'C'D' + AB'CD' = AB'D'$
 $A'B'D' + AB'D' = B'D'$
 $A'B'C' + AB'C' = B'C'$

FIGURE 3.10

Map for Example 3.6, $A'B'C' + B'CD' + A'BCD' + AB'C' = B'D' + B'C' + A'CD'$

Simplification using Prime Implicants

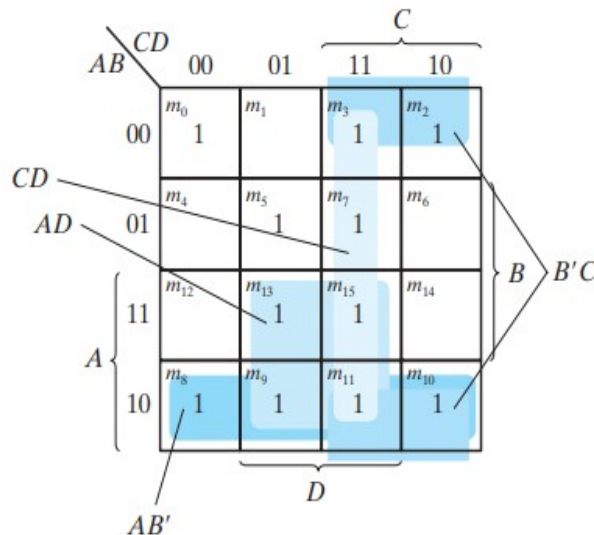
$$(a) F(w, x, y, z) \\ = \Sigma(0, 2, 5, 7, 8, 10, 13, 15)$$



Note: $A'B'C'D' + A'B'CD' = A'B'D'$
 $AB'C'D' + AB'CD' = AB'D'$
 $A'B'D' + AB'D' = B'D'$

(a) Essential prime implicants
 BD and $B'D'$

$$(b) F(w, x, y, z) \\ = \Sigma(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$$



(b) Prime implicants CD , $B'C$,
 AD , and AB'

$$\begin{aligned} F &= BD + B'D' + CD + AD \\ &= BD + B'D' + CD + AB' \\ &= BD + B'D' + B'C + AD \\ &= BD + B'D' + B'C + AB' \end{aligned}$$

Another K-Map Example

K-Map Minimization Example

<i>a</i>	<i>b</i>	<i>y</i>
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table

Another K-Map Example

K-Map Example (two inputs)

<i>a</i>	<i>b</i>	<i>y</i>
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table

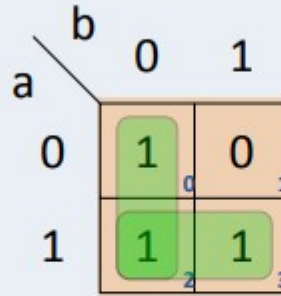
		<i>b</i>	
		0	1
<i>a</i>	0	1	0
	1	1	1

Another K-Map Example

K-Map Example (two inputs)

<i>a</i>	<i>b</i>	<i>y</i>
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table



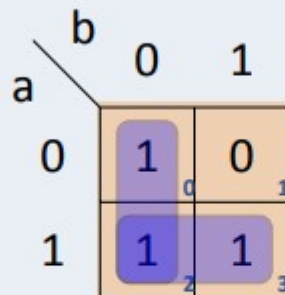
- Two prime implicants

Another K-Map Example

K-Map Example (two inputs)

<i>a</i>	<i>b</i>	<i>y</i>
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table



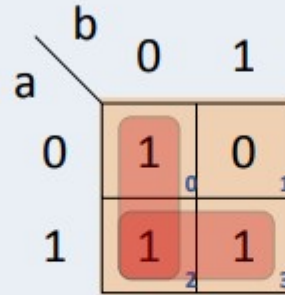
- Two prime implicants
- Two essential prime implicants

Another K-Map Example

K-Map Example (two inputs)

<i>a</i>	<i>b</i>	<i>y</i>
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table



- Two prime implicants
- Two essential prime implicants
- Two required prime implicants

Another K-Map Example

K-Map Example (two inputs)

<i>a</i>	<i>b</i>	<i>y</i>
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table

		<i>b</i>	
		0	1
<i>a</i>	0	1	0
	1	1	1

- Two prime implicants
- Two essential prime implicants
- Two required prime implicants
- Minimized Boolean formula:
$$f(a, b, c, d) = \bar{b} + a$$

Another K-Map Example

K-Map Minimization Example

<i>a</i>	<i>b</i>	<i>c</i>	<i>y</i>
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table

Another K-Map Example

K-Map Minimization Example

<i>a</i>	<i>b</i>	<i>c</i>	<i>y</i>
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table

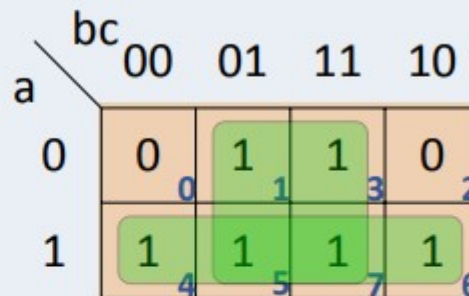
		<i>bc</i>			
		00	01	11	10
<i>a</i>	0	0 0	1 1	1 3	0 2
	1	1 4	1 5	1 7	1 6

Another K-Map Example

K-Map Minimization Example

<i>a</i>	<i>b</i>	<i>c</i>	<i>y</i>
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



- Two prime implicants

Another K-Map Example

K-Map Minimization Example

<i>a</i>	<i>b</i>	<i>c</i>	<i>y</i>
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table

		<i>bc</i>			
		00	01	11	10
<i>a</i>	0	0	1	1	0
	1	1	1	1	1

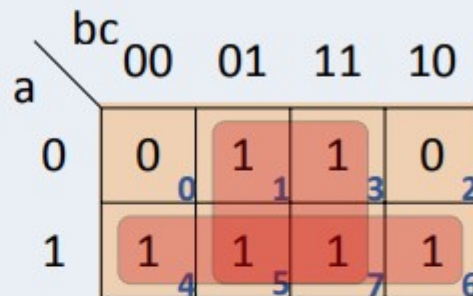
- Two prime implicants
- Two essential prime implicants

Another K-Map Example

K-Map Minimization Example

<i>a</i>	<i>b</i>	<i>c</i>	<i>y</i>
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



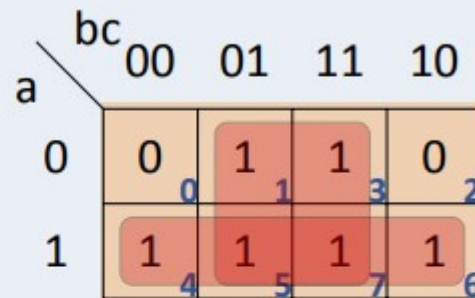
- Two prime implicants
- Two essential prime implicants
- Two required prime implicants: *c*, *a*

Another K-Map Example

K-Map Minimization Example

<i>a</i>	<i>b</i>	<i>c</i>	<i>y</i>
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



- Two prime implicants
- Two essential prime implicants
- Two required prime implicants: c, a
- Minimal Boolean formula:
 $f(a, b, c) = c + a$

Yet Another K-Map Example

K-Map Minimization Example

<i>a</i>	<i>b</i>	<i>c</i>	<i>y</i>
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table

Yet Another K-Map Example

K-Map Minimization Example

<i>a</i>	<i>b</i>	<i>c</i>	<i>y</i>
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table

		<i>bc</i>			
		00	01	11	10
<i>a</i>	0	0	1	0	1
	1	0	1	1	1

Yet Another K-Map Example

K-Map Minimization Example

<i>a</i>	<i>b</i>	<i>c</i>	<i>y</i>
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table

		<i>bc</i>			
		00	01	11	10
<i>a</i>	0	0	1	0	1
	1	0	1	1	1

- Four prime implicants

Yet Another K-Map Example

K-Map Minimization Example

<i>a</i>	<i>b</i>	<i>c</i>	<i>y</i>
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table

		<i>bc</i>			
		00	01	11	10
<i>a</i>	0	0	1	0	1
	1	0	1	1	1

- Four prime implicants
- Two essential prime implicants

Yet Another K-Map Example

K-Map Minimization Example

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>y</i>
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

Yet Another K-Map Example

K-Map Example (four inputs)

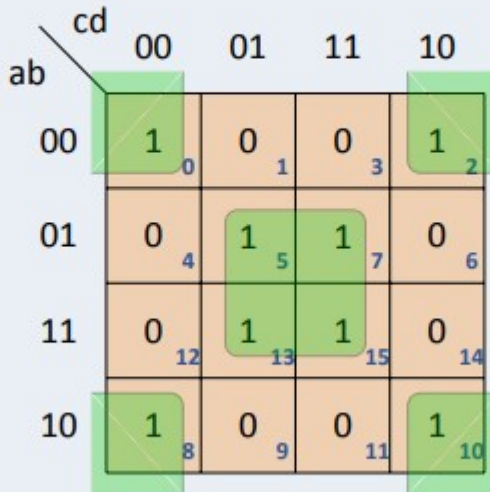
a	b	c	d	y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

		cd			
		00	01	11	10
ab	00	1 ₀	0 ₁	0 ₃	1 ₂
	01	0 ₄	1 ₅	1 ₇	0 ₆
	11	0 ₁₂	1 ₁₃	1 ₁₅	0 ₁₄
	10	1 ₈	0 ₉	0 ₁₁	1 ₁₀

Yet Another K-Map Example

K-Map Example (four inputs)

a	b	c	d	y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1



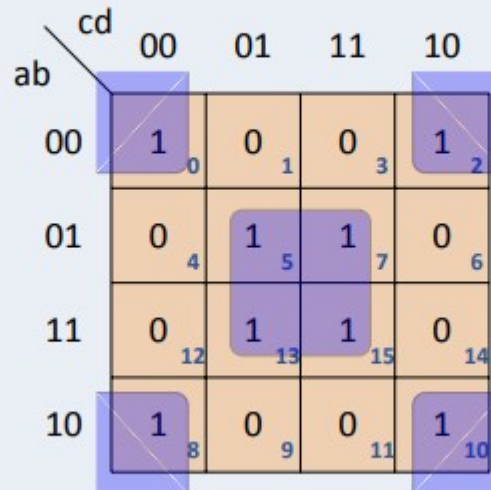
- Two prime implicants

Yet Another K-Map Example

K-Map Example (four inputs)

a	b	c	d	y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

Four input truth table



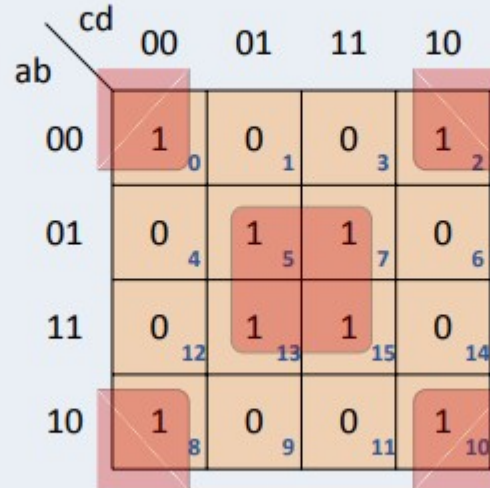
- Two prime implicants
- Two essential prime implicants

Yet Another K-Map Example

K-Map Example (four inputs)

a	b	c	d	y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

Four Input Truth Table

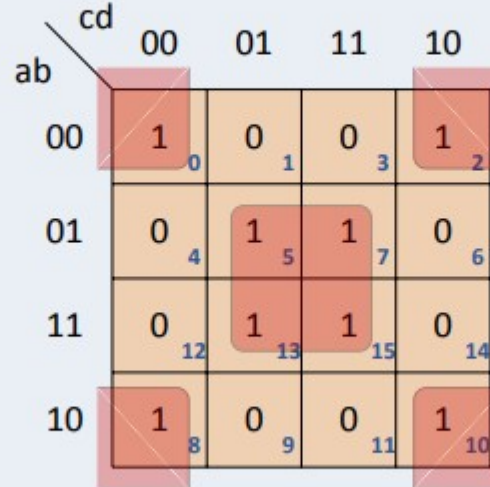


- Two prime implicants
- Two essential prime implicants
- Two required prime implicants: $bd, \overline{bd}$

Yet Another K-Map Example

K-Map Example (four inputs)

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>y</i>
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1



- Two prime implicants
- Two essential prime implicants
- Two required prime implicants: bd , $\overline{b}\overline{d}$
- Minimized Boolean formula:

$$f(a, b, c, d) = bd + \overline{b}\overline{d}$$

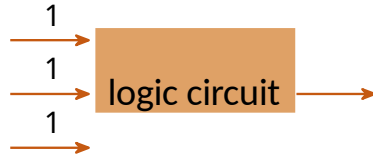
Gate-Level Minimization and Combinational logic **Don't-care conditions.**(section 3.6)

- Functions that have **unspecified outputs** for some input combinations are called incompletely specified functions .
- In most applications, we simply don't care what value is assumed by the function for the unspecified minterms.
- For this reason, it is customary to call the unspecified minterms of a function don't-care conditions

K-MAPS

Don't Cares

- Suppose we are asked to design a three input logic circuit all whose inputs can never be 1 simultaneously



guaranteed can't happen

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	X

Truth table

a \ bc	00	01	11	10
	0	1	1	0
0	1	1	0	0
1	1	1	X	1

- Case 0: $y = \bar{b} + \bar{c}a$
- Case 1: $y = \bar{b} + a$
- Either 0 or 1 can result in smaller formula
- So initially write X in truth table and K-Map

Don't Care

Output(s) we **don't care** about are denoted by X , can be treated as either 0 or 1

Gate-Level Minimization and Combinational logic Don't-care conditions.

(a) $F(w, x, y, z) = \sum m(1, 3, 7, 11, 15) + d(0, 2, 5)$

		y			
		00	01	11	10
wx	yz	m_0	m_1	m_3	m_2
	00	X	1	1	X
w'x'	01	m_4	m_5	m_7	m_6
	01	0	X	1	0
w	11	m_{12}	m_{13}	m_{15}	m_{14}
	11	0	0	1	0
10	m_8	m_9	m_{11}	m_{10}	
	10	0	0	1	0

Diagram (a) shows a 4x4 Karnaugh map for function F. The map is organized with wx on the vertical axis and yz on the horizontal axis. The cells are labeled with minterms m_0 through m_{15} . The values are: $m_0=X$, $m_1=1$, $m_3=1$, $m_2=X$, $m_4=0$, $m_5=X$, $m_7=1$, $m_6=0$, $m_{12}=0$, $m_{13}=0$, $m_{15}=1$, $m_{14}=0$, $m_8=0$, $m_9=0$, $m_{11}=1$, $m_{10}=0$. The map is grouped into four 2x2 blocks, each representing a prime implicant: yz (top-left), w'x' (top-right), w'z (bottom-left), and wz (bottom-right). The function is $F = yz + w'x'$.

(a) $F = yz + w'x'$

(b) $F(w, x, y, z) = \sum m(1, 3, 7, 11, 15) + d(0, 2, 5)$

		y			
		00	01	11	10
wx	yz	m_0	m_1	m_3	m_2
	00	X	1	1	X
w'z	01	m_4	m_5	m_7	m_6
	01	0	X	1	0
w	11	m_{12}	m_{13}	m_{15}	m_{14}
	11	0	0	1	0
10	m_8	m_9	m_{11}	m_{10}	
	10	0	0	1	0

Diagram (b) shows a 4x4 Karnaugh map for function F. The map is organized with wx on the vertical axis and yz on the horizontal axis. The cells are labeled with minterms m_0 through m_{15} . The values are: $m_0=X$, $m_1=1$, $m_3=1$, $m_2=X$, $m_4=0$, $m_5=X$, $m_7=1$, $m_6=0$, $m_{12}=0$, $m_{13}=0$, $m_{15}=1$, $m_{14}=0$, $m_8=0$, $m_9=0$, $m_{11}=1$, $m_{10}=0$. The map is grouped into four 2x2 blocks, each representing a prime implicant: yz (top-left), w'z (top-right), w'x' (bottom-left), and wz (bottom-right). The function is $F = yz + w'z$.

(b) $F = yz + w'z$

Gate-Level Minimization and Combinational logic

Product of sums simplification.

- The minimized Boolean functions derived from the map in all previous examples were expressed in sum-of-products form.
- With a minor modification, the product-of-sums form can be obtained.
- The procedure for obtaining a minimized function in product-of-sums form follows from the basic properties of Boolean functions.

Gate-Level Minimization and Combinational logic

Product of sums simplification.

- The 1's placed in the squares of the map represent the minterms of the function.
- The minterms not included in the standard sum-of-products form of a function denote the complement of the function.
- From this observation, we see that the complement of a function is represented in the map by the squares not marked by 1's
- If we mark the empty squares by 0's and combine them into valid adjacent squares, we obtain a simplified sum-of-products expression of the complement of the function (i.e., of F').
- The complement of F' gives us back the function F in product-of-sums form (a consequence of DeMorgan's theorem)

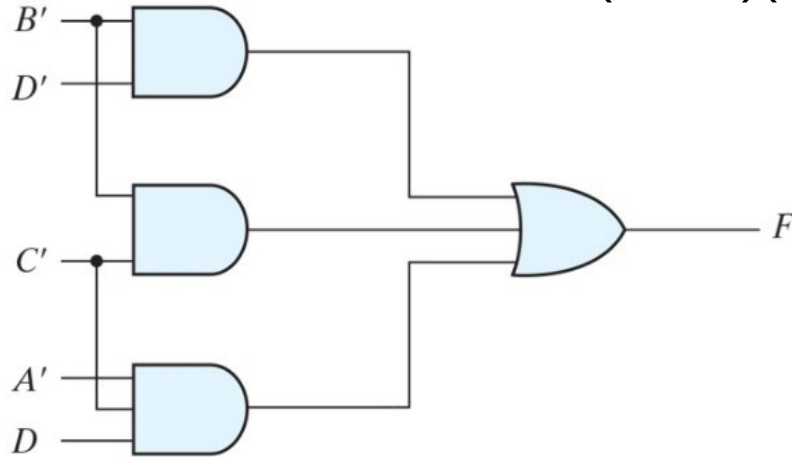
Gate-Level Minimization and Combinational logic

Product of sums simplification.

Gate implementations of the function –

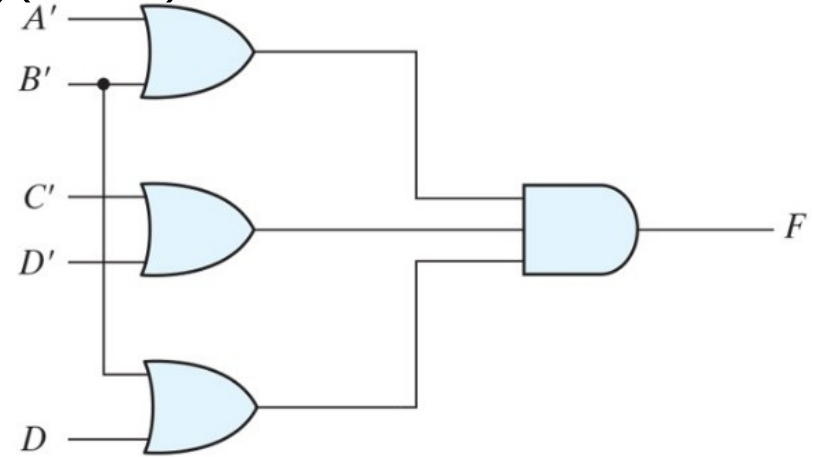
$$F(A, B, C, D) = (0, 1, 2, 5, 8, 9, 10)$$

$$= B'D' + B'C' + A'C'D = (A + B)(C + D)(B + D)$$



(a) $F = B'D' + B'C' + A'C'D$

SOP



(b) $F = (A' + B')(C' + D')(B' + D)$

POS

Real-Life Applications of K-Maps



Application: Seatbelt Warning System in Cars:

Trigger a seatbelt warning buzzer based on the driver's seat occupancy, seatbelt status, and car ignition.

Variable

Meaning

A

Seat Occupied (1 = Yes, 0 = No)

B

Seatbelt Buckled (1 = Yes, 0 = No)

C

Ignition ON (1 = ON, 0 = OFF)

W

Warning buzzer ON (1 = ON, 0 = OFF)- output

Real-Life Applications of K-Maps

A B ↓ \ C →	C = 0	C = 1
A=0, B=0	0	0
A=0, B=1	0	0
A=1, B=1	0	0
A=1, B=0	0	1

$$W = A \cdot B' \cdot C$$

Circuit:

1. NOT gate to invert B → B`
2. 3-input AND gate with inputs A, B` and C
3. Output drives buzzer (via transistor if needed)

Real-Life Applications of K-Maps



Digital Circuits: Karnaugh maps are widely used in the design of digital circuits. The simplified expressions obtained from K-Maps can be easily translated into logic gates, making it easier to design and implement the circuit.

Computer memory: K-Maps are used in the design of computer memory. The simplified expressions obtained from K-Maps help in reducing the size and complexity of the memory circuit.

Communication systems: K-Maps are used in the design of communication systems. The simplified expressions obtained from K-Maps help in reducing the complexity and improving the efficiency of the communication system.

Real-Life Applications of K-Maps



Consumer electronics: K-Maps are used in the design of consumer electronics such as televisions, radios, and other electronic devices. The simplified expressions obtained from K-Maps help in reducing the size and complexity of electronic devices.

Automotive electronics: K-Maps are used in the design of automotive electronics such as engine control units, braking systems, and other electronic systems. The simplified expressions obtained from K-Maps help in reducing the size and complexity of the electronic systems, making them more efficient and reliable. Used in error detection like [Parity Checkers](#), [Cyclic Redundancy Check \(CRC\)](#) etc.

Try it Out

Minimize using K-Maps

- $f(a, b, c) = \Sigma(0, 3, 5)$
- $f(a, b, c) = \Sigma(0, 1, 2, 4, 5, 6)$
- $f(a, b, c) = \Sigma(0, 2, 3, 4, 6, 7)$
- $f(a, b) = \Sigma(0, 1, 2, 3)$
- $f(a, b, c, d) = \Sigma(0, 1, 5, 7, 15, 14, 10)$
- $g(w, x, y, z) = \Sigma(1, 3, 6, 12, 15) + d(0, 2, 5)$
- $s(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 7, 9) + d(3, 8, 15)$



THANK YOU

Team DDCO

Department of Computer Science and Engineering



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Gate Level Minimization: NAND and NOR Implementation

Department of Computer Science and Engineering

DIGITAL DESIGN AND COMPUTER

ORGANIZATION NAND and NOR implementation

Department of Computer Science and Engineering

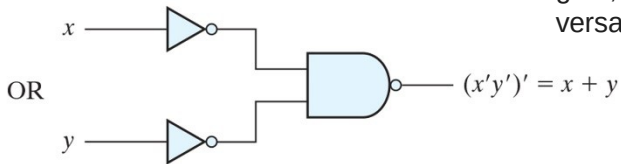
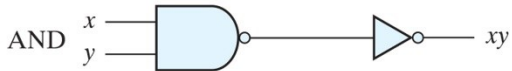
Gate-Level Minimization

NAND and NOR implementation(Universal Gates)

- A convenient way to implement a Boolean function with NAND gates is to obtain the simplified Boolean function in terms of Boolean operators and then convert the function to NAND logic.
- The conversion of an algebraic expression from AND, OR, and complement to NAND can be done by simple circuit manipulation techniques that change AND-OR diagrams to NAND diagrams.

Gate-Level Minimization

NAND and NOR implementation(T1-3.6)



Universal logic gates, NAND and NOR, are essential building blocks in digital electronics. They can create any other logic gate, making them highly versatile for circuit design

NAND



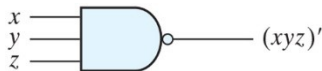
$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

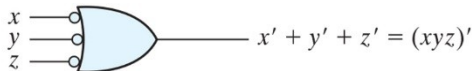
Logic operations with NAND gates.

Gate-Level Minimization

NAND and NOR implementation



(a) AND-invert



(b) Invert-OR

Two graphic symbols for a **three-input NAND gate**.

Gate-Level Minimization

NAND and NOR implementation



- The AND-invert symbol has been defined previously and consists of an AND graphic symbol followed by a small circle negation indicator referred to as a bubble.
- It is possible to represent a NAND gate by an OR graphic symbol that is preceded by a bubble in each input.
- The **invert-OR symbol for the NAND gate follows DeMorgan's theorem** and the convention that the negation indicator (bubble) denotes complementation.

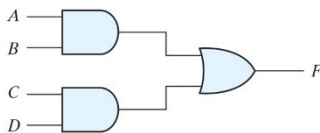
The implementation of Boolean functions with NAND gates requires that the functions be in sum-of-products form.

Implement the function $F = AB + CD$ with

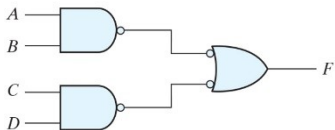
- And ,OR gate
- NAND gates only(AND-Invert only)
- AND -Invert and Invert-OR

Gate-Level Minimization

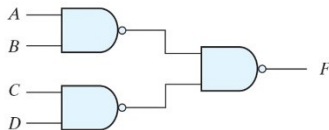
NAND and NOR implementation(2 level Implementation)



(a)



(b)



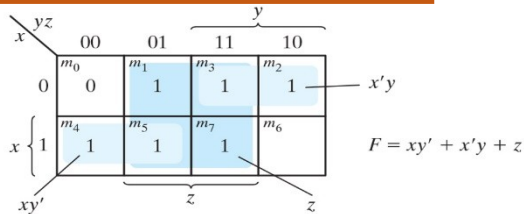
(c)

Three ways to implement $F = AB + CD$.

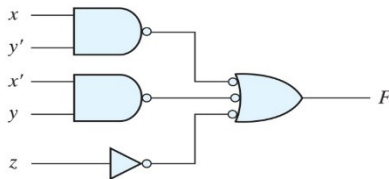
Implement the following Boolean function with NAND gates:

$$F(x, y, z) = (1, 2, 3, 4, 5, 7)$$

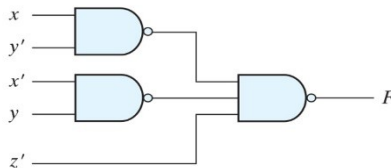
Gate-Level Minimization and Combinational logic **NAND and NOR** implementation



(a)



(b)



(c)

Solution to Solved Example

Gate-Level Minimization and Combinational logic

NAND and NOR implementation

The procedure for obtaining the logic diagram from a Boolean function is as follows:

1. Simplify the function and express it in sum-of-products form.
2. Draw a NAND gate for each product term of the expression that has at least two literals. The inputs to each NAND gate are the literals of the term. This procedure produces a group of first-level gates.
3. Draw a single gate using the AND-invert or the invert-OR graphic symbol in the second level, with inputs coming from outputs of first-level gates.
4. A term with a single literal requires an inverter in the first level. However, if the single literal is complemented, it can be connected directly to an input of the second level NAND gate

Gate-Level Minimization and Combinational logic

NAND and NOR implementation

(multilevel NAND gate)



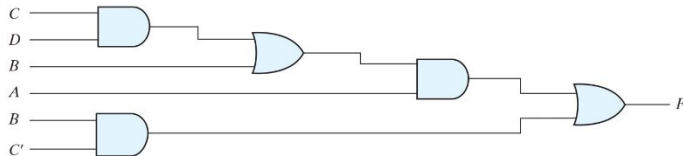
The standard form of expressing Boolean functions results in a two-level implementation. There are occasions, however, when the design of **digital systems results in gating structures with three or more levels**

The most common procedure in the design of multilevel circuits is to express the Boolean function in terms of AND, OR, and complement operations. The function can then be implemented with AND and OR gates. After that, if necessary, it can be converted into an all-NAND circuit.

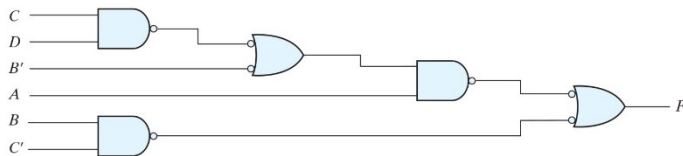
Implementing $F = A(CD + B) + BC'$ using NAND gate(AND invert, Invert OR) . Implement the same using AND-OR gates
Draw the Circuit .

Gate-Level Minimization

NAND and NOR implementation(multilevel NAND gate)



(a) AND-OR gates



(b) NAND gates

Implementing $F = A(CD + B) + BC'$.

Gate-Level Minimization and Combinational logic

NAND and NOR implementation(multilevel NAND gate)



Although it is possible to remove the parentheses and reduce the expression into a standard sum-of-products form, we choose to implement it as a multilevel circuit for illustration

The general procedure for converting a multilevel AND-OR diagram into an all-NAND diagram using mixed notation is as follows:

1. Convert all AND gates to NAND gates with AND-invert graphic symbols.
2. Convert all OR gates to NAND gates with invert-OR graphic symbols.
3. Check all the bubbles in the diagram. For every bubble that is not compensated by another small circle along the same line, insert an inverter (a one-input NAND gate) or complement the input literal.

Implement $F = (AB' + A'B)(C + D')$ using AND-OR gates, NAND gates

Gate-Level Minimization and Combinational logic

NAND and NOR implementation(multilevel NAND gate)

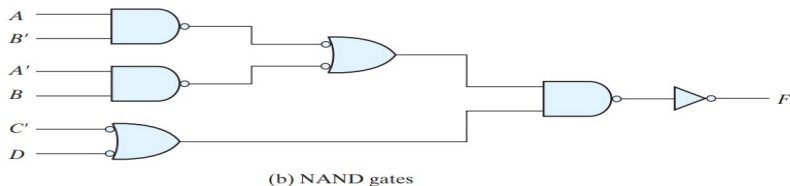
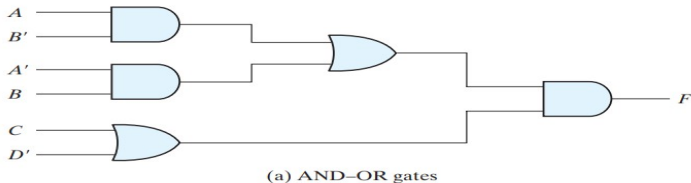


FIGURE 3.21
Implementing $F = (AB' + A'B)(C + D')$

Gate-Level Minimization and Combinational logic

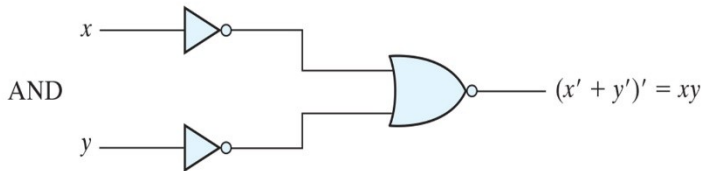
NOR implementation



The NOR operation is the dual of the NAND operation. Therefore, all procedures and rules for NOR logic are the **duals** of the corresponding procedures and rules developed for NAND logic. The NOR gate is another universal gate that can be used to implement any Boolean function

Gate-Level Minimization and Combinational logic

NOR implementation



NOR



$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

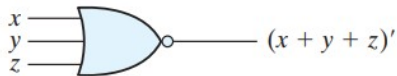
Logic operations with NOR gates.

Gate-Level Minimization and Combinational logic

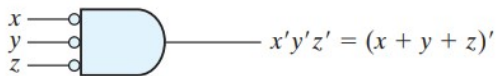
NOR implementation



Logic operations with NOR gates



(a) OR-invert



(b) Invert-AND

FIGURE 3.23

Two graphic symbols for the NOR gate

The procedure for converting a multilevel AND–OR diagram to an all-NOR diagram. we must convert each OR gate to an OR-invert symbol and each AND gate to an invert-AND symbol

Gate-Level Minimization and Combinational logic

NOR implementation



Implementing $F = (A + B)(C + D)E$

Implementing $F = (A'B + AB')(C + D')$ with NOR gates

Gate-Level Minimization and Combinational logic

NAND and NOR implementation

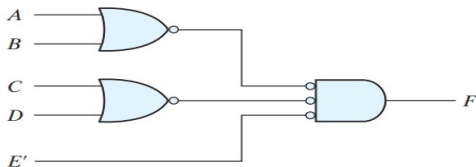


FIGURE 3.24
Implementing $F = (A + B)(C + D)E$

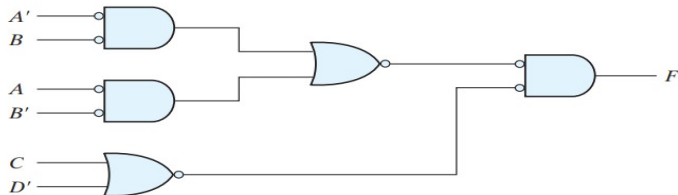


FIGURE 3.25
Implementing $F = (AB' + A'B)(C + D')$ with NOR gates

Exclusive-OR Function (T1: 3.8)

The exclusive-OR (XOR), denoted by the symbol $\oplus$, is a logical operation that performs the following Boolean operation:

$$x \oplus y = xy' + x'y$$

The exclusive-OR is equal to 1 if only x is equal to 1 or if only y is equal to 1 (i.e., x and y differ in value), but not when both are equal to 1 or when both are equal to 0. The exclusive-NOR, also known as equivalence, performs the following Boolean operation:

$$(x \oplus y)' = xy + x'y'$$

XOR



$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

XNOR



$$Y = \overline{A \oplus B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Exclusive-OR Function

The following identities apply to the exclusive-OR operation:

$$x \oplus 0 = x$$

$$x \oplus 1 = x'$$

$$x \oplus x = 0$$

$$x \oplus x' = 1$$

$$x \oplus y' = x' \oplus y = (x \oplus y)'$$

Any of these identities can be proven with a truth table or by replacing the $\oplus$ operation by its equivalent Boolean expression. Also, it can be shown that the exclusive-OR operation is both commutative and associative; that is,

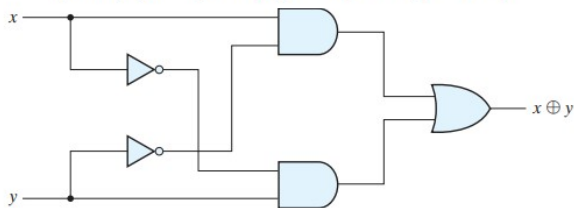
$$A \oplus B = B \oplus A$$

and

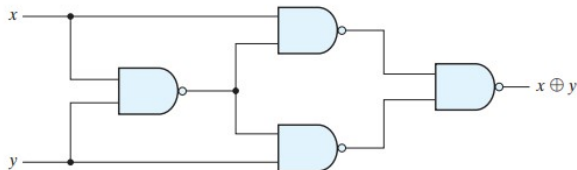
$$(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$$

Exclusive-OR Implementation

$$(x' + y')x + (x' + y')y = xy' + x'y = x \oplus y$$



(a) Exclusive-OR with AND-OR-NOT gates



(b) Exclusive-OR with NAND gates

Odd function - XOR for 3 variables

- Multiple-variable exclusive-OR operation is defined as an odd function.
- XOR – 3 variables: $A \text{ XOR } B \text{ XOR } C$
- The Boolean expression clearly indicates that the three-variable exclusive-OR function is equal to 1 if only one variable is equal to 1 or if all three variables are equal to 1.
- Multiple variable XOR – ODD Function: The odd function is identified from the four minterms whose binary values have an odd number of 1's- ODD parity.
- In general, an n -variable exclusive-OR function is an odd function defined as the logical sum of the $2^n / 2$ minterms whose binary numerical values have an odd number of 1's.

Odd function - XOR for 3 variables

$A \backslash BC$		B			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6
		1		1	

(a) Odd function $F = A \oplus B \oplus C$

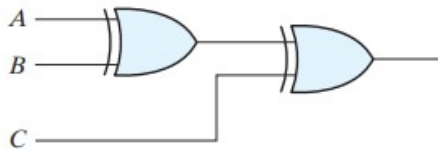
$A \backslash BC$		B			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6
		1			1

(b) Even function $F = (A \oplus B \oplus C)'$

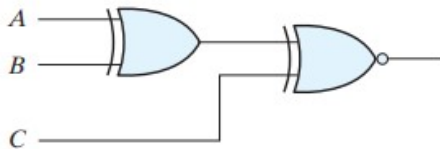
FIGURE 3.31

Map for a three-variable exclusive-OR function

Odd function - XOR for 3 variables



(a) 3-input odd function



(b) 3-input even function

FIGURE 3.32

Logic diagram of odd and even functions

Odd function - XOR for 4 variables

- There are 16 minterms for a four-variable Boolean function.
- Half of the minterms have binary numerical values with an odd number of 1's;
- The other half of the minterms have binary numerical values with an even number of 1's

Odd function - XOR for 4 variables

$$\begin{aligned}
 A \oplus B \oplus C \oplus D &= (AB' + A'B) \oplus (CD' + C'D) \\
 &= (AB' + A'B)(CD + C'D') + (AB + A'B')(CD' + C'D) \\
 &= \Sigma(1, 2, 4, 7, 8, 11, 13, 14)
 \end{aligned}$$

AB \ CD		C			
		00	01	11	10
A	00	m_0	m_1 1	m_3	m_2 1
	01	m_4 1	m_5	m_7 1	m_6
	11	m_{12}	m_{13} 1	m_{15}	m_{14} 1
	10	m_8 1	m_9	m_{11} 1	m_{10}
		D			

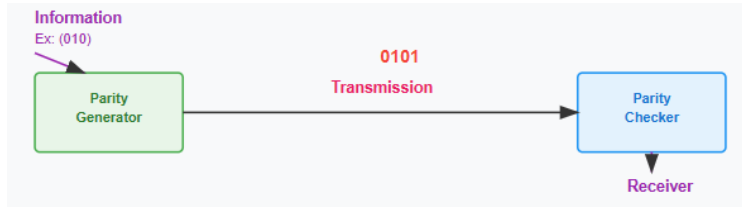
(a) Odd function $F = A \oplus B \oplus C \oplus D$

AB \ CD		C			
		00	01	11	10
A	00	m_0 1	m_1	m_3 1	m_2
	01	m_4	m_5 1	m_7	m_6 1
	11	m_{12} 1	m_{13}	m_{15} 1	m_{14}
	10	m_8	m_9 1	m_{11}	m_{10} 1
		D			

(b) Even function $F = (A \oplus B \oplus C \oplus D)'$

Parity Generation and Checking

- Exclusive-OR functions are very useful in systems requiring error detection and correction.
- A parity bit is an extra bit included with a binary message to make the number of 1's either odd or even. The message, including the parity bit, is transmitted and then checked at the receiving end for errors.
- The circuit that generates the parity bit in the transmitter is called a parity generator. The circuit that checks the parity in the receiver is called a parity checker.



Parity Generation and Checking

Parity bit **P** must be generated to make the number of 1's even if even parity is considered.

Even-Parity-Generator Truth Table

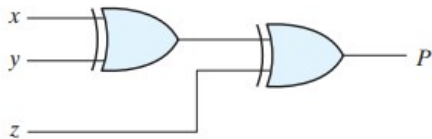
Three-Bit Message			Parity Bit
<i>x</i>	<i>y</i>	<i>z</i>	<i>P</i>
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Parity Generation and Checking

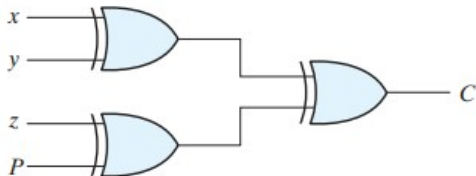
- A parity bit is an extra bit included with a binary message to make the number of 1's either odd or even. The message, including the parity bit, is transmitted and then checked at the receiving end for errors
- An error is detected if the checked parity does not correspond with the one transmitted. The circuit that generates the parity bit in the transmitter is called a parity generator. The circuit that checks the parity in the receiver is called a parity checker.

Parity Generation and Checking

Logic diagram of a parity generator and checker.



(a) 3-bit even parity generator



(b) 4-bit even parity checker

Parity Generation and Checking

Parity check

$A \oplus B \oplus C \oplus P$

Advantage- use same gates for parity generator and checker

Even-Parity-Checker Truth Table

Four Bits Received				Parity Error Check
x	y	z	P	C
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

- Alarm Systems

Scenario:

Industrial machines often have safety alarms that trigger if certain unsafe conditions occur (e.g., overheat, high pressure, door open).

Why NAND/NOR?

Safety logic can be built with a single type of universal gate:

- NAND can be configured to act as OR to trigger alarms when any unsafe condition is present.
- NAND can also be configured to act as AND to ensure alarms only trigger if multiple sensor fail simultaneously.

Processor Design (Control Logic)

Scenario: In CPUs, the control unit decides which micro-operation to perform based on opcode and status flags.

Why NAND/NOR?

At the hardware level, all combinational control logic can be implemented using only NAND or only NOR gates — reducing design complexity in IC fabrication.



THANK YOU

Team DDCO
Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Combinational Logic

Team DDCO

Department of Computer Science and Engineering

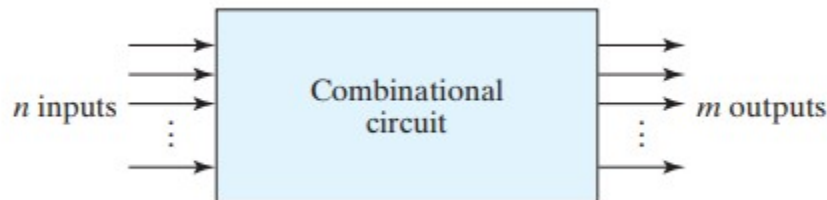
Introduction



- There are two types of Logic Circuits:
 - Combinational
 - Sequential
- Combinational Circuits consists of logic gates whose output at any time is function of the **present inputs only**.
- Sequential Circuits along with logic gates use storage elements. The value stored in these elements is due to a previous combination of inputs. Thus the present output of a sequential circuits **depends upon the present and the past inputs**.

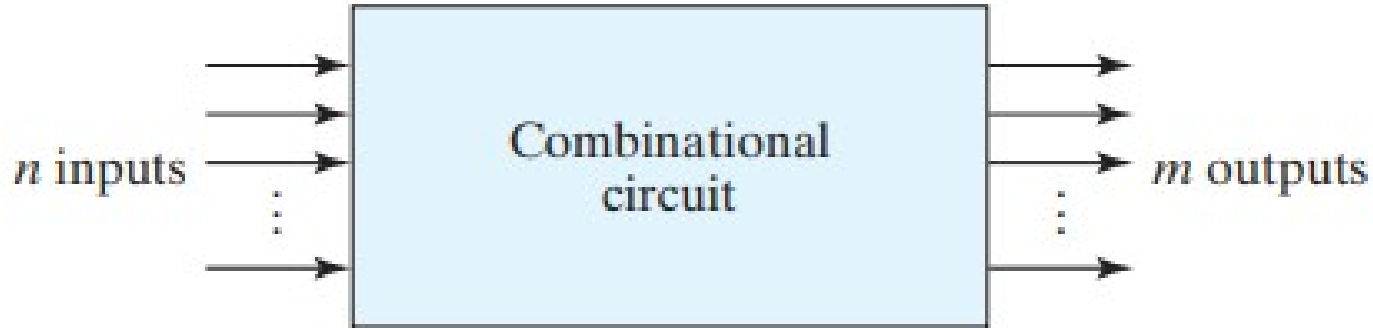
Combinational Circuits

- A combinational circuit consists of input variables, logic gates and output variables.
- For n input variables there will be a total 2^n input combinations and the pattern of m outputs w.r.t the different combination can be listed in the form of a truth table.
- A combinational circuit can be represented by a truth table having n inputs & m outputs or it can be represented by a Boolean function having m equations one for each output variable.

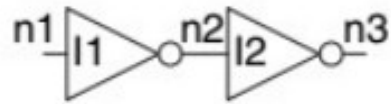


Combinational Circuits

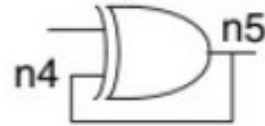
Block diagram of combinational circuit



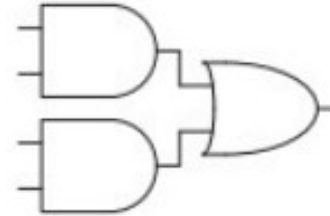
Identify the Circuit



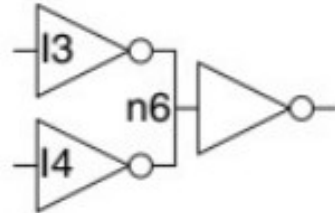
(a)



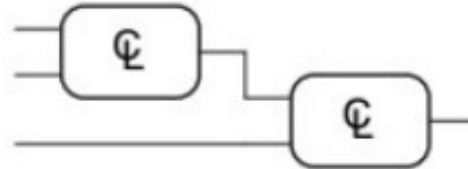
(b)



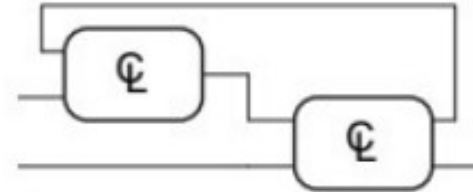
(c)



(d)



(e)



(f)

Identify the Circuit

- A. is **combinational**. It is constructed from two combinational circuit elements (inverters I1 and I2). It has three nodes: n1, n2, and n3. n1 is an input to the circuit and to I1; n2 is an internal node, which is the output of I1 and the input to I2; n3 is the output of the circuit and of I2.
- B. is **not combinational**, because there is a cyclic path: the output of the XOR feeds back to one of its inputs.
- C. is **combinational**.
- D. is **not combinational**, because node n6 connects to the output terminals of both I3 and I4.
- E. is **combinational**, illustrating two combinational circuits connected to form a larger combinational circuit.
- F. is **not combinational**, does not obey the rules of combinational composition because it has a cyclic path through the two elements.

Analysis Procedure



- The analysis starts with a given logic diagram and culminates with
 - **A set of Boolean functions**
 - **A Truth Table**
 - **or, possibly, an explanation of the circuit operation**
- The first step in the analysis is to make sure that the given circuit is **combinational and not sequential**.
- The diagram of a **combinational** circuit has logic gates with **no feedback paths or memory elements**.
- A feedback path is a connection from the output of one gate to the input of a second gate whose output forms part of the input to the first gate.

Analysis Procedure



To obtain the output Boolean functions from a logic diagram, we proceed as follows:

1. Label all gate outputs that are a function of input variables with arbitrary symbols— but with meaningful names. Determine the Boolean functions for each gate output.
2. Label the gates that are a function of input variables and previously labeled gates with other arbitrary symbols. Find the Boolean functions for these gates.
3. Repeat the process outlined in step 2 until the outputs of the circuit are obtained.
4. By repeated substitution of previously defined functions, obtain the output Boolean functions in terms of input variables.

Analysis Procedure

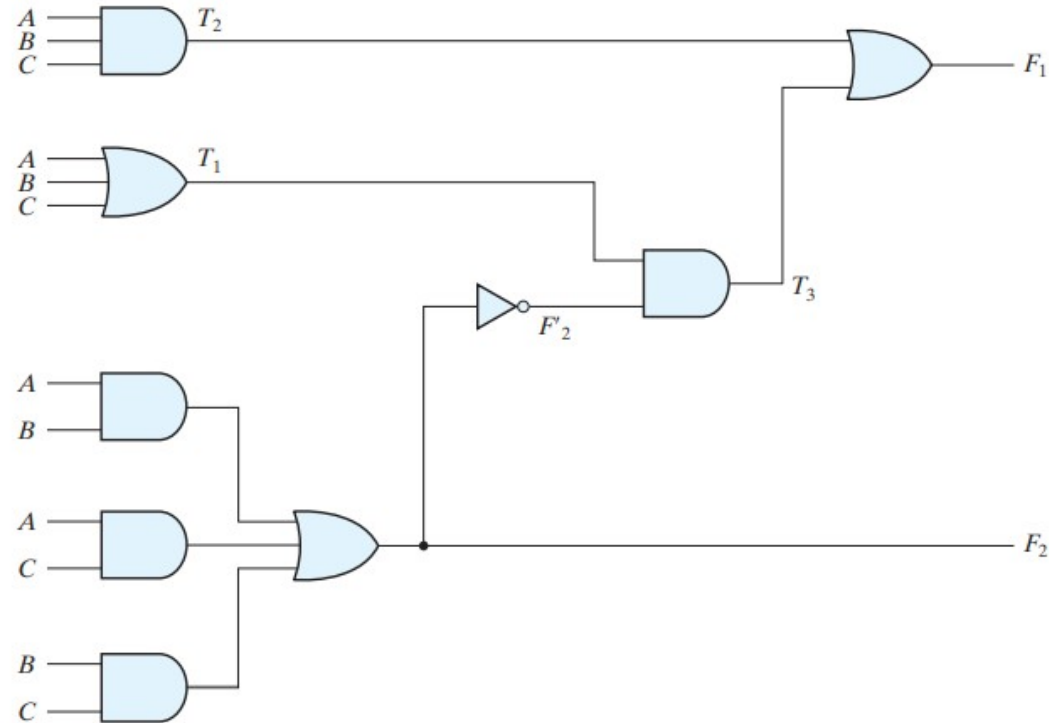


FIGURE 4.2
Logic diagram for analysis example

Analysis Procedure

The analysis of the combinational circuit of Fig. 4.2 illustrates the proposed procedure. We note that the circuit has three binary inputs— A , B , and C —and two binary outputs— F_1 and F_2 . The outputs of various gates are labeled with intermediate symbols. The outputs of gates that are a function only of input variables are T_1 and T_2 . Output F_2 can easily be derived from the input variables. The Boolean functions for these three outputs are

$$F_2 = AB + AC + BC$$

$$T_1 = A + B + C$$

$$T_2 = ABC$$

Next, we consider outputs of gates that are a function of already defined symbols:

$$T_3 = F_2' T_1$$

$$F_1 = T_3 + T_2$$

To obtain F_1 as a function of A , B , and C , we form a series of substitutions as follows:

$$\begin{aligned} F_1 &= T_3 + T_2 = F_2' T_1 + ABC = (AB + AC + BC)'(A + B + C) + ABC \\ &= (A' + B')(A' + C')(B' + C')(A + B + C) + ABC \\ &= (A' + B'C')(AB' + AC' + BC' + B'C) + ABC \\ &= A'BC' + A'B'C + AB'C' + ABC \end{aligned}$$

Analysis Procedure

The derivation of the truth table for a circuit is a straightforward process once the output Boolean functions are known. To obtain the truth table directly from the logic diagram without going through the derivations of the Boolean functions, we proceed as follows:

1. Determine the number of input variables in the circuit. For n inputs, form the 2^n possible input combinations and list the binary numbers from 0 to $(2^n - 1)$ in a table.
2. Label the outputs of selected gates with arbitrary symbols.
3. Obtain the truth table for the outputs of those gates which are a function of the input variables only.
4. Proceed to obtain the truth table for the outputs of those gates which are a function of previously defined values until the columns for all outputs are determined.

Analysis Procedure

Table 4.1
Truth Table for the Logic Diagram of Fig. 4.2

A	B	C	F₂	F'₂	T₁	T₂	T₃	F₁
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

Design Procedure



The design of combinational circuits starts from the specification of the design objective and culminates in a logic circuit diagram or a set of Boolean functions from which the logic diagram can be obtained. The procedure involves the following steps:

1. From the specifications of the circuit, determine the required number of inputs and outputs and assign a symbol to each.
2. Derive the truth table that defines the required relationship between inputs and outputs.
3. Obtain the simplified Boolean functions for each output as a function of the input variables.
4. Draw the logic diagram and verify the correctness of the design (manually or by simulation).

Code Conversion Example



It is sometimes necessary to use the **output of one system as the input to another**. A conversion circuit must be inserted between the two systems if each uses different codes for the same information. Thus, a code converter is a circuit that makes the two systems compatible even though each uses a different binary code

Code Conversion Example

Design a circuit that converts from a BCD code to excess-3 code.

Code Conversion Example

- Some circuits use excess-3 code and we need to feed this circuit with the right input.
- In excess-3 code, numbers are represented as decimal digits, and each digit is represented by four bits as the digit value plus 3

BCD stands for **Binary Coded Decimal**. Binary coded decimal (BCD) is a system of writing numerals that assigns a four-digit binary code to each digit 0 through 9 in a decimal (base-10) numeral.

BCD code is also known as the 8 4 2 1 code.

In truth table 10-15 is don't care output.

Code Conversion Example

Truth Table for Code Conversion Example

Input BCD				Output Excess-3 Code			
<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>w</i>	<i>x</i>	<i>y</i>	<i>z</i>
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Code Conversion Example

Maps for BCD-to-excess-3
code converter

$$z = D'$$

$$y = CD + C'D' = CD + (C + D)'$$

$$x = B'C + B'D + BC'D' = B'(C + D) + BC'D'$$

$$= B'(C + D) + B(C + D)'$$

$$w = A + BC + BD = A + B(C + D)$$

		C			
		00	01	11	10
A	00	m_0 1	m_1	m_3	m_2 1
	01	m_4 1	m_5	m_7	m_6 1
	11	m_{12} X	m_{13} X	m_{15} X	m_{14} X
	10	m_8 1	m_9	m_{11} X	m_{10} X

$$z = D'$$

		C			
		00	01	11	10
A	00	m_0 1	m_1	m_3 1	m_2
	01	m_4 1	m_5	m_7 1	m_6
	11	m_{12} X	m_{13} X	m_{15} X	m_{14} X
	10	m_8 1	m_9	m_{11} X	m_{10} X

$$y = CD + C'D'$$

		C			
		00	01	11	10
A	00	m_0	m_1 1	m_3 1	m_2 1
	01	m_4 1	m_5	m_7	m_6 1
	11	m_{12} X	m_{13} X	m_{15} X	m_{14} X
	10	m_8	m_9 1	m_{11} X	m_{10} X

$$x = B'C + B'D + BC'D'$$

		C			
		00	01	11	10
A	00	m_0	m_1	m_3	m_2
	01	m_4	m_5 1	m_7 1	m_6 1
	11	m_{12} X	m_{13} X	m_{15} X	m_{14} X
	10	m_8 1	m_9 1	m_{11} X	m_{10} X

$$w = A + BC + BD$$



Code Conversion Example

Not counting input inverters, the implementation in sum-of-products form requires seven AND gates and three OR gates (without simplification).

The implementation of the figure in the previous slide requires four AND gates, four OR gates, and one inverter (after simplification)

Thus, the three-level logic circuit requires fewer gates, all of which in turn require no more than two inputs.

8 4 -2 -1 Code and BCD Code

	8	4	-2	-1	BCD Code			
Dec.	A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	0	0
1	0	1	1	1	0	0	0	1
2	0	1	1	0	0	0	1	0
3	0	1	0	1	0	0	1	1
4	0	1	0	0	0	1	0	0
5	1	0	1	1	0	1	0	1
6	1	0	1	0	0	1	1	0
7	1	0	0	1	0	1	1	1
8	1	0	0	0	1	0	0	0
9	1	1	1	1	1	0	0	1

This slide is correct;
ex: For 9: $8 + 4 + (-2) + (-1)$

Gray code

Decimal	Binary Code	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Think about it

In Gray code, **only one bit** changes at a time compared to binary. What is the **Gray code equivalent** of the binary number 1101?

- A) 1111
- B) 1011
- C) 1000
- D) 1010

Think about it

In Gray code, **only one bit** changes at a time compared to binary. What is the **Gray code equivalent** of the binary number 1101?

- A) 1111
- B) 1011
- C) 1000
- D) 1010

Answer: B) 1011

Explanation:

Binary bit position	Binary bit	Rule	Gray bit
1 (MSB)	1	Copy as-is	1
2	1	$1 \oplus 1 = 0$	0
3	0	$1 \oplus 0 = 1$	1
4 (LSB)	1	$0 \oplus 1 = 1$	1

Think about it

A combinational logic circuit is designed to convert a **4-bit BCD** input into **Gray code**. If the BCD input is restricted to digits 0–9, how many valid Gray code outputs are expected?

- A) 10
- B) 15
- C) 9
- D) 16

Think about it

A combinational logic circuit is designed to convert a **4-bit BCD** input into **Gray code**. If the BCD input is restricted to digits 0–9, how many valid Gray code outputs are expected?

- A) 10
- B) 15
- C) 9
- D) 16

Answer: A) 10

Explanation:

Since BCD digits range from 0000 to 1001 (0 to 9), only **10 valid inputs** exist. So, the circuit will produce 10 valid Gray code outputs corresponding to these.

Think about it

In a BCD to Excess-3 code converter, what is the output for the BCD input **0101 (5)**?

- A) 1000
- B) 0110
- C) 1010
- D) 1100

Think about it

In a BCD to Excess-3 code converter, what is the output for the BCD input **0101 (5)**?

- A) 1000
- B) 0110
- C) 1010
- D) 1100

Answer: A) 1000

Explanation:

Excess-3 code = BCD + 0011.

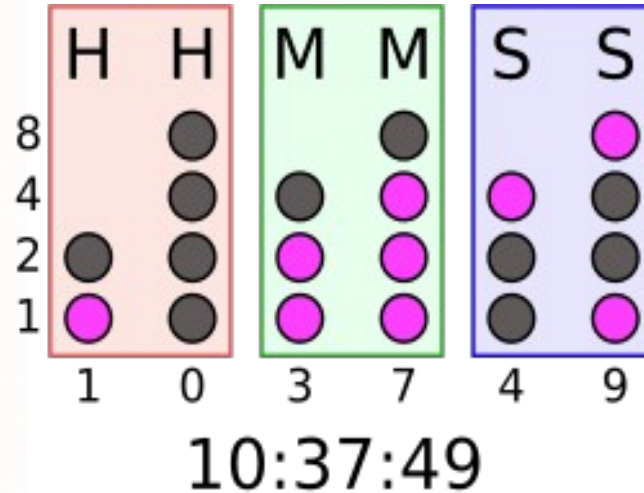
BCD(0101) + 0011 = 1000 (8).

So, the output is **1000**.

Applications

Code Conversion or Real Device

Code Conversion	Real Device
Binary to Gray	 Rotary Encoder in a Mouse or Robot Wheel
Decimal to BCD	 Digital Clock
BCD to Excess-3	 Old Electronic Calculators
Binary to ASCII	 Keyboard Typing
Binary to Parity	 Pen Drive Data Transfer





THANK YOU

Team DDCO

Department of Computer Science and Engineering



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Binary Adder Subtractor, Decimal Adder

Team DDCO

Department of Computer Science and Engineering

DIGITAL DESIGN AND COMPUTER ORGANIZATION



Adder Subtractor Decimal Adder

Department of Computer Science and Engineering

Combinational logic

Adder Subtractor

Binary Adder-Subtractor

- A binary adder–subtractor is a combinational circuit that performs the arithmetic operations of addition and subtraction with binary numbers.
- A combinational circuit that performs the **addition of two bits is called a half adder.**
- One that performs the **addition of three bits (two significant bits and a previous carry) is a full adder**
- The names of the circuits stem from the fact that two half adders can be employed to implement a full adder.

Combinational logic

Adder Subtractor

Binary Adder

- Simple addition consists of four possible elementary operations:

$$0+0 = 0$$

$$0+1 = 1$$

$$1+0 = 1$$

$$1+1 = 10 \leftarrow \text{Sum has Carry}$$

Half Adder (two '2' input bits)

Full Adder (three '3' input bits)

Combinational logic

Adder Subtractor



Half Adder

- The simplified Boolean functions for the two outputs can be obtained directly from the truth table. The simplified sum-of-products expressions are

$$S = x'y + xy'$$

$$C = xy$$

Combinational logic

Adder Subtractor

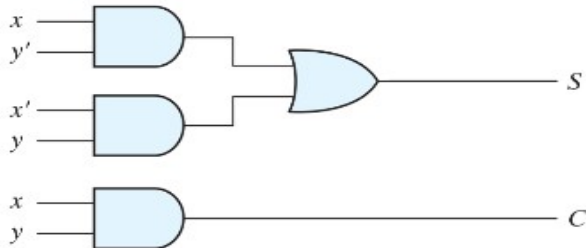
Truth Table of Half Adder

<i>x</i>	<i>y</i>	<i>c</i>	<i>s</i>
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

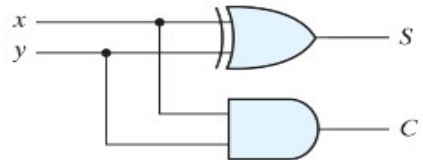
Combinational logic

Adder Subtractor

Implementation of half adder.



$$(a) \begin{aligned} S &= xy' + x'y \\ C &= xy \end{aligned}$$



$$(b) \begin{aligned} S &= x \oplus y \\ C &= xy \end{aligned}$$

Combinational logic

Adder Subtractor

Full Adder

- A full adder is a combinational circuit that forms the arithmetic sum of three bits. It consists of three inputs and two outputs. Two of the input variables, denoted by x and y , represent the two significant bits to be added.

Combinational logic

Adder Subtractor

Truth Table of Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Combinational logic

Adder Subtractor

K-map for full adder.

		y			
		yz		11	10
x	0	m_0	m_1 1	m_3	m_2 1
	1	m_4 1	m_5	m_7 1	m_6

(a) $S = x'y'z + x'yz' + xy'z' + xyz$

		y			
		yz		11	10
x	0	m_0	m_1	m_3 1	m_2
	1	m_4	m_5 1	m_7 1	m_6 1

(b) $C = xy + xz + yz$

Combinational logic

Adder Subtractor

Full Adder

- The S output from the second half adder is the exclusive-OR of z and the output of the first half adder, giving

$$S = z \oplus (x \oplus y)$$

$$= z'(xy' + x'y) + z(xy' + x'y)'$$

$$= z'(xy' + x'y) + z(xy + x'y')$$

$$= xy'z' + x'yz' + xyz + x'y'z$$

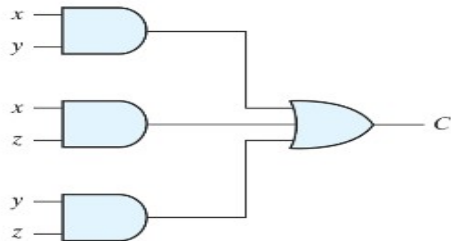
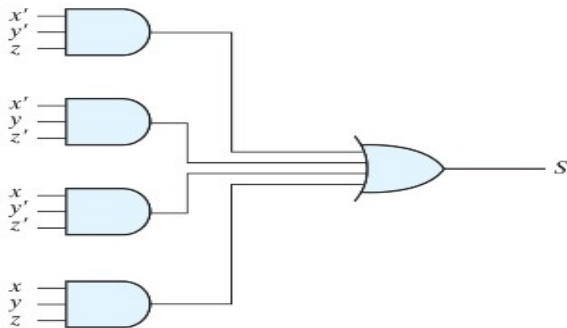
The carry output is

$$C = z(xy' + x'y) + xy = xy'z + x'yz + xy$$

Combinational logic

Adder Subtractor

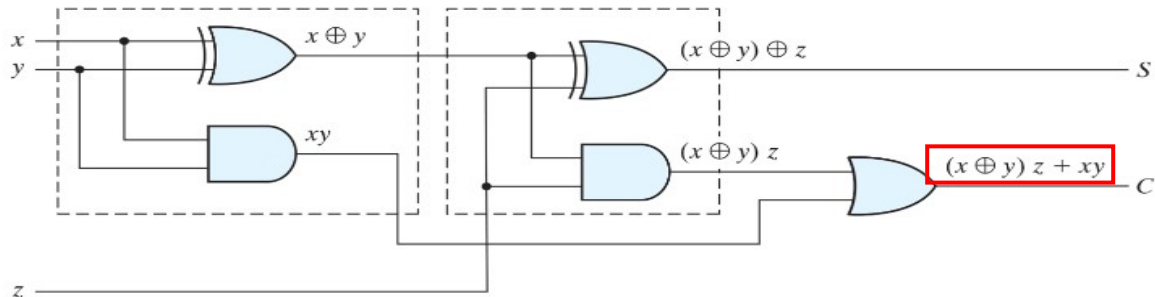
Implementation of full adder in sum-of-products form.



Combinational logic

Adder Subtractor

Implementation of full adder with two half adders and an OR



Combinational logic

Adder Subtractor

Full Adder

- Full adders are used in the Arithmetic Logic Unit (ALU) of processors to perform binary addition in devices like computers, smartphones, and calculators.

Binary Adder

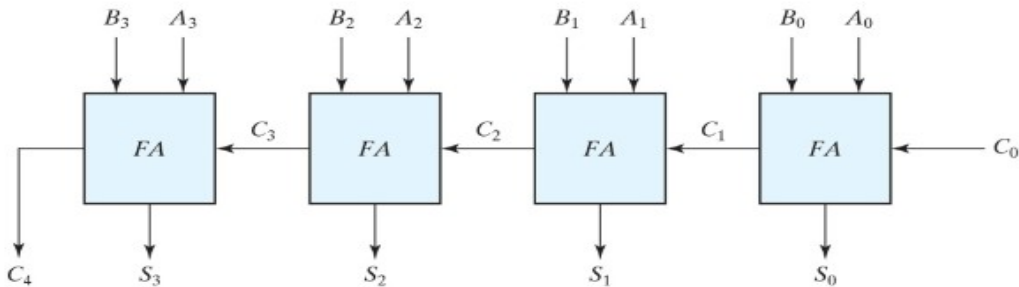
A binary adder is a digital circuit that produces the arithmetic sum of two binary numbers. It can be constructed with full adders connected in cascade, with the output carry from each full adder connected to the input carry of the next full adder in the chain

Addition of n -bit numbers requires a chain of n full adders or a chain of one-half adder and $n - 1$ full adders.

Combinational logic

Adder Subtractor

Implementation of Four-bit binary adder can be implemented by using 4 full adders $2^9 = 512$ entries (classical method, by using adders it can be avoided)



Binary Adder

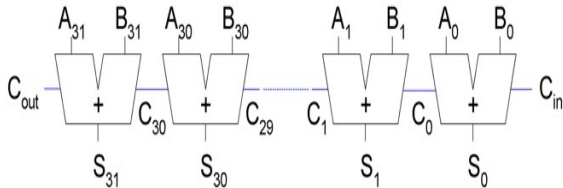
- Consider a 4-bit adder. Let's denote A_i and B_i as the input bits of the two binary numbers and C_i is the carry from previous bit addition of the numbers.
- We also let S_i be the sum and C_{i+1} the carry output in the current addition.

Subscript i :	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}

Combinational logic

Ripple Carry Adder

- Chain 1-bit adders together
- Carry ripples through entire chain
- Disadvantage: **slow**
- In ripple carry adders, for each adder block, the two bits that are to be added are available instantly.
- However, each adder block waits for the carry to arrive from its previous block.



- The Cout of one stage acts as the Cin of the next stage
- The i th block waits for the $(i-1)$ th block to produce its carry.
- So there will be a considerable time delay which is carry propagation delay.
- The propagation time is equal to the propagation delay of each adder block, multiplied by the number of adder blocks in the circuit.
- Time complexity:

$$t_{\text{ripple}} = N t_{\text{FA}}$$

where t_{FA} is the delay of a
1-bit full adder

For example, if each full adder stage has a propagation delay of 20 nanoseconds, then will reach its final correct value after 80 (20×4) nanoseconds.

The situation gets worse, if we extend the number of stages for adding more number of bits.

The ripple carry adder has the disadvantage of being slow when N is large

Ripple-carry addition requires $\Theta(n)$ time because of the rippling of carry bits through the circuit.

Instead of generating and propagating carry bit-by-bit, **can we generate all of them in parallel** and break the sequential chain?

CLA is another type of carry propagate adder that solves this problem by **dividing the adder into blocks** and providing circuitry to quickly determine the carry out of a block as soon as the carry in is known.

Thus it is said **to look ahead across the blocks rather than waiting to ripple through all the full adders inside a block**

In Ripple Carry Adder, each full adder has to wait for its carry-in from its previous stage full adder.

Thus, n^{th} full adder has to wait until all **(n-1) full adders** have completed their operations.

This causes a delay and makes ripple carry adder extremely slow.

The situation becomes worst when the value of n becomes very large.
To overcome this disadvantage, Carry Look Ahead Adder comes into play
Carry Look Ahead Adder is an improved version of the ripple carry adder.
It generates the carry-in of each full adder simultaneously without causing any delay.

- In addition of two binary numbers in parallel implies that all the bits of the augend and addend are available for computation at the same time.
- The total propagation time is equal to the propagation delay of a typical gate, times the number of gate levels in the circuit.
- Carry propagation time is an important attribute of the adder because it limit the speed with which two numbers are added. Consider the circuit of the full adder

Combinational logic

Carry Propagation

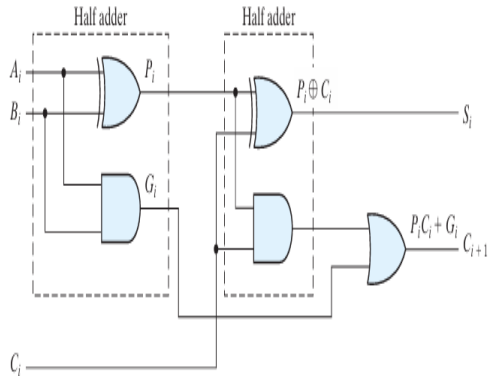


FIGURE 4.10

Full adder with P and G shown

- A block is said to generate a carry if it produces a carry out independent of the carry in to the block
- **Column i will generate a carry out if A_i AND B_i are both 1.**

$$G_i = A_i B_i$$

- G_i produces the carry when both A_i , B_i are 1 regardless of the input carry.
- The block is said to propagate a carry if it produces a carry out whenever there is a carry in to the block
- The i th column will propagate a carry C_i , if either A_i or B_i is 1.

$$\text{Thus, } P_i = A_i \text{ XOR } B_i.$$

Combinational logic

Carry Propagation

Full adder with P and G shown.

These two signals are common to all half adders and depend on only the input augend and addend bits. The signal from the input carry C_i to the output carry C_{i+1} propagates through an AND gate and an OR gate, which constitute two gate levels. If there are four full adders in the adder, the output carry C_4 would have $2 * 4 = 8$ gate levels from C_0 to C_4 . For an n -bit adder, **there are $2n$ gate levels for the carry to propagate from input to output.**

Combinational logic

Carry Propagation

- Consider the circuit of the full adder. If we define two new binary variables

$$P_i = A_i \oplus B_i$$
$$G_i = A_i B_i$$

output sum and carry can respectively be expressed as

$$S_i = P_i \oplus C_i$$
$$C_{i+1} = G_i + P_i C_i$$

- Pi=carry propagate
- Gi=Carry generate
- Carry lookahead logic-most widely used technique to reduce the carry delay time

Carry Propagation

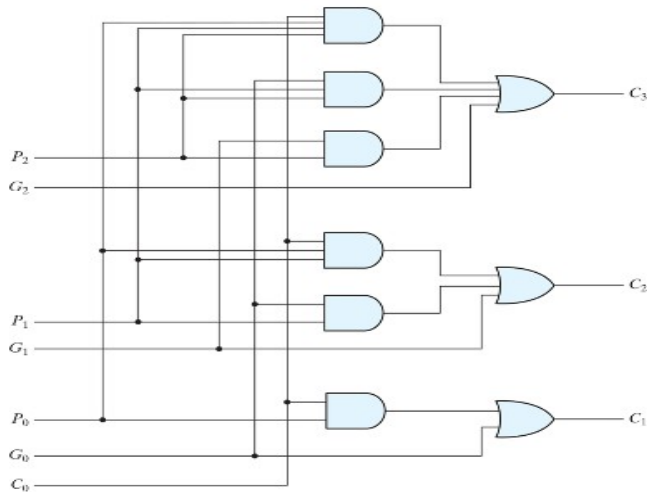
C_0 = input carry

$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1C_1 = G_1 + P_1(G_0 + P_0C_0) = G_1 + P_1G_0 + P_1P_0C_0$$

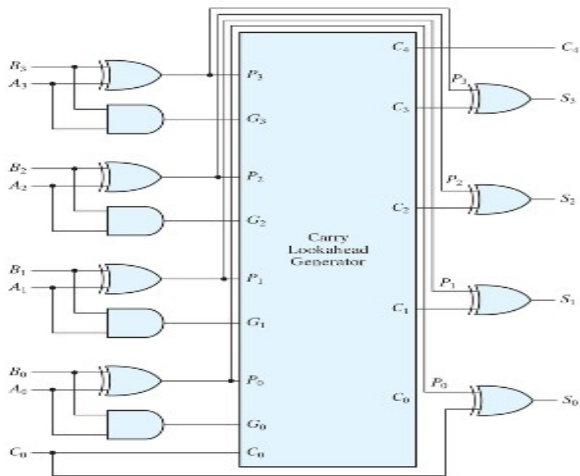
$$C_3 = G_2 + P_2C_2 = G_2 + P_2G_1 + P_2P_1G_0 = P_2P_1P_0C_0$$

Logic diagram of carry lookahead generator



Combinational logic

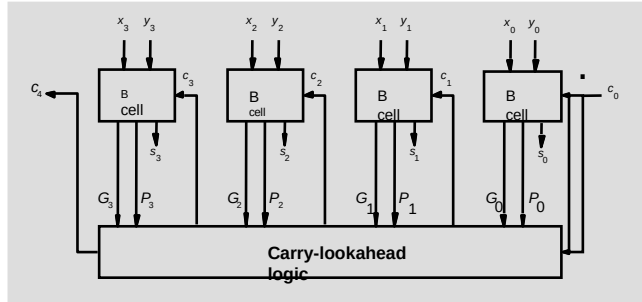
Carry Propagation Four-bit adder with carry lookahead.



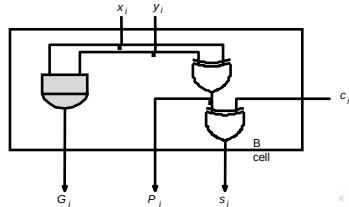
Combinational logic

Carry Look Ahead Adder

4 bit Carry-lookahead adder



4-bit
carry-lookahead
adder

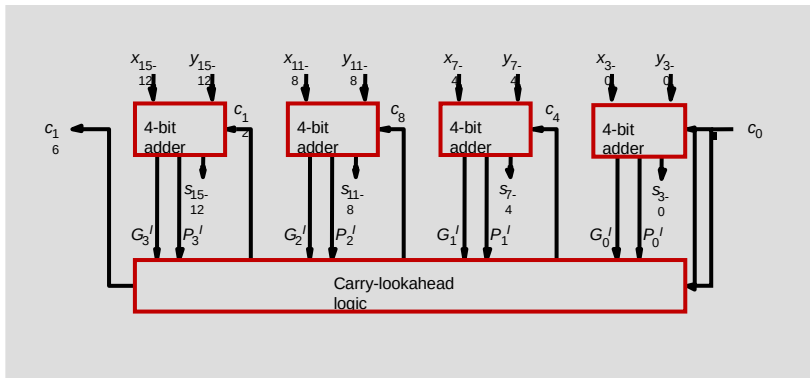


B-cell for a single stage

Combinational logic

Carry Look Ahead Adder

16- bit Block Carry-Look ahead adder



Binary Subtractor

- Binary Subtraction is the process of finding the difference between two binary numbers. Instead of directly subtracting, it is implemented by adding the 2's complement of the subtrahend to the minuend
- This method allows the same hardware circuit to perform both addition and subtraction, making it efficient and widely used in digital computers and processors.

Combinational logic

Adder Subtractor

Binary Subtractor

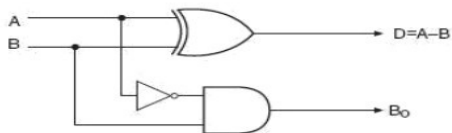
$$D = \overline{A}.B + A.\overline{B}$$

$$B_o = \overline{A}.B$$



A	B	D	B _o
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Half Subtractor

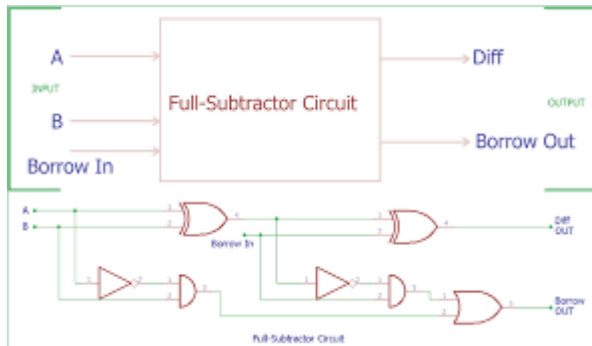


Combinational logic

Adder Subtractor

Full subtractor

Inputs			Outputs	
A	B	Borrow _{in}	Diff	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



Combinational logic

Adder Subtractor



Adder-subtractor circuit

A-B is equal to $A + 2$'s complement of B = $a + (1$'s complement of B + 1)

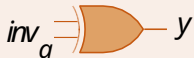
For unsigned numbers, that gives $A - B$ if $A \geq B$ or the 2's complement of $(B - A)$ if $A < B$. For signed numbers, the result is $A - B$, provided that there is no overflow.

XOR as Controlled Inverter

- Truth table:

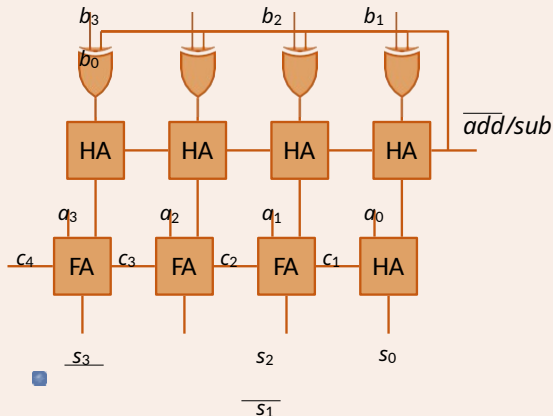
inv	a	y
0	0	0
0	1	1
1	0	1
1	1	0

- Symbol:



- When $inv = 0$, $y = a$
- When $inv = 1$, $y = \bar{a}$

Two's Complement Adder / Subtractor



Subtraction when $\overline{add/sub} = 1$

Addition when $\overline{add/sub} = 0$

Combinational logic

Adder Subtractor

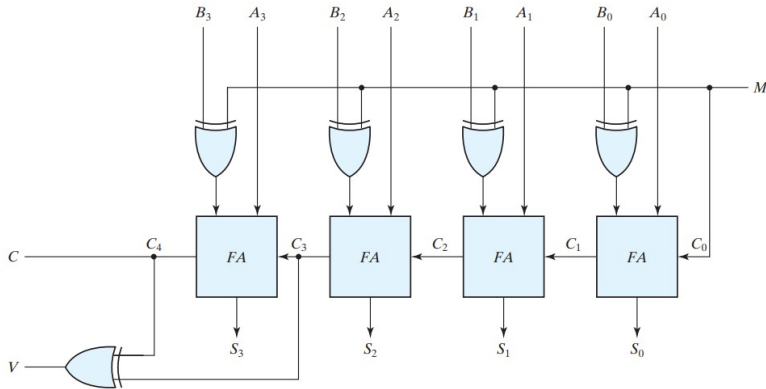


FIGURE 4.13

Four-bit adder-subtractor (with overflow detection)

Combinational logic

Adder Subtractor

Overflow condition

When two numbers with n digits each are added and the sum is a number occupying $n + 1$ digits, we say that an overflow occurred

The detection of an overflow after the addition of two binary numbers depends on whether the numbers are considered to be signed or unsigned.

When two **unsigned numbers** are added, an overflow is detected from the end carry out of the most **significant position**.

EX $A-B = 1001 - 0110 = 1_0011$

Signed numbers

MSB(leftmost bit)- sign

Negative numbers are represented in 2's complement.

Overflow will not occur if opposite sign

Overflow will occur if same sign

Combinational logic

Overflow Condition



carries:	0	1
+70	0	1000110
+80	0	1010000
+150	1	0010110

carries:	1	0
-70	1	0111010
-80	1	0110000
-150	0	1101010

+70 and +80 are eight bit numbers Range: +127 to -128

70 + 80=150 - not in range

-70-80= -150 not in range

Note that the eight-bit result that should have been positive has a negative sign bit (i.e., the eighth bit) and the eight-bit result that should have been negative has a positive sign bit. If, however, the carry out of the sign bit position is taken as the sign bit of the result, then the nine-bit answer so obtained will be correct. But since the answer cannot be accommodated within eight bits, we say that an overflow has occurred

ADDER, SUBTRACTOR, OVERFLOW - 4

Two's complement addition

No overflow when
numbers have opposite signs
(or one/both are zero)

Overflow can occur when

When both positive
msb is 1 (c_{msb} is 1)

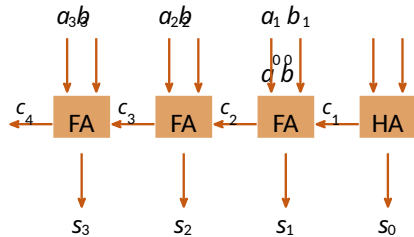
c_{msb-1} is 1 and c_{msb} is 0

When both negative

msb is 0 (c_{msb-1} is 0)

c_{msb-1} is 0 and c_{msb} is 1

overflow = $c_{msb} \oplus c_{msb-1}$



Gate-Level Minimization and Combinational logic

Adder Subtractor with Overflow Detection

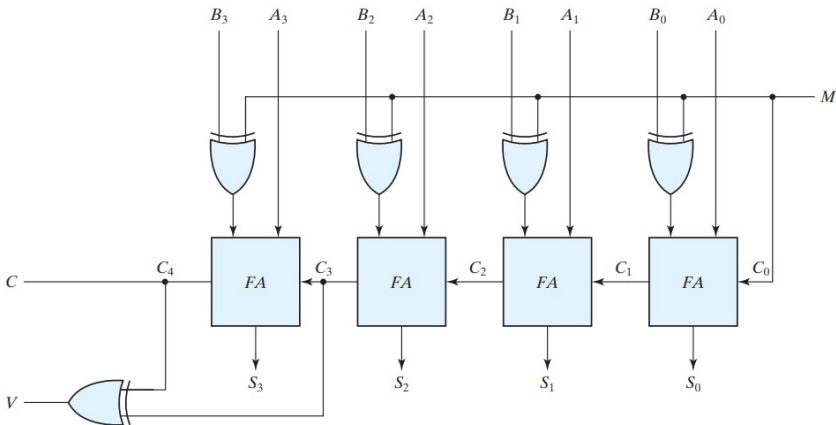


FIGURE 4.13

Four-bit adder-subtractor (with overflow detection)

Combinational logic

Decimal Adder-BCD Adder



Decimal Adder- BCD adder

Binary to BCD conversion

BCD-0 to 9

Add 6 to convert the number from Binary to BCD(8421 representation)
if it exceeds 9

The number 6 is chosen because it is the smallest value that, when added to a binary number between 10 (1010) and 15 (1111), produces a valid BCD representation.

BCD addition

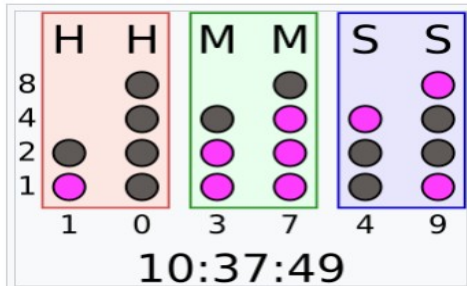
45+28

M. Morris Mano, Michael D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, 5th ed., Prentice Hall, 2012, Section 4.6.

Combinational logic

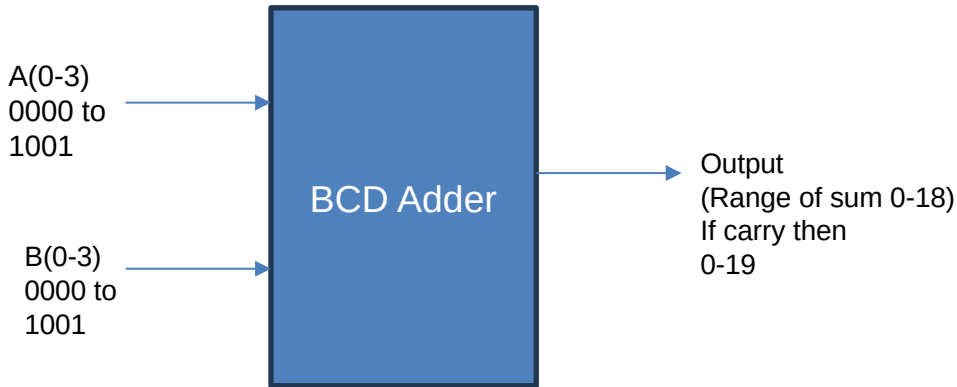
Decimal Adder-BCD Adder

Why BCD?



Reading a **binary-coded decimal** clock: Add the values of each column of **LEDs** to get six decimal digits. There are two columns each for hours, minutes and seconds.

BCD adder



Combinational logic

Decimal Adder-BCD Adder

Since each input digit does not exceed 9, the output sum cannot be greater than $9 + 9 + 1 = 19$, the 1 in the sum being an input carry



Table 4.5

Derivation of BCD Adder

Binary Sum					BCD Sum					Decimal
K	Z ₈	Z ₄	Z ₂	Z ₁	C	S ₈	S ₄	S ₂	S ₁	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

Combinational logic

Decimal Adder-BCD Adder

Since each input digit does not exceed 9, the output sum cannot be greater than $9 + 9 + 1 = 19$, the 1 in the sum being an input carry



Table 4.5

Derivation of BCD Adder

Binary Sum					BCD Sum					Decimal
K	Z ₈	Z ₄	Z ₂	Z ₁	C	S ₈	S ₄	S ₂	S ₁	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

In examining the contents of the table, it becomes apparent that when the binary sum is equal to or less than 1001, the corresponding BCD number is identical, and therefore no conversion is needed. When the binary sum is greater than 1001, we obtain an invalid BCD representation. The addition of binary 6 (0110) to the binary sum converts it to the correct BCD representation and also produces an output carry as required.

The logic circuit that detects the necessary correction can be derived from the entries in the table. It is obvious that a correction is needed when the binary sum has an output carry $K=1$. The other six combinations from 1010 through 1111 that need a correction have a 1 in position Z_8 . To distinguish them from binary 1000 and 1001, which also have a 1 in position Z_8 , we specify further that either Z_4 or Z_2 must have a 1. The condition for a correction and an output carry can be expressed by the Boolean function

$$C = K + Z_8 Z_4 + Z_8 Z_2$$

Combinational logic

Decimal Adder-BCD Adder

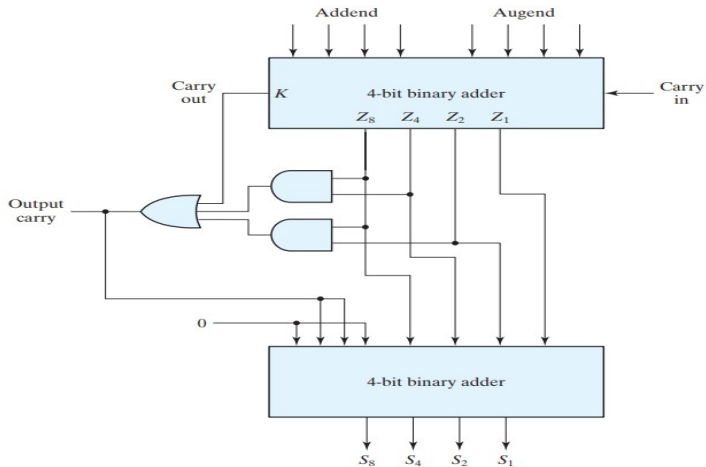


FIGURE 4.14
Block diagram of a BCD adder

A BCD adder that adds two BCD digits and produces a sum digit in BCD . The two decimal digits, together with the input carry, are first added in the top four-bit adder to produce the binary sum. **When the output carry is equal to 0, nothing is added to the binary sum. When it is equal to 1, binary 0110 is added to the binary sum through the bottom four-bit adder.**

The output carry generated from the bottom adder can be ignored, since it supplies information already available at the output carry terminal. A decimal parallel adder that adds n decimal digits needs n BCD adder stages. The output carry from one stage must be connected to the input carry of the next higher order stage.

. What is the Boolean expression used to detect correction in a BCD adder?

A) $C = K + Z_8 Z_4 + Z_8 Z_2$

B) $C = K + Z_4 Z_2$

C) $C = K \cdot Z_8 C$

D) $C = Z_8 + Z_4 + Z_2$

In a 4-bit ripple carry adder, the worst-case carry propagation delay occurs when:

A) All input bits are 0

B) Only LSB has 1 and rest are 0

C) All inputs are 1

D) Inputs cause a carry to propagate through all stages

. What is the Boolean expression used to detect correction in a BCD adder?

A) $C = K + Z_8 Z_4 + Z_8 Z_2$

B) $C = K + Z_4 Z_2$

C) $C = K \cdot Z_8 C$

D) $C = Z_8 + Z_4 + Z_2$

Ans: A

In a 4-bit ripple carry adder, the worst-case carry propagation delay occurs when:

A) All input bits are 0

B) Only LSB has 1 and rest are 0

C) All inputs are 1

D) Inputs cause a carry to propagate through all stages

Answer: D) Inputs cause a carry to propagate through all stages

Combinational logic

Think about it

In 2's complement representation, what is the result of $1001-0110$?

- A) 0011
- B) 1111
- C) 1011
- D) 0111

In a BCD adder, a correction factor is added when:

- A) The binary sum exceeds 1111
- B) The binary sum exceeds 1001
- C) The output carry is 0
- D) The binary sum is less than 1001

Combinational logic

Think about it



In 2's complement representation, what is the result of $1001-0110$?

- A) 0011
- B) 1111
- C) 1011
- D) 0111

Answer: A) 0011

In a BCD adder, a correction factor is added when:

- A) The binary sum exceeds 1111
- B) The binary sum exceeds 1001
- C) The output carry is 0
- D) The binary sum is less than 1001

Answer: B) The binary sum exceeds 1001



THANK YOU

Team DDCO
Department of Computer Science

DIGITAL DESIGN AND COMPUTER ORGANIZATION



Binary Multipliers and Magnitude Comparator

Department of Computer Science and Engineering

Combinational logic

Binary Multipliers

- Multiplication of binary numbers is performed in the same way as multiplication of decimal numbers.
- The **multiplicand is multiplied by each bit of the multiplier, starting from the least significant bit.** Each such multiplication forms a **partial product.** Successive partial products are shifted one position to the left.
- The final product is obtained from the sum of the partial products.

M. Morris Mano, Michael D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, 5th ed., Prentice Hall, 2012, Section 4.7

Combinational logic

Binary Multipliers

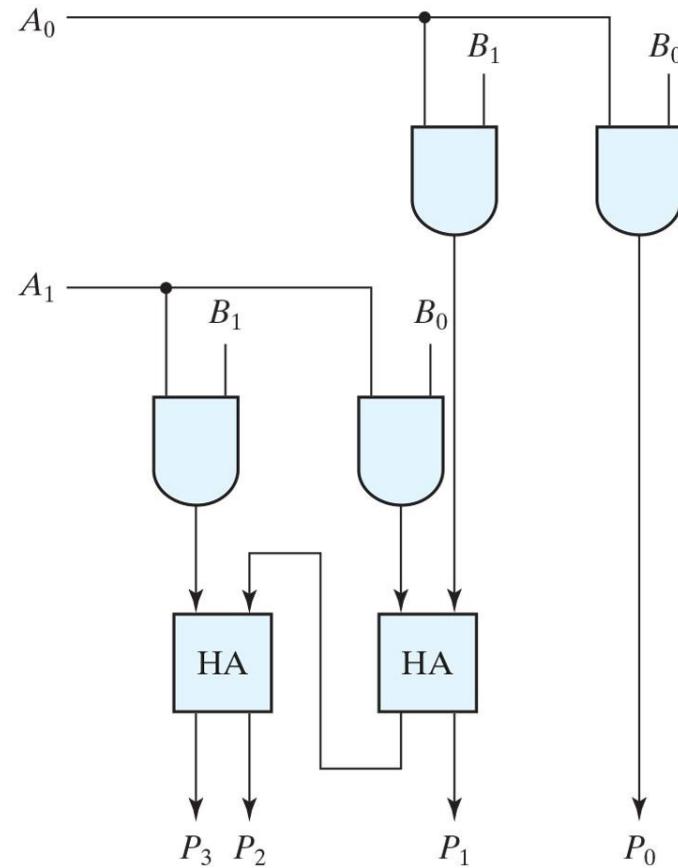
1 0 1 0	→	Multiplicand
× 1 0 1 1	→	Multiplier
1 0 1 0	→	Partial product 1
1 0 1 0	→	Partial product 2
0 0 0 0	→	Partial product 3
1 0 1 0	→	Partial product 4
1 1 0 1 1 1 0		

Combinational logic

Binary Multipliers

Two-bit by two-bit binary multiplier.

$$\begin{array}{r} B_1 \quad B_0 \\ A_1 \quad A_0 \\ \hline A_0 B_1 \quad A_0 B_0 \\ A_1 B_1 \quad A_1 B_0 \\ \hline P_3 \quad P_2 \quad P_1 \quad P_0 \end{array}$$



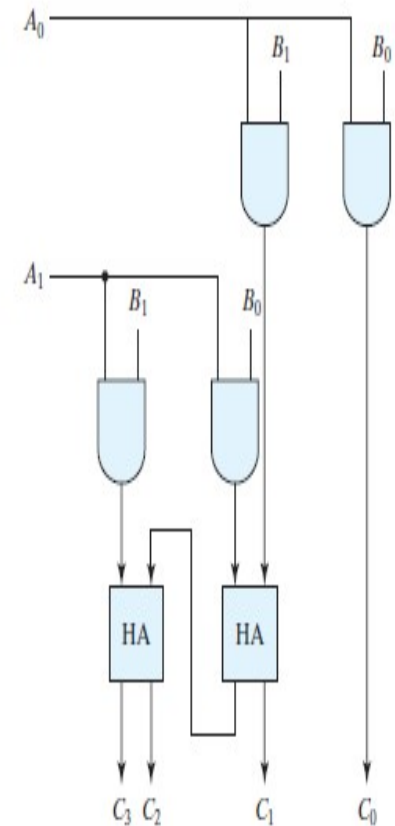
Combinational logic

Binary Multipliers

Consider the multiplication of two 2-bit numbers

- The multiplicand bits are B_1 and B_0 , the multiplier bits are A_1 and A_0 , and the product is $C_3C_2C_1C_0$.
- **The first partial product is formed by multiplying B_1B_0 by A_0 .**
- The multiplication of two bits such as A_0 and B_0 produces a 1 if both bits are 1; otherwise, it produces a 0. This is identical to an **AND** operation. Therefore, the partial product can be implemented with AND gates as shown in the diagram.

$$\begin{array}{r} B_1 \quad B_0 \\ A_1 \quad A_0 \\ \hline A_0B_1 \quad A_0B_0 \\ A_1B_1 \quad A_1B_0 \\ \hline C_3 \quad C_2 \quad C_1 \quad C_0 \end{array}$$

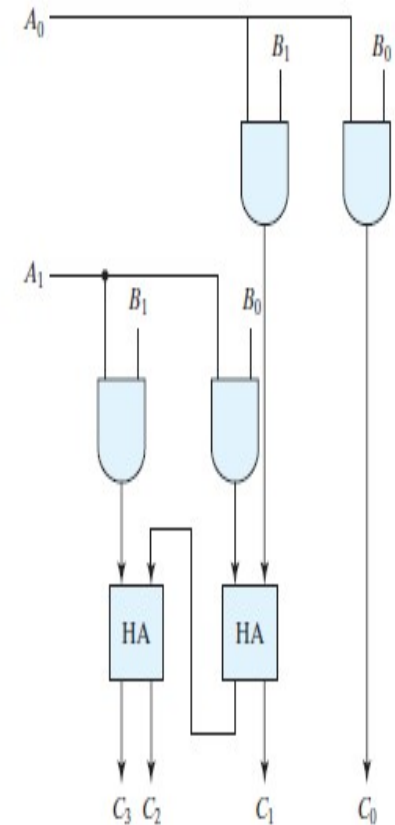


Combinational logic

Binary Multipliers

- The second partial product is formed by **multiplying** B_1B_0 by A_1 and shifting one position to the left.
- The **two partial products** are added with two **half-adder (HA) circuits**. Usually, there are more bits in the partial products and it is necessary to use full adders to produce the sum of the partial products.
- Note that the least significant bit of the product does not have to go through an adder, since it is formed by the output of the first AND gate.

$$\begin{array}{r} B_1 \quad B_0 \\ A_1 \quad A_0 \\ \hline A_0B_1 \quad A_0B_0 \\ A_1B_1 \quad A_1B_0 \\ \hline C_3 \quad C_2 \quad C_1 \quad C_0 \end{array}$$



Combinational logic

Binary Multipliers

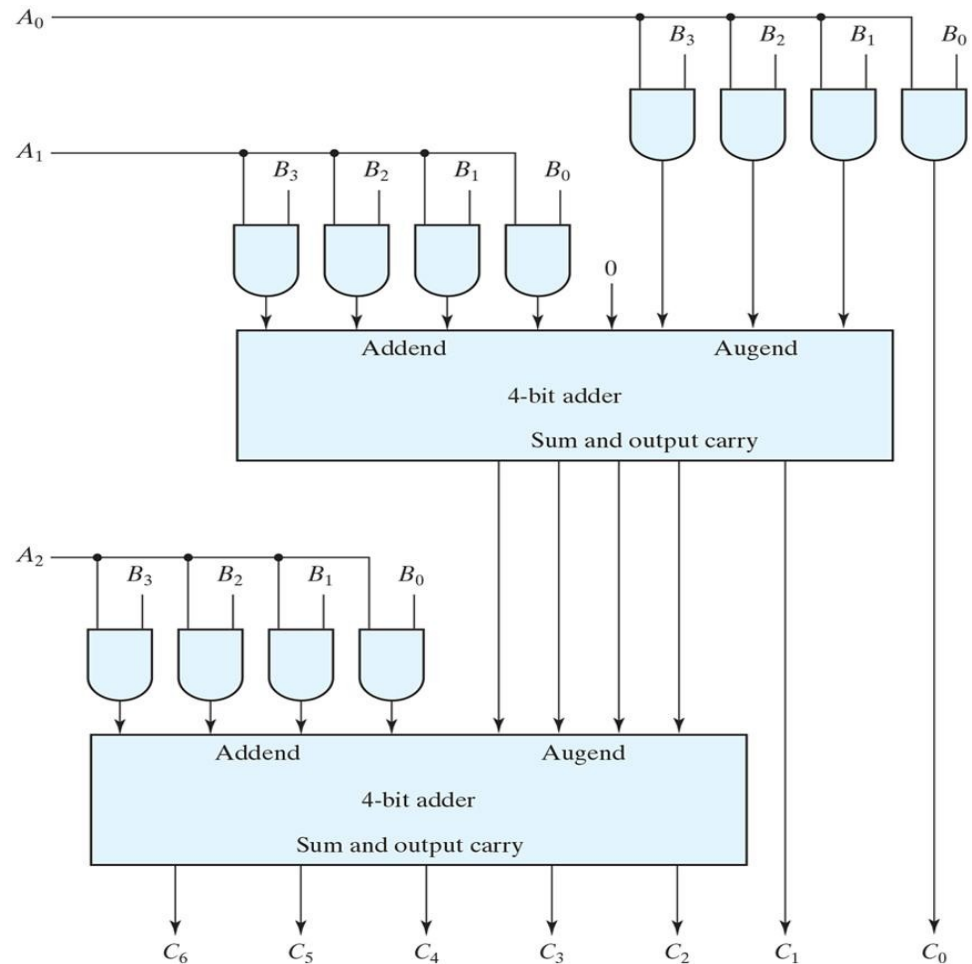
- A combinational circuit binary multiplier with more bits can be constructed in a similar fashion.
- A bit of the multiplier is ANDed with each bit of the multiplicand in as many levels as there are bits in the multiplier.
- The binary output in each level of AND gates is added with the partial product of the previous level to form a new partial product. The last level produces the product.
- For J multiplier bits and K multiplicand bits, we need $J * K$ AND gates and
- $(J - 1) K$ -bit adders to produce a product of $(J + K)$ bits.

Combinational logic

Binary Multipliers- 1011 X 001

Four-bit by three-bit binary multiplier.

- Consider a multiplier circuit that multiplies a binary number represented by four bits by a number represented by three bits.
- Let the multiplicand be represented by $B_3B_2B_1B_0$ and the multiplier by $A_2A_1A_0$.
- Since $K = 4$ and $J = 3$, we need 12 AND gates and two 4-bit adders to produce a product of seven bits



Combinational logic

Magnitude Comparators

- **A magnitude comparator is a combinational circuit that compares two numbers A and B and determines their relative magnitudes. The outcome of the comparison is specified by three binary variables that indicate whether $A > B$, $A = B$, or $A < B$.**

- On the one hand, the circuit for comparing two n -bit numbers has 2^{2n} entries in the truth table and becomes too cumbersome, even with $n = 3$

- Consider two numbers, A and B , with four digits each. Write the coefficients of the numbers in descending order of significance:

$$A = A_3 A_2 A_1 A_0$$

$$B = B_3 B_2 B_1 B_0$$

- **The two numbers are equal if all pairs of significant digits are equal:**

- **$A_3 = B_3$, $A_2 = B_2$, $A_1 = B_1$, and $A_0 = B_0$.**

M. Morris Mano, Michael D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, 5th ed., Prentice Hall, 2012, Section 4.8

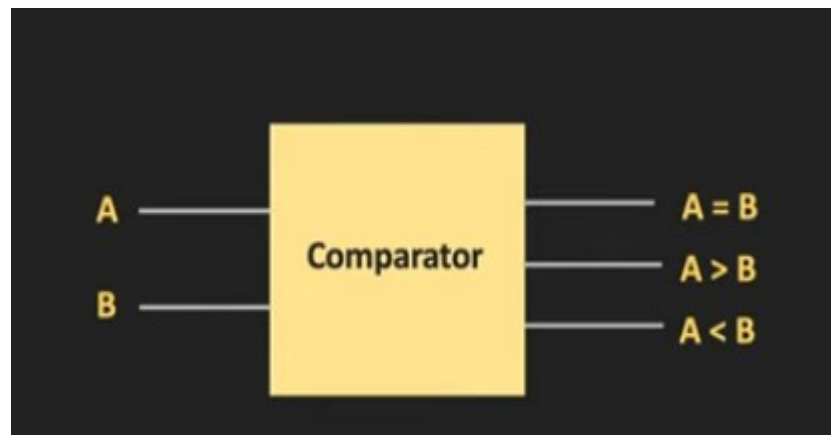
Combinational logic

Magnitude Comparators

- When the numbers are binary, the digits are either 1 or 0, and the equality of each pair of bits can be expressed logically with an exclusive-NOR function as

$$x_i = A_i B_i + A_i' B_i' \text{ for } i = 0, 1, 2, 3$$

where $x_i = 1$ only if the pair of bits in position i are equal (i.e., if both are 1 or both are 0).



Combinational logic

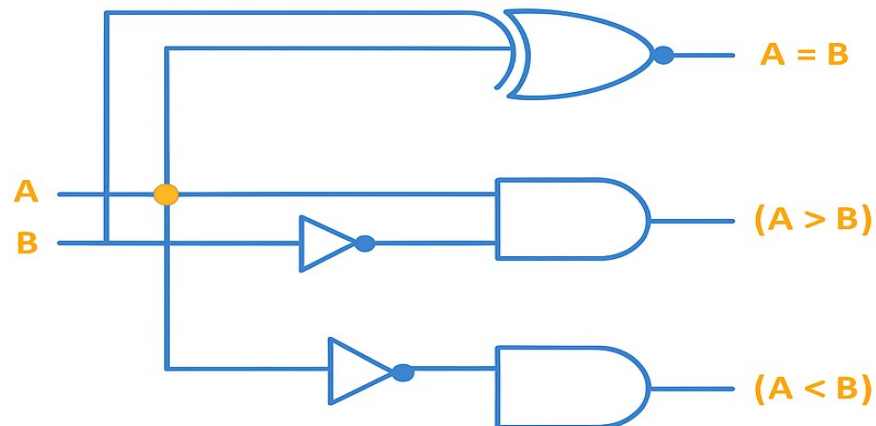
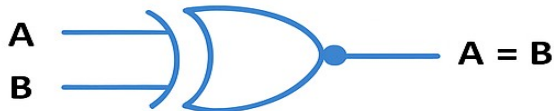
Magnitude Comparators

1-bit Comparator

Truth Table

A	B	A = B	A > B	A < B
0	0	1	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0

$$\text{Output} = \overline{A} \overline{B} + AB$$



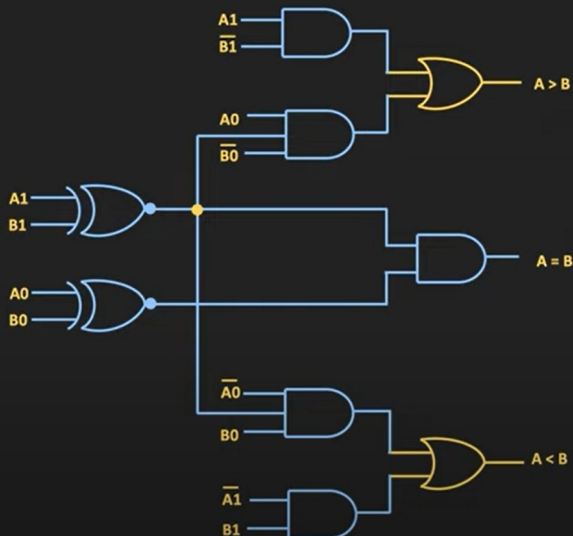
2 bit comparator

1- bit Comparator $\rightarrow$ 2 variables $\rightarrow$ 4 rows

2- bit Comparator $\rightarrow$ 4 variables $\rightarrow$ 16 rows

n- bit Comparator $\rightarrow$ 2n variables $\rightarrow$ 2^{2n} rows

2-bit Comparator



A1	A0	B1	B0	A > B	A < B	A = B
0	0	0	0	0	0	1
0	0	0	1	0	1	0
0	0	1	0	0	1	0
0	0	1	1	0	1	0
0	1	0	0	1	0	0
0	1	0	1	0	0	1
0	1	1	0	0	1	0
0	1	1	1	0	1	0
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	0	1
1	0	1	1	0	1	0
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	0	1

Combinational logic

Magnitude Comparators

4 Bit Comparator

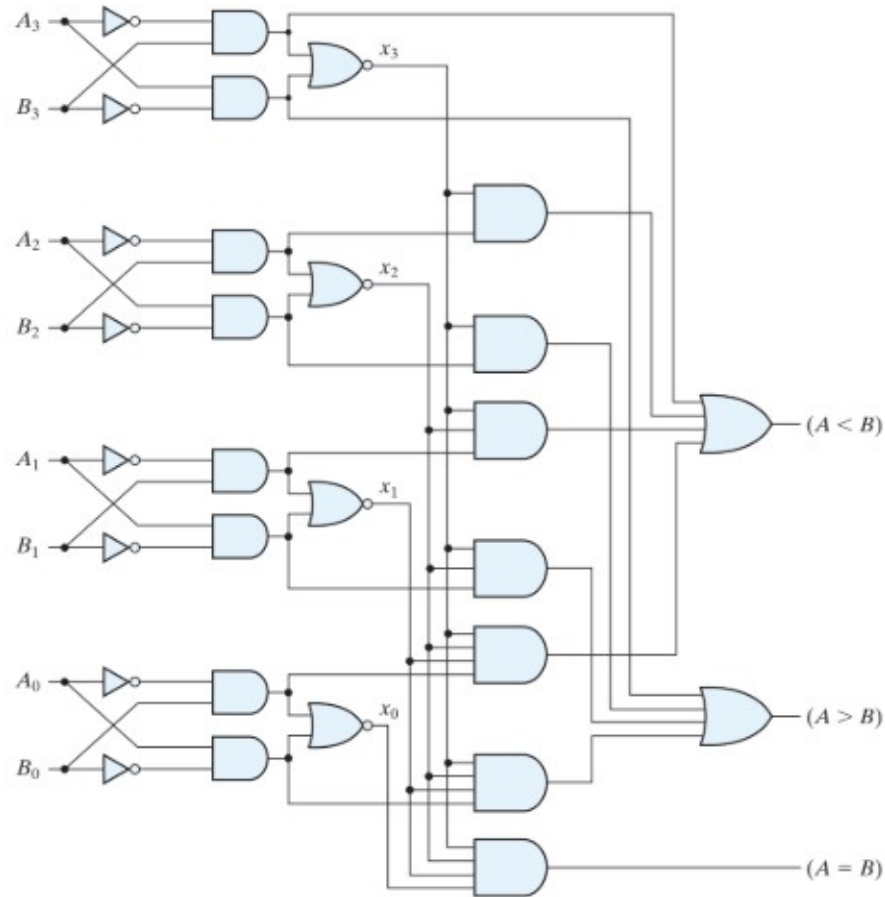


FIGURE 4.17
Four-bit magnitude comparator

$$A = A_3 A_2 A_1 A_0$$

$$B = B_3 B_2 B_1 B_0$$

$$A > B \rightarrow A_3 > B_3$$

OR

$$\text{If } A_3 = B_3 \text{ and } A_2 > B_2$$

OR

$$\text{If } A_3 = B_3 \text{ and } A_2 = B_2 \text{ and } A_1 > B_1$$

OR

$$\text{If } A_3 = B_3 \text{ and } A_2 = B_2 \text{ and } A_1 = B_1 \text{ and } A_0 > B_0$$

$$(A > B) = A_3 \overline{B_3} + (A_3 \odot B_3) A_2 \overline{B_2} + (A_3 \odot B_3) (A_2 \odot B_2) A_1 \overline{B_1} + (A_3 \odot B_3) (A_2 \odot B_2) (A_1 \odot B_1) A_0 \overline{B_0}$$

$$(A < B) = A'_3 B_3 + x_3 A'_2 B_2 + x_3 x_2 A'_1 B'_1 + x_3 x_2 x_1 A' n_0 B'_0$$

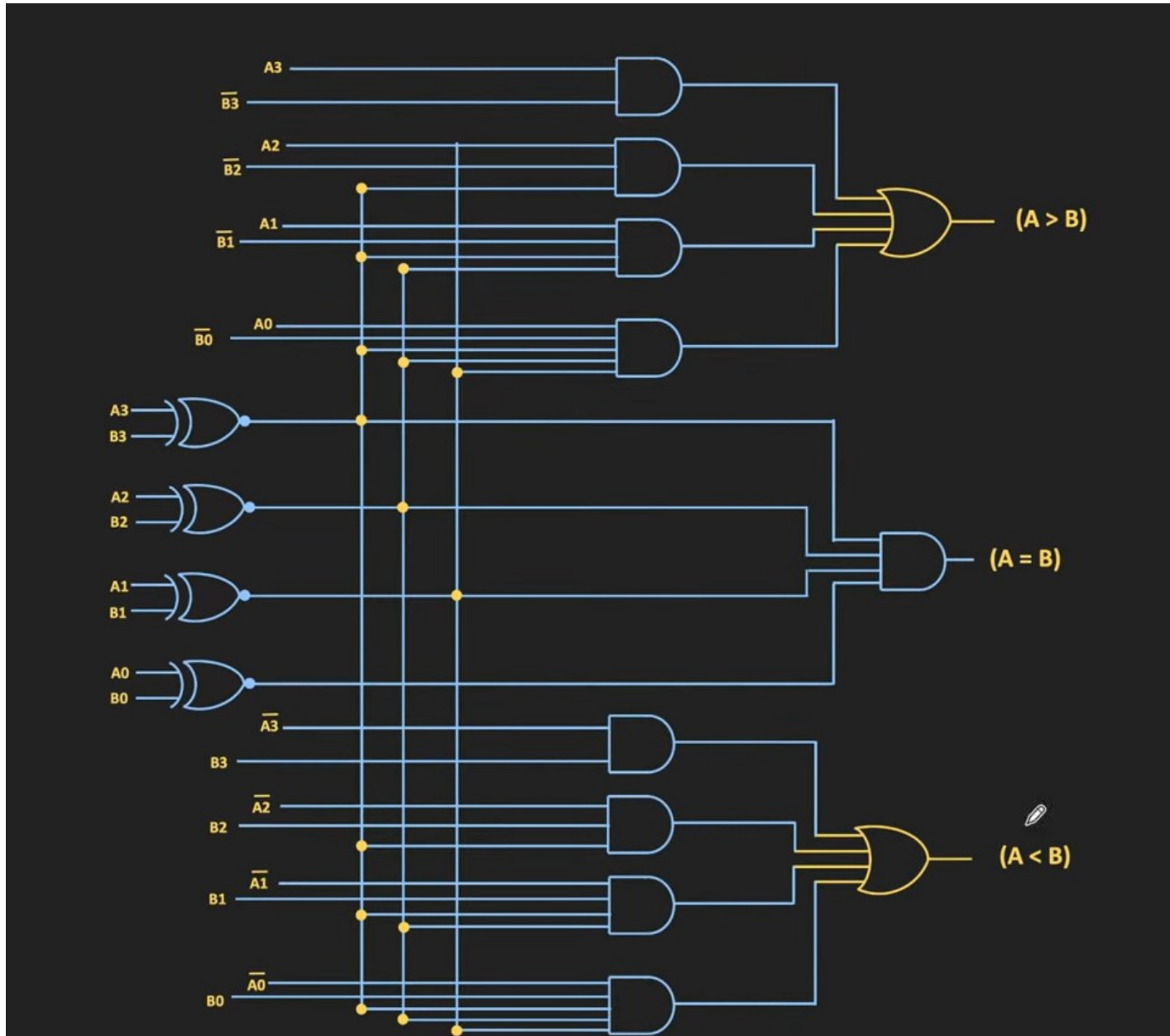
$$A = A_3 A_2 A_1 A_0$$

$$B = B_3 B_2 B_1 B_0$$

$A < B \rightarrow A_3 < B_3$
 OR
 If $A_3 = B_3$ and $A_2 < B_2$
 OR
 If $A_3 = B_3$ and $A_2 = B_2$ and $A_1 < B_1$
 OR
 If $A_3 = B_3$ and $A_2 = B_2$ and $A_1 = B_1$ and $A_0 < B_0$

$$(A < B) = \overline{A_3} B_3 + (A_3 \odot B_3) \overline{A_2} B_2 + (A_3 \odot B_3) (A_2 \odot B_2) \overline{A_1} B_1 + (A_3 \odot B_3) (A_2 \odot B_2) (A_1 \odot B_1) \overline{A_0} B_0$$

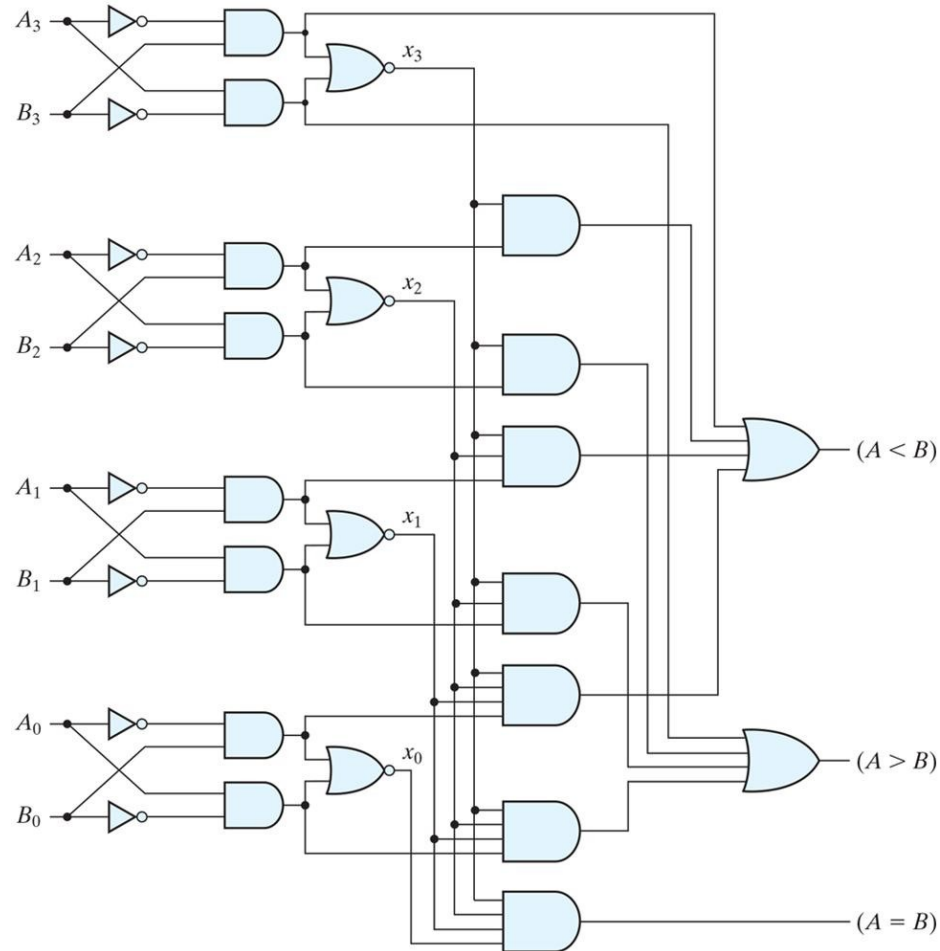
Magnitude Comparator



Combinational logic

Magnitude Comparators

Four-bit Magnitude Comparator



Combinational logic

Magnitude Comparators

- For equality to exist, all x_i variables must be equal to 1, a condition that dictates an AND operation of all variables:
$$(A = B) = x_3 x_2 x_1 x_0$$
- The *binary* variable $(A = B)$ is equal to 1 only if all pairs of digits of the two numbers are equal.
- To determine whether A is greater or less than B , we inspect the relative magnitudes of pairs of significant digits, starting from the most significant position.
- If the two digits of a pair are equal, we compare the next lower significant pair of digits.
- The comparison continues until a pair of unequal digits is reached. If the corresponding digit of A is 1 and that of B is 0, we conclude that $A > B$.
- If the corresponding digit of A is 0 and that of B is 1, we have $A < B$.

Combinational logic

Magnitude Comparators

- The sequential comparison can be expressed logically by the two Boolean functions

$$(A > B) = A_3B'_3 + x_3A_2B'_2 + x_3x_2A_1B'_1 + x_3x_2x_1A_0B'_0$$

$$(A < B) = A'_3B_3 + x_3A'_2B_2 + x_3x_2A'_1B_1 + x_3x_2x_1A'_0B_0$$

- The symbols $(A > B)$ and $(A < B)$ are binary output variables that are equal to 1 when $A > B$ and $A < B$, respectively.
- The unequal outputs can use the same gates that are needed to generate the equal output
- The four x outputs are generated with exclusive-NOR circuits and are applied to an AND gate to give the output binary variable $(A = B)$.
- The other two outputs use the x variables to generate the Boolean functions listed previously. This is a multilevel implementation and has a regular pattern.

A 4-bit by 4-bit binary multiplier is implemented using AND gates and adders. How many AND gates are required?

- A. 16
- B. 8
- C. 32
- D. 64

For comparing two 4-bit numbers A and B, a combinational circuit gives three outputs $A > B$, $A = B$, and $A < B$. If each bit comparison uses XNOR and AND/OR logic, how many XNOR gates are needed?

- A. 4
- B. 3
- C. 2
- D. 1

Combinational logic

Think about it

A 4-bit by 4-bit binary multiplier is implemented using AND gates and adders. How many AND gates are required?

- A. 16
- B. 8
- C. 32
- D. 64

Answer: A. 16

Explanation: $4 \times 4 = 16$ partial products via AND gates.

For comparing two 4-bit numbers A and B, a combinational circuit gives three outputs $A > B$, $A = B$, and $A < B$. If each bit comparison uses XNOR and AND/OR logic, how many XNOR gates are needed?

- A. 4
- B. 3
- C. 2
- D. 1

Answer: A. 4

Explanation: Each bit position requires 1 XNOR gate to check equality.

Combinational logic

Think about it



Describe a real-world application where a magnitude comparator is essential. Explain how the comparator's outputs are used in decision-making.

Describe a real-world application where a magnitude comparator is essential. Explain how the comparator's outputs are used in decision-making.

Ans: **Digital Thermostat in an Air Conditioning (AC) System**

A **digital thermostat** constantly monitors the **room temperature** and compares it with the **desired set temperature**. A **magnitude comparator** is used to perform this comparison.

A: Current room temperature (in binary)

B: Desired set temperature (in binary)

Comparator Outputs:

$A > B \rightarrow$ Room is **warmer** than desired

$A = B \rightarrow$ Room is at **desired temperature**

$A < B \rightarrow$ Room is **colder** than desired

Comparator Output	System Decision
$A > B$	Turn ON cooling (AC compressor starts)
$A = B$	Maintain current state (system remains idle)
$A < B$	Turn OFF cooling or turn ON heating (if available)

Other Examples Where Comparators Are Used:

Speed regulators in vehicles (comparing set vs. actual speed)

Digital counters (checking if max value is reached)

Elevator control systems (comparing current floor with target floor)

Battery chargers (comparing voltage levels)



THANK YOU

Team DDCO

Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Decoders

Team DDCO

Department of Computer Science and Engineering

DIGITAL DESIGN AND COMPUTER ORGANIZATION



Decoders

Department of Computer Science and Engineering

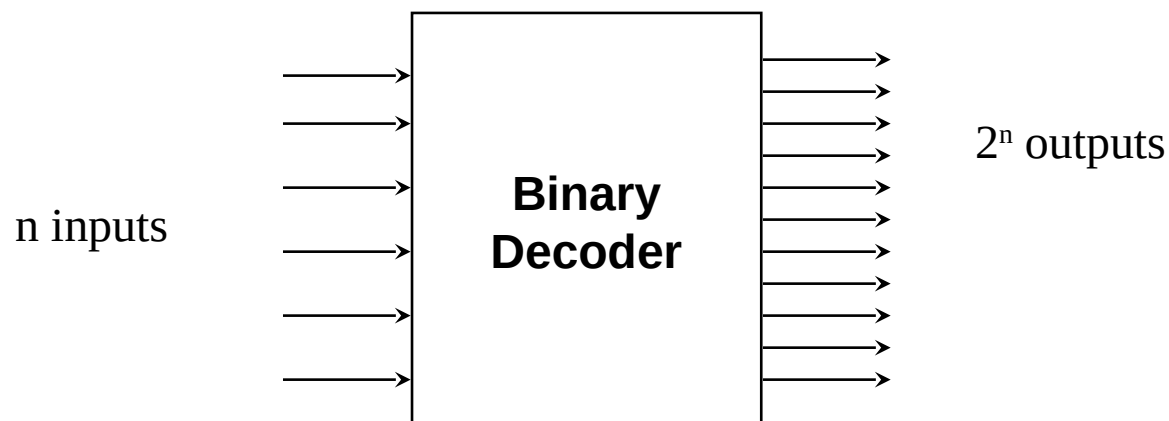
- Discrete quantities of information are represented in digital systems by binary codes.
- A binary code of n bits is capable of representing up to 2^n distinct elements of coded information
- A decoder is a combinational circuit that converts binary information from n input lines to a maximum of 2^n unique output lines. If the n -bit coded information has unused combinations, the decoder may have fewer than 2^n outputs.
- The decoders presented here are called n -to- m -line decoders, where $m \leq 2^n$
- The name decoder is also used in conjunction with other code converters, such as a BCD-to-seven-segment decoder.

Combinational logic

Decoders

Decoders

Black box with n input lines and 2^n output lines



Only one output is a 1 for any given input

Combinational logic

Decoders

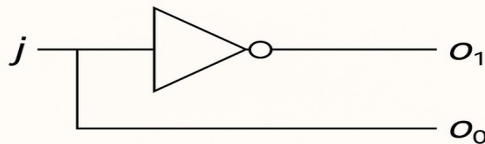
1: 2 Decoder:

A **1-to-2 decoder** is a simple combinational logic circuit that takes **1 input** and produces **2 outputs**. It is used to activate exactly **one of the two outputs** based on the binary value of the input.

Truth table :

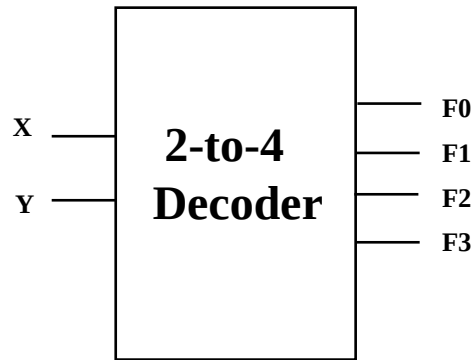
Input (J)	Output(O_1)	Output(O_0)
0	1	0
1	0	1

1:2 decoder logic circuit:



Combinational logic

Decoders

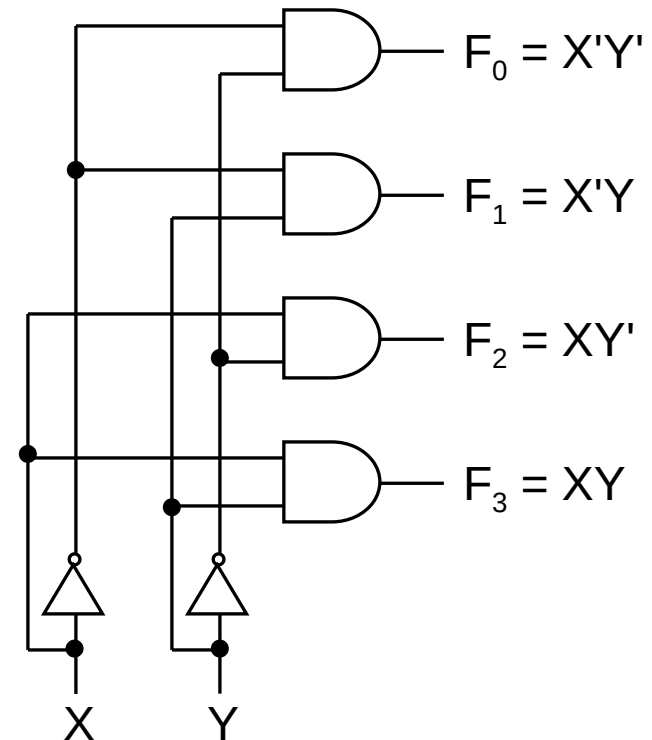


Truth Table:

X	Y	F ₀	F ₁	F ₂	F ₃
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

From truth table, circuit for 2x4 decoder is:

Note: Each output is a 2-variable minterm ($X'Y'$, $X'Y$, XY' or XY)



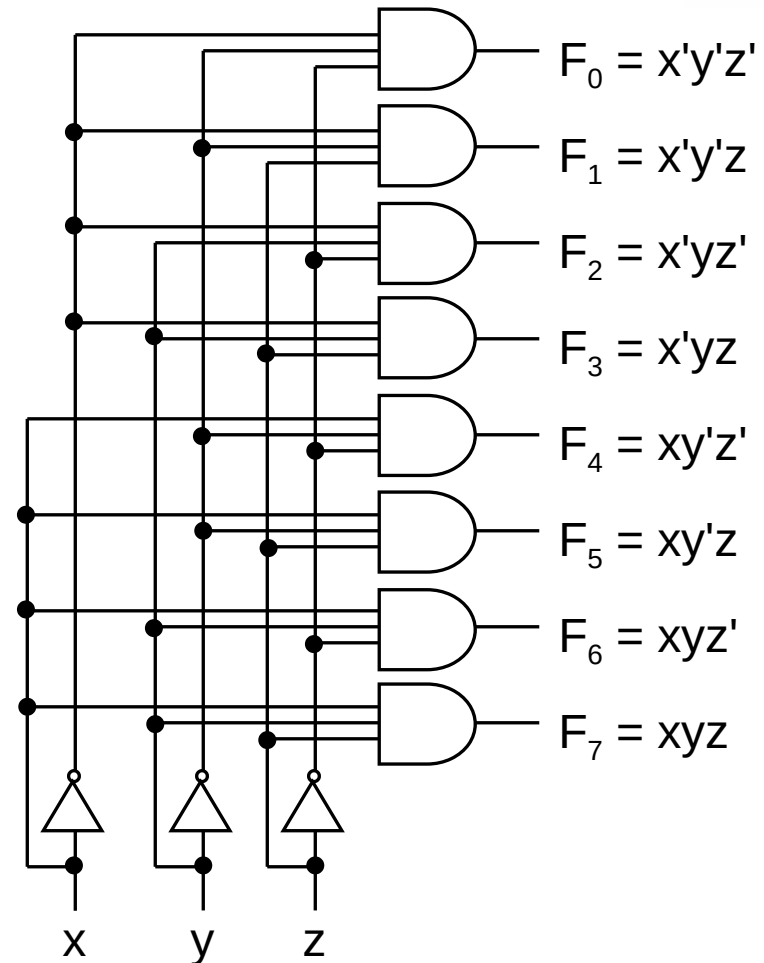
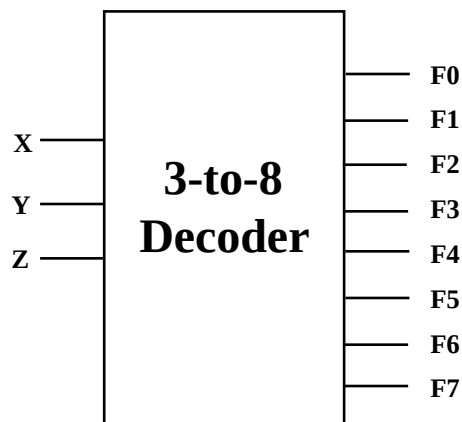
Combinational logic

Decoders (similar to code converter – binary to octal)

3-to-8 Binary Decoder

Truth Table:

x	y	z	F ₀	F ₁	F ₂	F ₃	F ₄	F ₅	F ₆	F ₇
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1



Combinational logic

Decoders with enable input

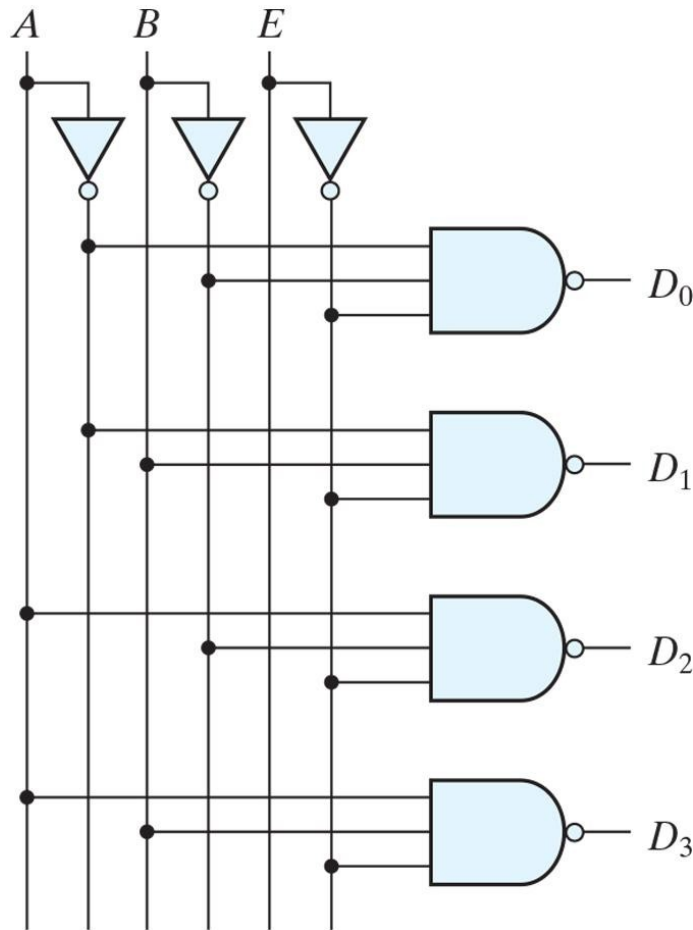
Enable input- allows the decoders outputs to be turned “ON” or “OFF” as required. Output is only generated when the Enable input has value 1; otherwise, all outputs are 0.

Some decoders are constructed with NAND gates. Since a NAND gate produces the AND operation with an inverted output, it becomes more economical to generate the decoder minterms in their complemented form

Combinational logic

Decoders with NAND gates

Two to four line Decoder with Enable Input



Start with a 2-bit decoder
Add an enable signal (E)

Note: use of NANDs
only one 0 active! (**active low**)
if E = 0

AB=10 then output is D2

E	A	B	D ₀	D ₁	D ₂	D ₃
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

Combinational logic

Decoders with enable input

A decoder with enable input can function as a demultiplexer—a circuit that receives information from a single line and directs it to one of 2^n possible output lines. The selection of a specific output is controlled by the bit combination of n selection lines.

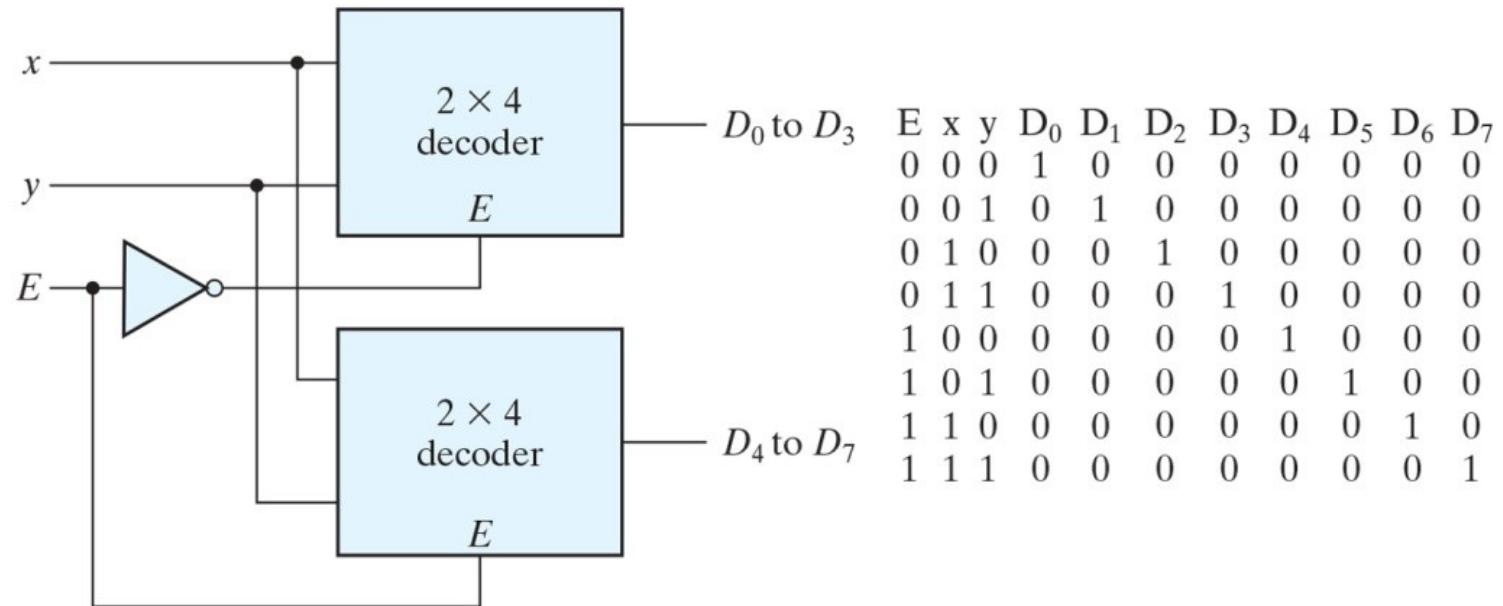
Because decoder and demultiplexer operations are obtained from the same circuit, a decoder with an enable input is referred to as a decoder – demultiplexer

Decoders with enable inputs can be connected together to form a larger decoder

Combinational logic

Decoders

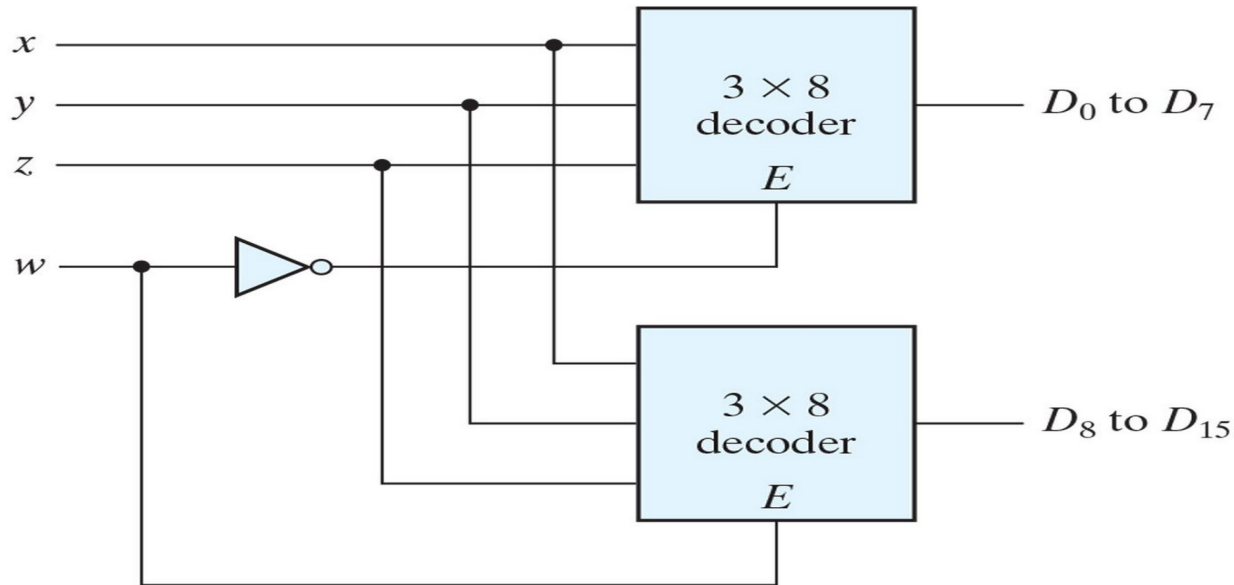
3 x 8 decoder constructed with two 2 x 4 decoders.



Combinational logic

Decoders

4 x 16 decoder constructed with two 3 x 8 decoders.



Enable can also be active high

In this example, only one decoder can be active at a time.

x , y , z effectively select output line for w

Implementing Functions Using Decoders

Any n -variable logic function can be implemented using a single n -to- 2^n decoder to generate the minterms

- **OR gate forms the sum.**
- **The output lines of the decoder corresponding to the minterms of the function are used as inputs to the or gate.**

Any combinational circuit with n inputs and m outputs can be implemented with an n -to- 2^n decoder with m OR gates.

Suitable when a circuit has many outputs, and each output function is expressed with few minterms.

Combinational logic

Decoders-Combinational Logic implementation

A decoder provides the 2^n minterms of n input variables

Each asserted output of the decoder is associated with a unique pattern of input bits.

Since any Boolean function can be expressed in sum-of-minterms form, a decoder that generates the minterms of the function, together with an external OR gate that forms their logical sum, provides a hardware implementation of the function.

In this way, any combinational circuit with n inputs and m outputs can be implemented with an n -to- 2^n -line decoder and m OR gates.

Questions:

1. Build XNOR Gate with 2 inputs using Decoders.
2. Implement full adder using decoders

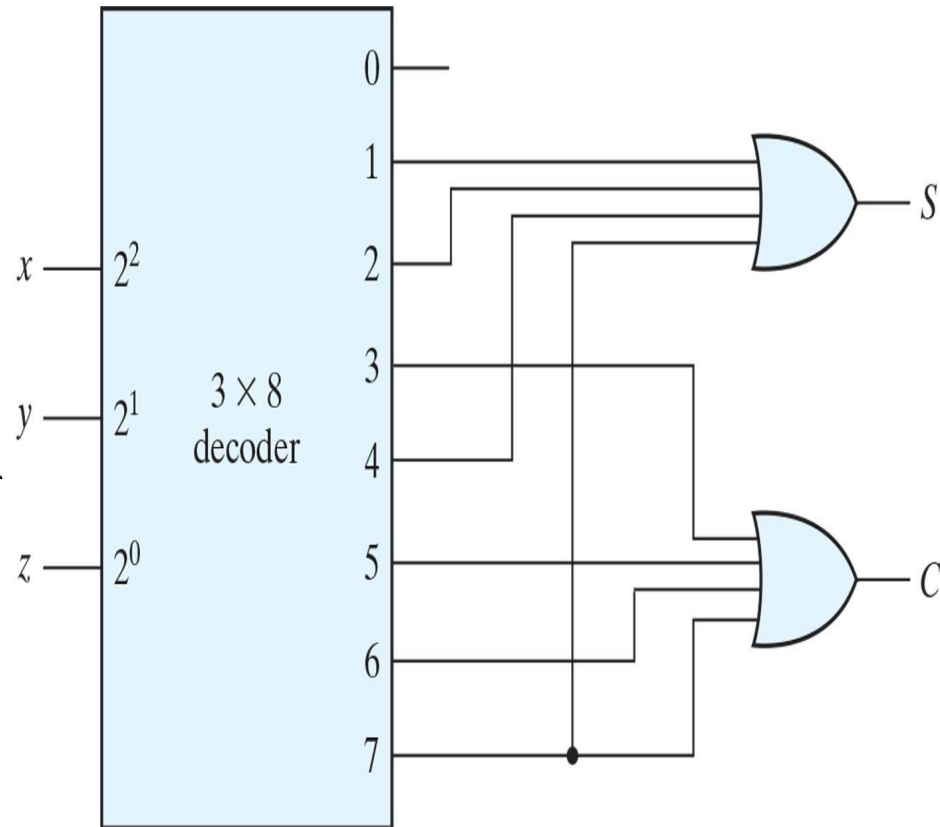
Combinational logic

Decoders-Combinational Logic implementation

A decoder provides the 2^n minterms of n input variables

Implementation of a full adder with a decoder.

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Full adder

$$S(x, y, z) = \sum (1, 2, 4, 7)$$

$$C(x, y, z) = \sum (3, 5, 6, 7)$$

Combinational logic

Decoders



A function with a long list of minterms requires an OR gate with a large number of inputs.

A function having a list of k minterms can be expressed in its complemented form F' with $2^n - k$ minterms.

If the number of minterms in the function is greater than $2^{n/2}$, then F' can be expressed with fewer minterms.

(Example: For 3 variables ($n=3$), total = 8 minterms.

If $F = 6$ minterms, better to write

$$F = (\sum 6 \text{ minterms}) = (\sum \text{ of all } 8 - 6) = \text{complement of 2 minterms}$$

$$F = (\sum 6 \text{ minterms}) = (\sum \text{ of all } 8 - 6) = \text{complement of 2 minterms.}$$

In such a case, it is advantageous to use a NOR gate to sum the minterms of F .

Combinational logic

Decoders-Application

To manage traffic at a **4-way intersection** (North, South, East, West) **efficiently using fewer control lines**, by using a **2-to-4 line decoder**.

Why Use a Decoder?

In a 4-way intersection:

You need to **control each direction's traffic light individually**.

Without a decoder, you'd need **4 separate control signals**, one for each direction.

A **2-to-4 decoder** allows you to **use only 2 input lines** to control 4 outputs, reducing hardware complexity

Combinational logic

Decoders-Application

Understanding the 2-to-4 Decoder

Inputs: 2 bits (say A and B)

Outputs: 4 lines: Y0, Y1, Y2, Y3

Operation: When inputs = 00, output Y0 is active (others inactive)

When inputs = 01, output Y1 is active

When inputs = 10, output Y2 is active

When inputs = 11, output Y3 is active

Decoder Output	Controls Traffic Light at
Y0	North direction
Y1	South direction
Y2	East direction
Y3	West direction

Combinational logic

Decoders-Think about it

1. Build XNOR gate using Decoder
2. A function with more than $2^n/2$ minterms is better implemented using:
 - (A) A single AND gate
 - (B) Complemented function and a NOR gate
 - (C) Only XOR gates
 - (D) Full adders
3. In a 2-to-4 decoder with an active-low enable input, what happens when the enable input is 1?
 - (A) All outputs are enabled
 - (B) Decoder functions normally
 - (C) All outputs are 0
 - (D) All outputs are 1

Combinational logic

Decoders-Think about it

1. Build XNOR gate using Decoder

2. A function with more than $2^n/2$ minterms is better implemented using:

- (A) A single AND gate
- (B) Complemented function and a NOR gate
- (C) Only XOR gates
- (D) Full adders

Answer: (B)

3. In a 2-to-4 decoder with an active-low enable input, what happens when the enable input is 1?

- (A) All outputs are enabled
- (B) Decoder functions normally
- (C) All outputs are 0
- (D) All outputs are 1

Answer: (D)



THANK YOU

Team DDCO

Department of Computer Science

DIGITAL DESIGN AND COMPUTER ORGANIZATION



Encoders

Department of Computer Science and Engineering

Combinational logic

Encoders

Encoders

An encoder is a digital circuit that performs the inverse operation of a decoder. An encoder has 2^n (or fewer) input lines and n output lines. The output lines, as an aggregate, generate the binary code corresponding to the input value.

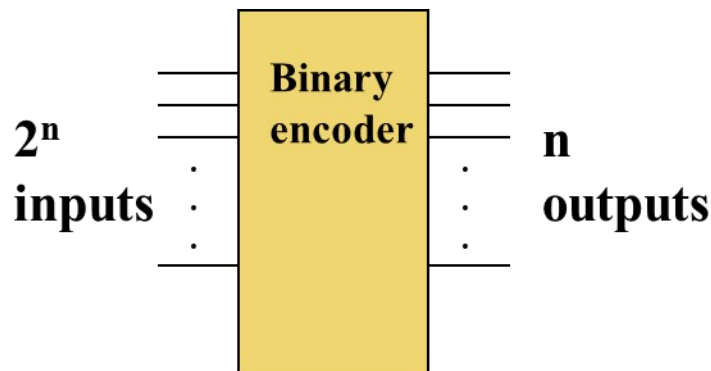
If the a decoder's output code has fewer bits than the input code, the device is usually called an encoder.

e.g. 2^n -to- n

The simplest encoder is a 2^n -to- n binary encoder

One of 2^n inputs = 1

Output is an n -bit binary number



Combinational logic

Encoders

4:2 Encoder

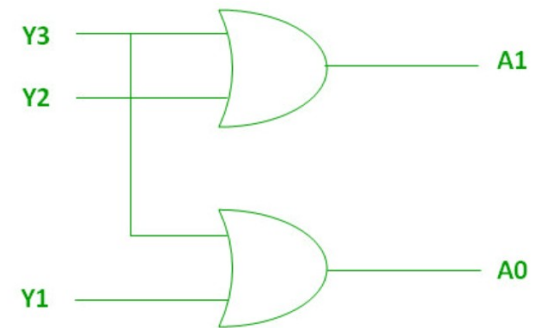


$$A1 = Y3 + Y2$$

$$A0 = Y3 + Y1$$

The Truth table of 4 to 2 encoders is as follows.

INPUTS				OUTPUTS	
Y3	Y2	Y1	Y0	A1	A0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1



Implementation using OR Gate

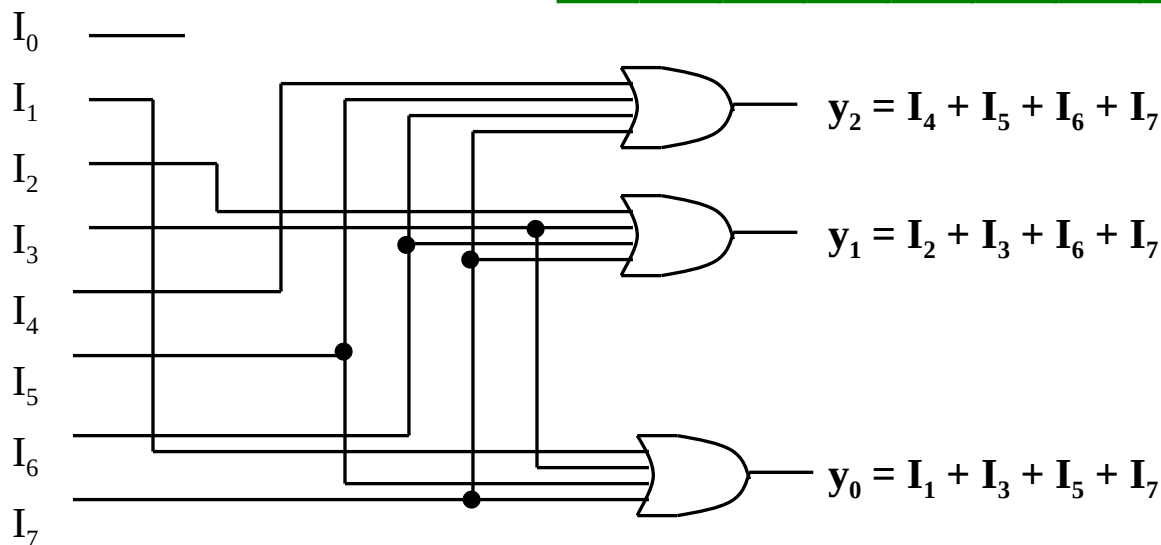
Combinational logic

Encoders (octal to binary)

8-to-3 Binary Encoder

At any one time, only one input line has a value of 1.

Inputs								Outputs		
I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	y_2	y_1	y_0
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1



Combinational logic

Encoders (octal to binary)

Truth Table for Octal-to-Binary Encoder

Truth Table of an Octal-to-Binary Encoder

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

Combinational logic

Priority Encoder

A priority encoder is an encoder circuit that includes the priority function. The operation of the priority encoder is such that if two or more inputs are equal to 1 at the same time, the input having the highest priority will take precedence.

Combinational logic

Priority Encoder

Truth Table of a Priority Encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

In addition to the two outputs x and y , the circuit has a third output designated by V ; this is a valid bit indicator that is set to 1 when one or more inputs are equal to 1. If all inputs are 0, there is no valid input and V is equal to 0. The other two outputs are not inspected when V equals 0 and are specified as don't-care conditions.

higher the subscript number, the higher the priority of the input. Input D_3 has the highest priority, so, regardless of the values of the other inputs, when this input is 1, the output for xy is 11 (binary 3). D_2 has the next priority level. The output is 10 if $D_2=1$, provided that $D_3=0$, regardless of the values of the other two lower priority inputs. The output for D_1 is generated only if higher priority inputs are 0, and so on down the priority levels.

Combinational logic

Priority Encoders

Maps for a Priority Encoder

$$x = D_2 + D_3$$

$$y = D_3 + D_1 D_2'$$

$$V = D_0 + D_1 + D_2 + D_3$$

D_0D_1		D_2D_3			
		00	01	11	10
D_0	00	m_0 X	m_1 1	m_3 1	m_2 1
	01	m_4	m_5 1	m_7 1	m_6 1
	11	m_{12}	m_{13} 1	m_{15} 1	m_{14} 1
	10	m_8	m_9 1	m_{11} 1	m_{10} X

$x = D_2 + D_3$

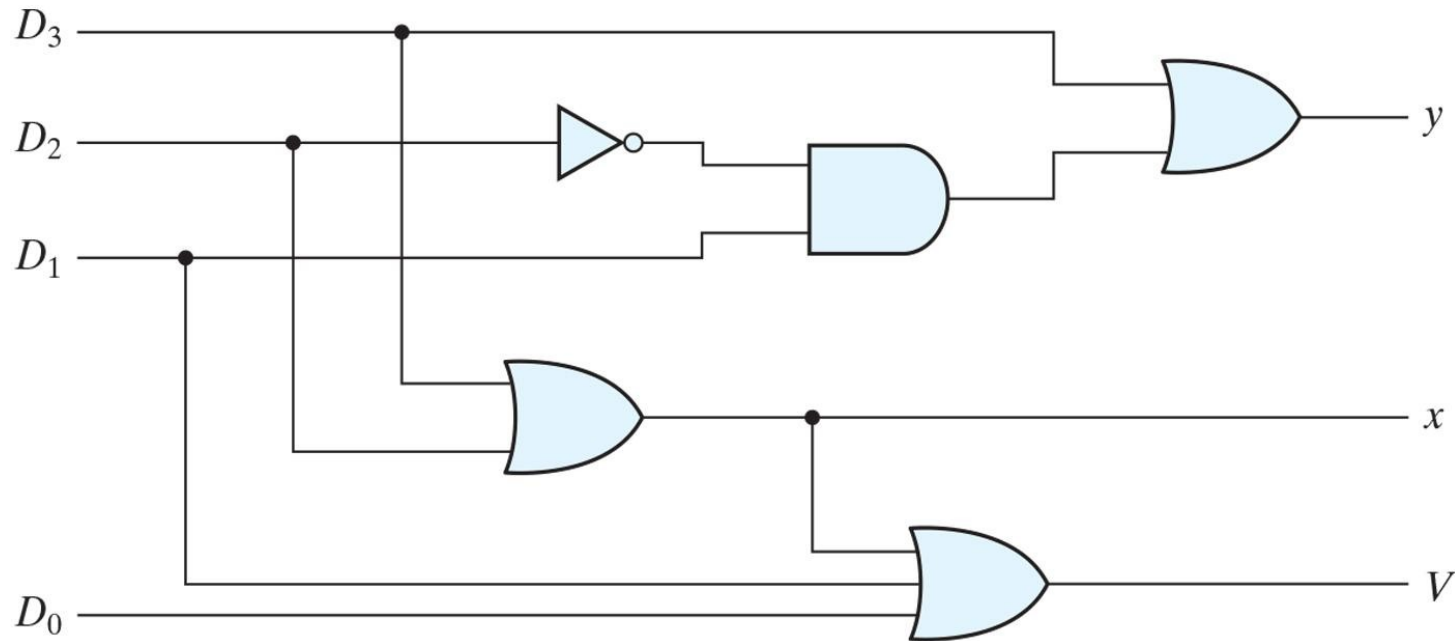
D_0D_1		D_2D_3			
		00	01	11	10
D_0	00	m_0 X	m_1 1	m_3 1	m_2
	01	m_4 1	m_5 1	m_7 1	m_6
	11	m_{12} 1	m_{13} 1	m_{15} 1	m_{14}
	10	m_8	m_9 1	m_{11} 1	m_{10}

$y = D_3 + D_1 D_2'$

Combinational logic

Priority Encoders

Four input Priority Encoder



Combinational logic

Priority Encoders

•8-to-3 Priority Encoder

- What if more than one input line has a value of 1?
- Ignore “lower priority” inputs.
- Idle indicates that no input is a 1.
- Note that polarity of Idle is opposite from Table 4-8 in Mano

Inputs								Outputs			Idle
I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇	y ₂	y ₁	y ₀	
0	0	0	0	0	0	0	0	x	x	x	1
1	0	0	0	0	0	0	0	0	0	0	0
X	1	0	0	0	0	0	0	0	0	1	0
X	X	1	0	0	0	0	0	0	1	0	0
X	X	X	1	0	0	0	0	0	1	1	0
X	X	X	X	1	0	0	0	1	0	0	0
X	X	X	X	X	1	0	0	1	0	1	0
X	X	X	X	X	X	1	0	1	1	0	0
X	X	X	X	X	X	X	1	1	1	1	0

Combinational logic

Encoders

Priority Encoder (8 to 3 encoder)

Assign priorities to the inputs

When more than one input are asserted, the output generates the code of the input with the highest priority

Priority Encoder :

$H7 = I7$ (Highest Priority)

$H6 = I6 \cdot I7'$

$H5 = I5 \cdot I6' \cdot I7'$

$H4 = I4 \cdot I5' \cdot I6' \cdot I7'$

$H3 = I3 \cdot I4' \cdot I5' \cdot I6' \cdot I7'$

$H2 = I2 \cdot I3' \cdot I4' \cdot I5' \cdot I6' \cdot I7'$

$H1 = I1 \cdot I2' \cdot I3' \cdot I4' \cdot I5' \cdot I6' \cdot I7'$

$H0 = I0 \cdot I1' \cdot I2' \cdot I3' \cdot I4' \cdot I5' \cdot I6' \cdot I7'$

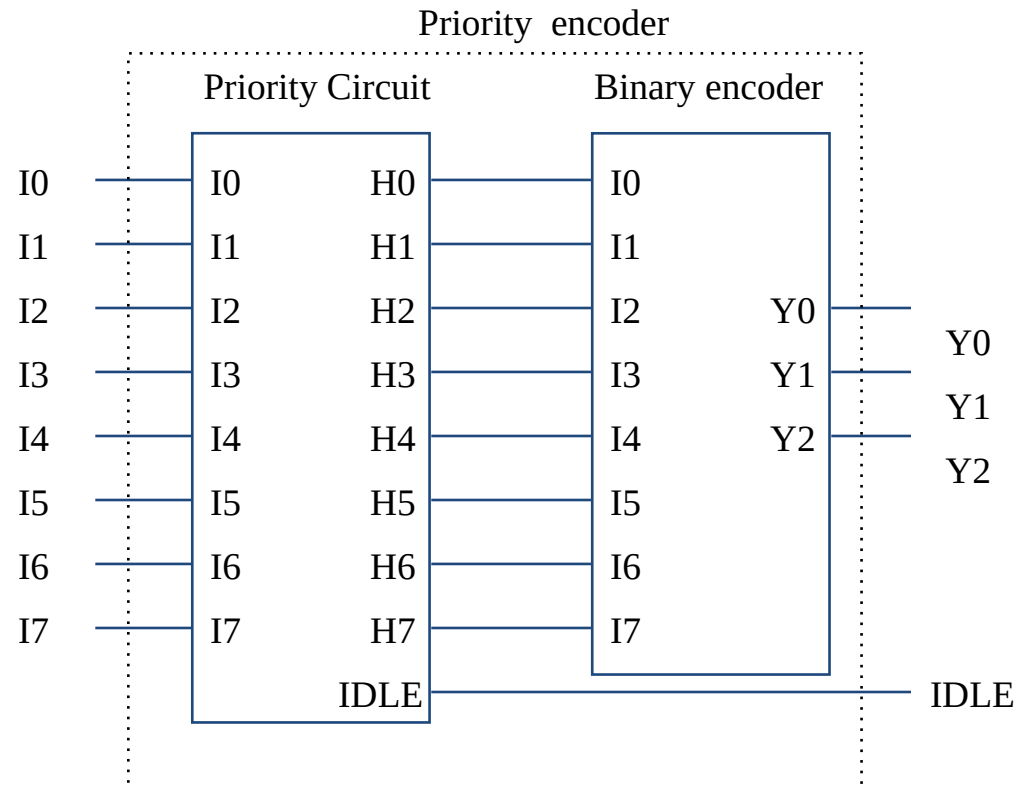
$IDLE = I0' \cdot I1' \cdot I2' \cdot I3' \cdot I4' \cdot I5' \cdot I6' \cdot I7'$

Encoder

$Y0 = I1 + I3 + I5 + I7$

$Y1 = I2 + I3 + I6 + I7$

$Y2 = I4 + I5 + I6 + I7$

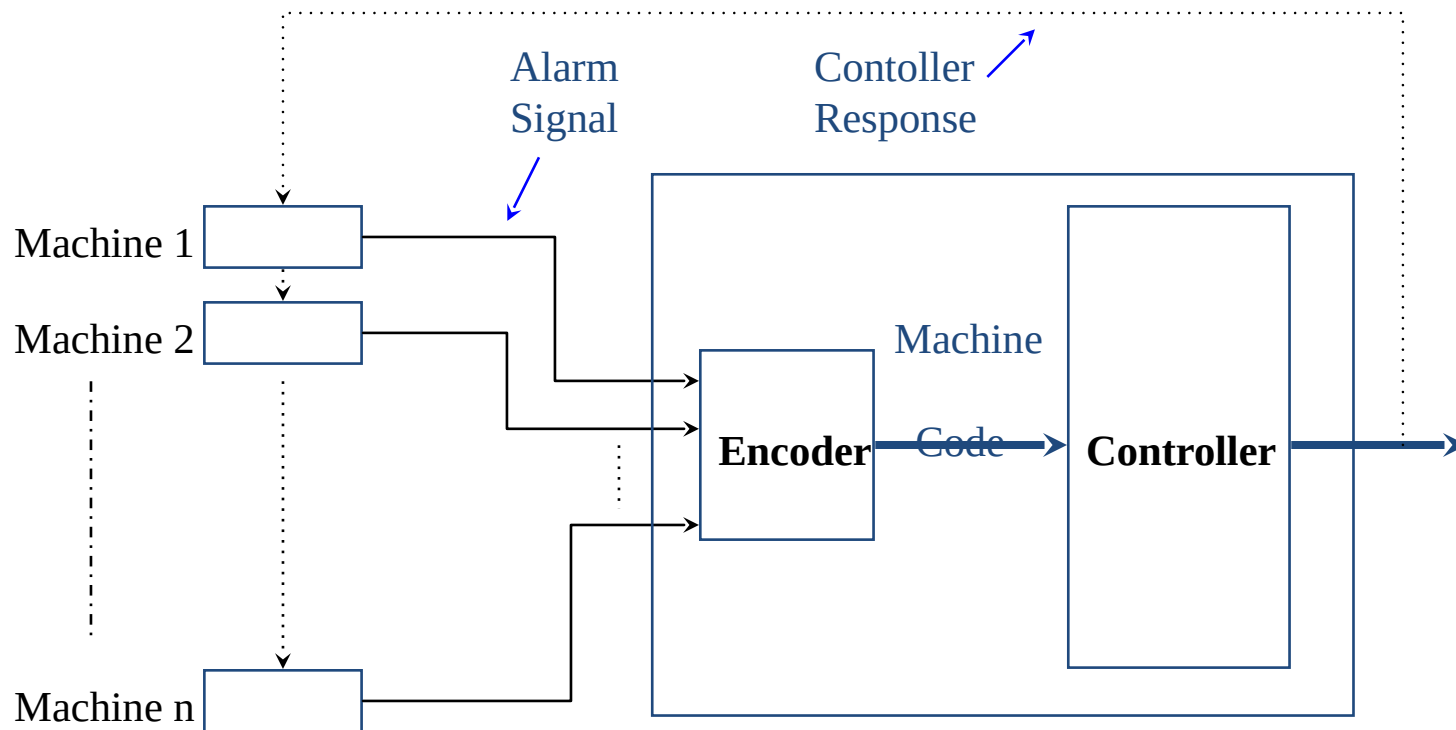


Combinational logic

Encoders

Encoder Application (Monitoring Unit)

Encoder identifies the requester and encodes the value
Controller accepts digital inputs.



Combinational logic

Encoders



Machines (Machine 1, Machine 2, ... Machine n):

- These are the different machines connected to the encoder. Each machine sends data or a request to the encoder.

Encoder:

- **The encoder's primary role is to identify which machine is sending the request and then encode the value or information from that machine into a machine code.**
- It also generates an alarm signal when needed, possibly indicating an issue or a specific condition that requires attention.

controller:

- The controller receives the encoded machine code from the encoder.
- Based on the received machine code, the **controller performs specific actions or responses, such as sending signals back to the machines.**

Alarm Signal:

- **The alarm signal seems to be an alert generated by the encoder if a certain condition is met.** This could indicate a **malfunction**, a process completion, or any other situation that needs immediate attention.

Controller Response:

- After processing the machine code, **the controller might send a response back to the machine or trigger other actions as needed.**

Applications:

Encoders- Application: In digital systems like computers, a keyboard encoder converts the pressing of keys into binary codes that the computer's processor can understand and act upon.

Priority encoders are commonly used in interrupt systems in microprocessors, where multiple interrupt signals may be received at the same time, and the processor needs to know which one to service first based on priority.

Decoders- In memory systems, a decoder takes the binary address provided by the CPU and activates the corresponding memory cell, allowing data to be read from or written to that specific location. For example, a 3-to-8 decoder can select one of eight memory locations based on a 3-bit address input.

Combinational logic

Encoders- think about it

Which of the following best describes the functionality of a **priority encoder**?

- A. It encodes the input having the lowest subscript among all active inputs.
- B. It encodes only the first input line, irrespective of other inputs.
- C. It encodes the input having the highest subscript (priority) among active inputs.
- D. It generates output only when a single input is active.

In a **4-to-2 priority encoder**, the output is “11” when:

- A. Only D0 is 1
- B. D1 and D2 are 1
- C. D3 is 1
- D. D3 is 0 and others are 1

Combinational logic

Encoders- think about it

Which of the following best describes the functionality of a **priority encoder**?

- A. It encodes the input having the lowest subscript among all active inputs.
- B. It encodes only the first input line, irrespective of other inputs.
- C. It encodes the input having the highest subscript (priority) among active inputs.
- D. It generates output only when a single input is active.

Answer: C

Explanation: In a priority encoder, if multiple inputs are active, the one with the **highest priority (usually highest subscript)** is considered.

In a **4-to-2 priority encoder**, the output is “11” when:

- A. Only D0 is 1
- B. D1 and D2 are 1
- C. D3 is 1
- D. D3 is 0 and others are 1

Answer: C

Explanation: D3 has highest priority.



THANK YOU

Team DDCO

Department of Computer Science



DIGITAL DESIGN AND COMPUTER ORGANIZATION

Multiplexers

Team DDCO

Department of Computer Science and Engineering

Combinational logic

Multiplexers

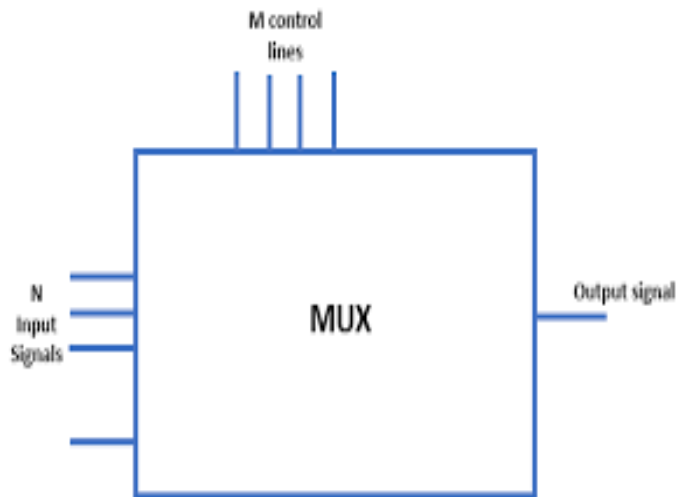
A multiplexer (also called a mux) *multiplexes* many inputs onto a single output

Data selector

Many to one

N:1 MUX

2^N inputs n select lines , one output



www.educba.com



Combinational logic

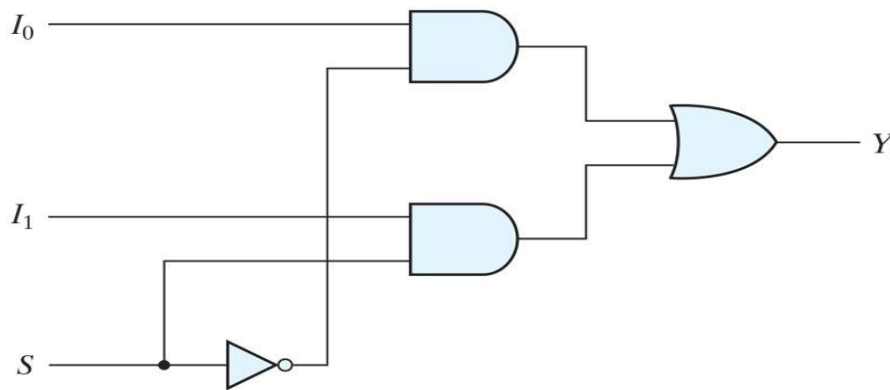
Multiplexers

A multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line. The selection of a particular input line is controlled by a set of selection lines. Normally, there are 2^n input lines and n selection lines whose bit combinations determine which input is selected.

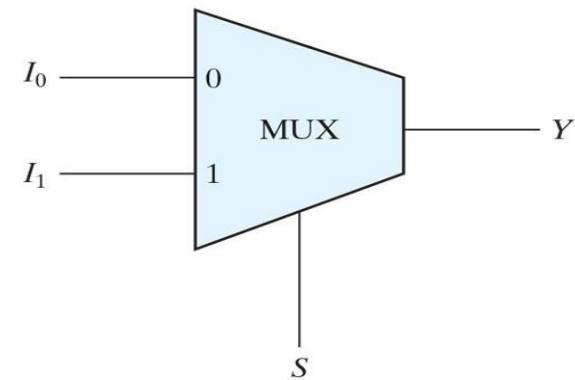
Combinational logic

Multiplexers

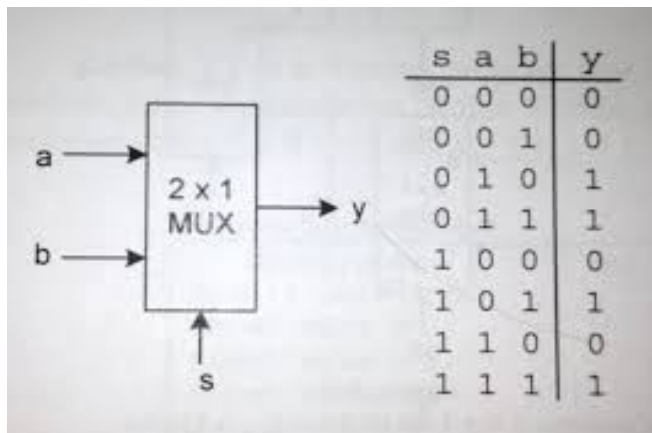
Two to One Line Multiplexer



(a) Logic diagram



(b) Block diagram

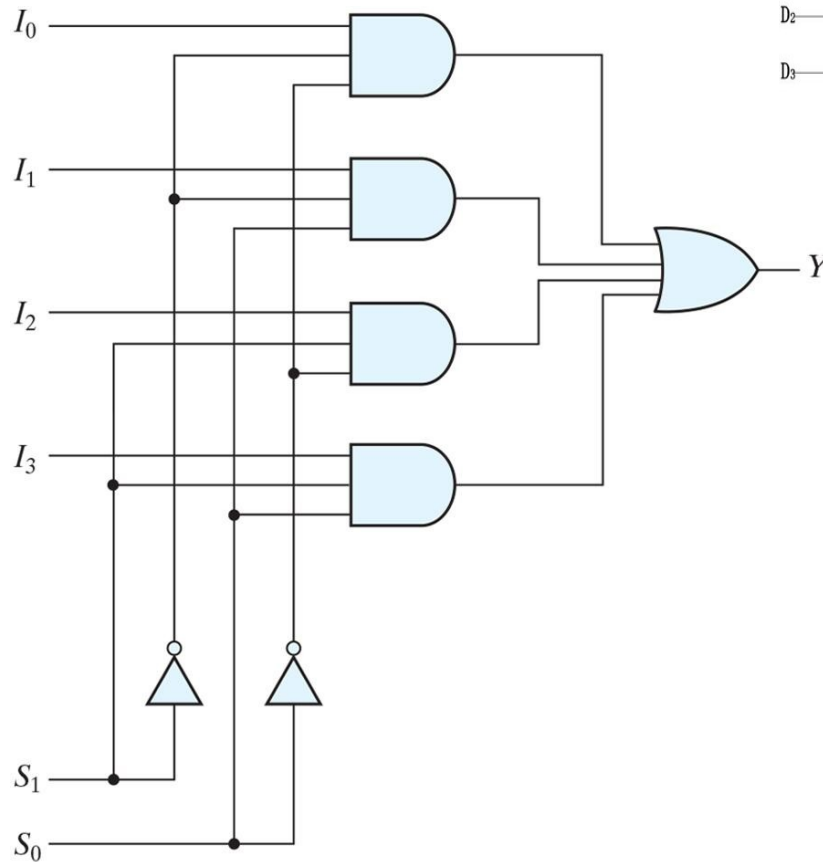


Combinational logic

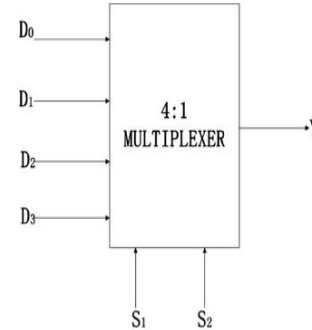
Multiplexers



Four to One Line Multiplexer



(a) Logic diagram



(a)

INPUT		OUTPUT
S_2	S_1	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

(b)

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

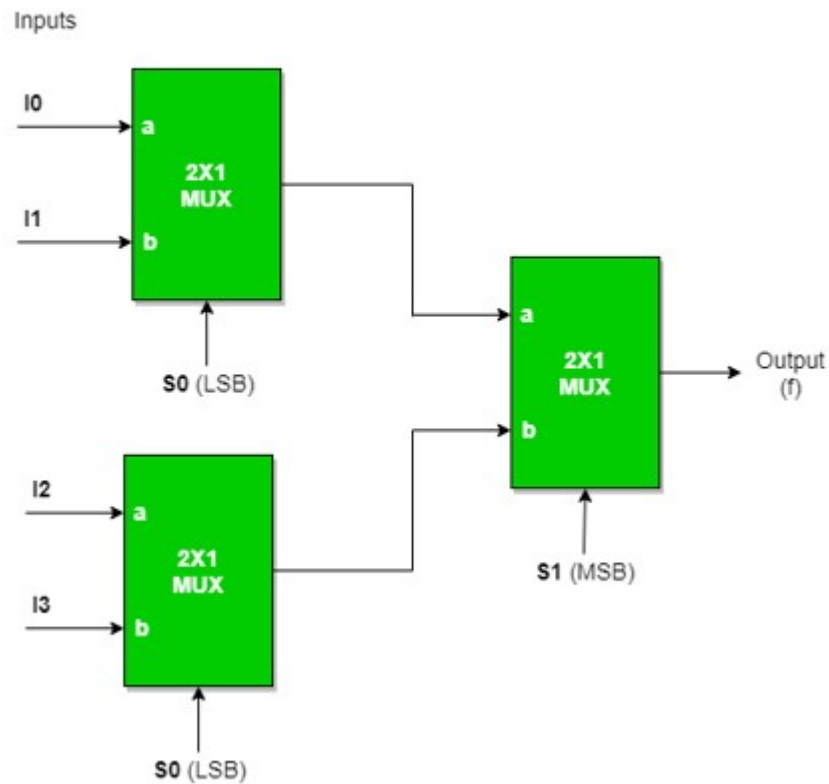
(b) Function table

Combinational logic

Multiplexers

Implementing 4:1 MUX using 2:1 MUX

Implementing 4:1 MUX using 2:1 MUX



Truth Table

S_1	S_0	f
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Combinational logic

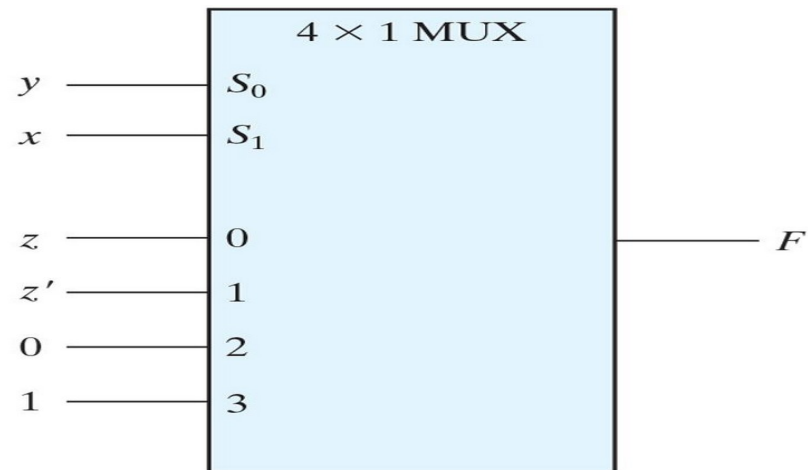
Multiplexers-Boolean Function Implementation

Implementing a Boolean Function with a Multiplexer

$$F(x, y, z) = (1, 2, 6, 7)$$

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

(a) Truth table



(b) Multiplexer implementation

Connect input variables to select inputs of multiplexer ($n-1$ for n variables)

Set data inputs to multiplexer equal to values of function for corresponding assignment of select variables

Using a variable at data inputs reduces size of the multiplexer

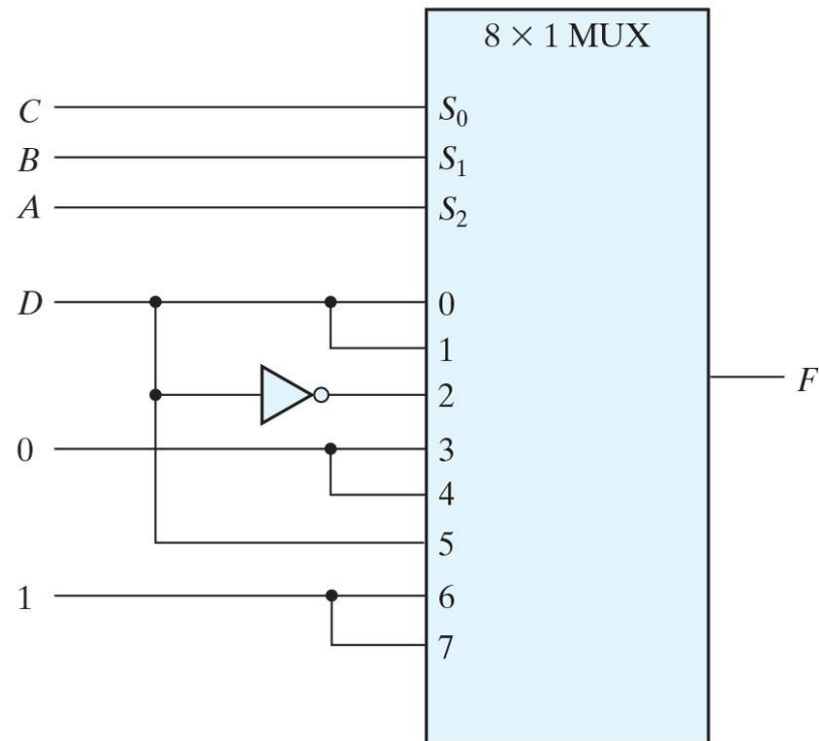
Combinational logic

Multiplexers

Implementing a four input Function with a 8X1 Multiplexer

$$F(A, B, C, D) = (1, 3, 4, 11, 12, 13, 14, 15)$$

A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	



Combinational logic

Multiplexers- Three State Gates

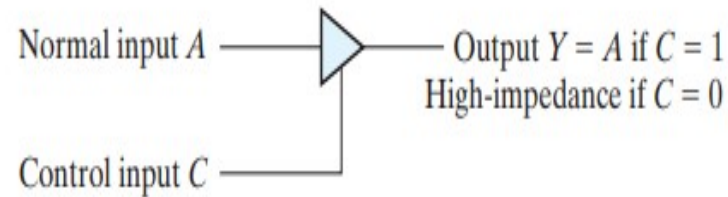


FIGURE 4.29

Graphic symbol for a three-state buffer

Two of the states are signals equivalent to logic 1 and logic 0 as in a conventional gate. The third state is a high-impedance state in which (1) the logic behaves like an **open circuit**, which means that the output appears to be disconnected, (2) the circuit has **no logic significance**, and (3) the circuit connected to the **output** of the **three-state gate** is not affected by the inputs to the gate.

Combinational logic

Multiplexers- Three State Gates

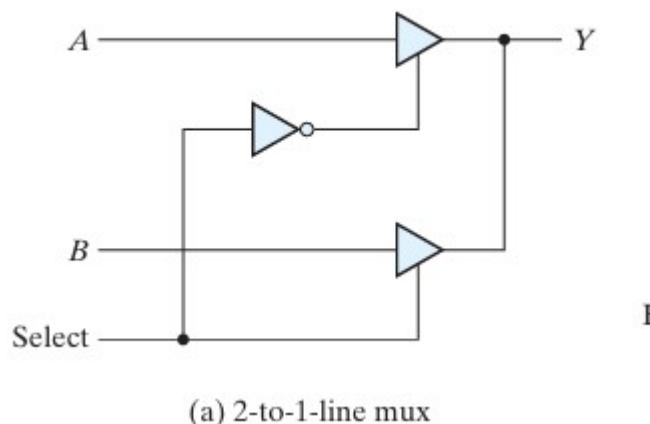


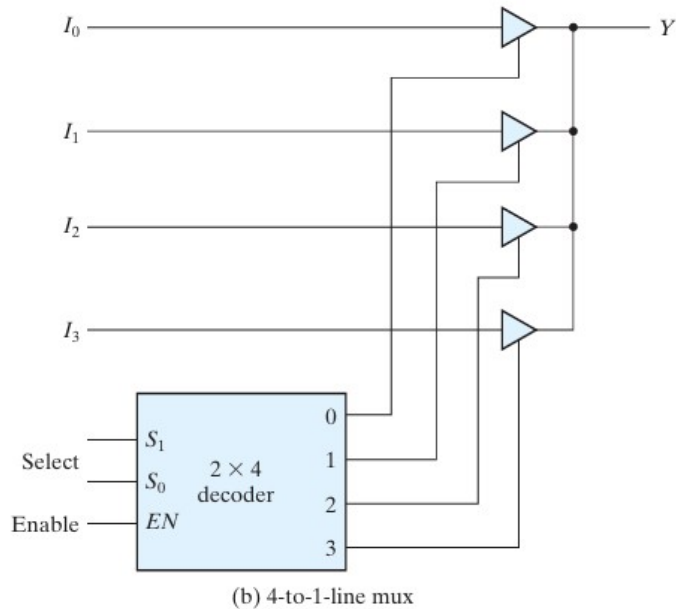
FIGURE 4.30
Multiplexers with three-state gates

When the select input is 0, the upper buffer is enabled by its control input and the lower buffer is disabled. Output Y is then equal to input A. When the select input is 1, the lower buffer is enabled and Y is equal to B.

The construction of multiplexers with three-state buffers is demonstrated in Fig. 4.30. Figure 4.30(a) shows the construction of a two-to-one-line multiplexer with 2 three-state buffers and an inverter. The two outputs are connected together to form a single output line.

Combinational logic

Multiplexers- Three State Gates



No more than one buffer may be in the active state at any given time.

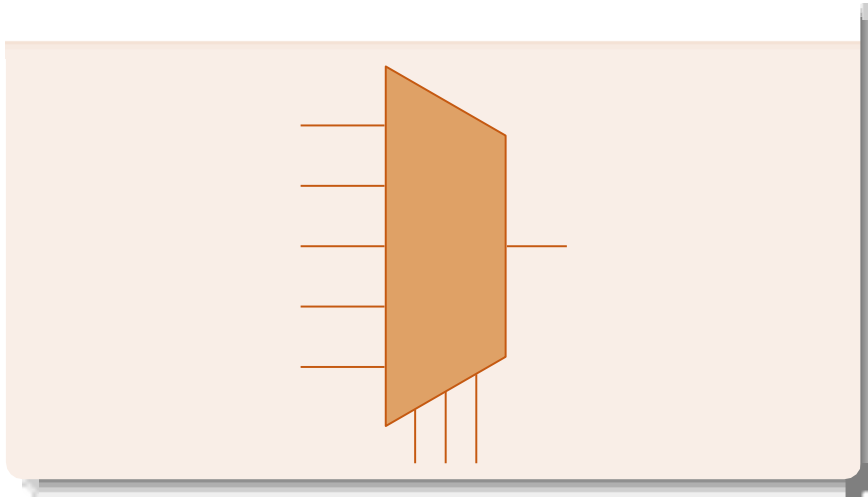
One way to ensure that no more than one control input is active at any given time is to use a decoder, as shown in the diagram

When the **enable** input of the decoder is 0, all of its four outputs are 0 and the **bus line** is in a **high-impedance state** because all four buffers are disabled. When the enable input is active, one of the three state buffers will be active, depending on the binary value in the select inputs of the decoder.

Combinational logic

Multiplexers- Think about it

5:1 Mux



A combinational logic circuit having n data inputs, $\lceil \log_2 n \rceil$ control inputs and one output, that connects the data input indicated by the control inputs to the output

- What is the Boolean formula for a 3:1 mux? Construct a 3:1 mux using 2:1 muxes AND, OR and NOT gates

Combinational logic

Multiplexers- Think about it



Using 2 X 1 MUX design $y=AB$

Using 4 X 1 MUX design $y=AB' + B'C' + A'BC$

Combinational logic

Multiplexers-

A 4-to-1 multiplexer is used to implement the Boolean function

$$F(A,B,C)=\sum m(1,3,5,7)$$

Which variables should be connected to the select lines of the MUX?

- A) A and B
- B) B and C
- C) A and C
- D) Any two variables

How many select lines are required for a **5:1 multiplexer**?

- A) 2
- B) 3
- C) 4
- D) 5

Combinational logic

Multiplexers-

A 4-to-1 multiplexer is used to implement the Boolean function

$$F(A,B,C)=\sum m(1,3,5,7)$$

Which variables should be connected to the select lines of the MUX?

- A) A and B
- B) B and C
- C) A and C
- D) Any two variables

Answer: D

. How many select lines are required for a **5:1 multiplexer**?

- A) 2
- B) 3
- C) 4
- D) 5

Answer: B) 3

Combinational logic

Multiplexers-

The Boolean function $F(x,y,z)=\sum m(1,2,6,7)$ is implemented using an 8:1 multiplexer. What should be the values of data inputs D_0 to D_7 ?

- A) $D = \{1,1,1,0,0,0,1,1\}$
- B) $D = \{0,1,0,1,0,1,0,1\}$
- C) $D = \{1,0,0,1,1,1,0,0\}$
- D) $D = \{0,1,1,0,0,0,1,1\}$

Combinational logic

Multiplexers-

The Boolean function $F(x,y,z)=\sum m(1,2,6,7)$ is implemented using an 8:1 multiplexer. What should be the values of data inputs D_0 to D_7 ?

A) $D = \{1,1,1,0,0,0,1,1\}$

B) $D = \{0,1,0,1,0,1,0,1\}$

C) $D = \{1,0,0,1,1,1,0,0\}$

D) $D = \{0,1,1,0,0,0,1,1\}$

Answer: D) $D = \{0,1,1,0,0,0,1,1\}$

Explanation:

The minterms correspond to the positions where $F = 1$.

So, $D_1 = D_2 = D_6 = D_7 = 1$, rest are 0.



THANK YOU

Team DDCO

Department of Computer Science

Feature	Decoder	Encoder	Multiplexer (MUX)
Basic Function	Converts binary input code into a one-hot output (activates exactly one output line).	Converts one active input line into a binary output code .	Selects one input line out of many and forwards it to a single output .
Mapping	n inputs → 2ⁿ outputs	2ⁿ inputs → n outputs	n data inputs → 1 output (with select lines)
Direction of Selection	Input → One active output	Active input → Binary code output	Select lines → Choose one input → Single output
Example (4 lines)	2-to-4 Decoder: 2-bit input activates 1 of 4 outputs.	4-to-2 Encoder: 1 active input produces 2-bit output code.	4-to-1 MUX: 4 inputs, 2 select lines, 1 output.
Typical Use Case	Memory addressing, selecting devices (chip select).	Keyboards (key press → binary code), priority encoding.	Routing data signals, using one ADC for many sensors.
Output Lines	Multiple outputs (one active at a time).	Fewer outputs (binary code).	Single output only.
Control Inputs	Input lines directly determine output.	Only one input should be active at a time (or needs priority logic).	Needs select lines to choose which input to pass.
One-to-Many vs Many-to-One	One-to-Many (decode a binary code).	Many-to-One (coded form) .	Many-to-One (data selection) .

--